AWS:

Security:

1. What is IAM roles and what is advantage of using it. IAM roles vs IAM users vs IAM groups.

   IAM Roles are a way to grant permissions to AWS services, applications, or users without requiring permanent credentials. Unlike IAM users, roles provide temporary security credentials and can be assumed by AWS services (e.g., EC2, Lambda) or external users.

   Advantages of IAM Roles:

   1. Temporary Credentials – Reduces security risks compared to static credentials.
   2. Cross-Account Access – Allows secure access between AWS accounts.
   3. Service Access – Enables AWS services (e.g., EC2, Lambda) to access other AWS resources securely.
   4. Least Privilege Access – Grants only required permissions.
   5. No Credential Management – Eliminates the need for hardcoded passwords or access keys.

IAM Roles vs IAM Users vs IAM Groups

Feature	IAM Roles	IAM Users	IAM Groups
Definition	Temporary identity with permissions.	Permanent identity with credentials.	Collection of users with shared permissions.
Use Case	AWS services or external entities needing access.	Individuals needing AWS access.	Managing permissions for multiple users.
Credentials	Temporary (via STS).	Permanent (Access Key/Password).	No credentials; users inherit permissions.
Security	More secure, no static keys.	Less secure due to long-term credentials.	Permissions are managed centrally.

When to Use What?

- IAM Roles → For AWS services (EC2, Lambda) or cross-account access.
- IAM Users → For individual users needing AWS access.
- IAM Groups → To assign the same permissions to multiple users efficiently.

2. What is MFA why do we use it?

   Multi-Factor Authentication (MFA) is a security mechanism that requires users to provide **two or more authentication factors** before gaining access to AWS resources. It adds an extra layer of protection beyond just a username and password.

   Why Do We Use MFA?

   1. **Enhanced Security** – Even if a password is compromised, an attacker cannot access the account without the second factor.
   2. **Protection Against Unauthorized Access** – Prevents unauthorized logins even if credentials are leaked.
   3. **Compliance and Best Practices** – Helps meet security standards like ISO 27001, SOC, and PCI DSS.
   4. **Mitigates Phishing and Password Attacks** – Reduces risks from stolen or weak passwords.

   Types of MFA in AWS:

   1. **Virtual MFA** – Apps like Google Authenticator, Authy (generates time-based OTPs).
   2. **Hardware MFA** – Physical security keys like YubiKey.
   3. **SMS MFA** – One-time password (OTP) sent via SMS.
   4. **FIDO Security Keys** – USB-based authentication devices.

   When to Use MFA?

   - AWS root user (Highly recommended).
   - IAM users with sensitive permissions.
   - Privileged accounts accessing production environments.

3. Difference between Inline policy and custom policy and its uses.

Feature	Inline Policy	Custom Policy (Managed Policy)
Definition	A policy directly attached to a single IAM user, group, or role.	A reusable policy created separately and attached to multiple IAM users, groups, or roles.
Reusability	Cannot be reused; specific to one entity.	Can be attached to multiple IAM identities.
Management	Hard to manage and track as it is embedded.	Easier to manage, update, and track.
Use Case	When a policy is specific to a single IAM identity and shouldn't be shared.	When multiple IAM identities need the same permissions.
Best Practice	Not recommended unless policy is highly specific.	Recommended for scalability and better access management.

4. What are IAM policies? How have you used it?

   IAM **Policies** are JSON-based documents that define permissions for IAM identities (users, groups, roles) or AWS resources. They specify **who** can perform **what actions** on **which resources** under **what conditions**.

   Types of IAM Policies:

   1. **AWS Managed Policies** – Predefined by AWS (e.g., AmazonS3FullAccess).
   2. **Customer Managed Policies** – Custom policies created by users for reuse.
   3. **Inline Policies** – Directly attached to a single IAM user, group, or role (not reusable).
   4. **Service Control Policies (SCPs)** – Used in AWS Organizations to enforce permissions across multiple accounts.
   5. **Permissions Boundaries** – Limits the maximum permissions a user or role can have.

   How Have I Used IAM Policies?

   - **Least Privilege Access** – Created **customer-managed policies** to grant only necessary permissions (e.g., EC2 read access, S3 write access).
   - **CI/CD Pipeline Security** – Attached **IAM roles with policies** to allow Jenkins pipelines to access S3, ECR, and deploy applications to Kubernetes.
   - **Security & Compliance** – Used **AWS Managed Policies** for enforcing security best practices (e.g., ReadOnlyAccess for auditors).
   - **Cross-Account Access** – Created **IAM roles with policies** to allow limited access between AWS accounts.

Compute:

1. From where and how Autoscaling should know to increase or decrease the resources?

   AWS Auto Scaling monitors predefined metrics and **automatically scales resources** based on configured **scaling policies**. It determines when to increase or decrease instances based on:

   1. Scaling Policies:

   Auto Scaling decisions are based on **Scaling Policies** that define conditions to adjust resources. Common types:

   - **Target Tracking Policy** – Maintains a specific metric (e.g., CPU at 50%).
   - **Step Scaling Policy** – Adds/removes instances in steps based on metric thresholds.
   - **Scheduled Scaling** – Increases/decreases resources at fixed times.

   2. Metrics Monitored by Auto Scaling:

   Auto Scaling uses **Amazon CloudWatch** metrics, such as:

   - **CPU Utilization** – If CPU > 80%, scale out; if < 30%, scale in.
   - **Memory Usage** – If high, launch new instances.
   - **Request Count per Target (ALB)** – Adjust instances based on incoming traffic.
   - **Custom Metrics** – Defined using CloudWatch (e.g., queue length in SQS).

   3. Scaling Triggers (Auto Scaling Signals):

   - **CloudWatch Alarms** – Trigger actions when thresholds are breached.
   - **Health Checks** – If an instance becomes unhealthy, Auto Scaling replaces it.
   - **Elastic Load Balancer (ELB) Requests** – Adjusts resources based on traffic.

Example: How It Works in Practice

1. CloudWatch monitors CPU utilization of EC2 instances in an Auto Scaling Group.
2. If CPU > 75% for 5 minutes, a scaling policy triggers and adds more instances.
3. If CPU < 30%, another policy removes instances to save cost.

2. Have you worked on auto scaling? Why do we need them? How do you work with it explain steps.

Yes, I have worked on **AWS Auto Scaling** to ensure high availability, cost optimization, and automatic resource management for applications. I have configured **EC2 Auto Scaling Groups (ASG)**, **Application Auto Scaling (for ECS, RDS, DynamoDB)**, and used **CloudWatch alarms** to dynamically scale resources based on real-time demand.

Why Do We Need Auto Scaling?

1. **High Availability** – Ensures application uptime by automatically replacing failed instances.
2. **Cost Optimization** – Scales resources up or down based on traffic, reducing costs.
3. **Performance Efficiency** – Maintains optimal performance by scaling based on load.
4. **Fault Tolerance** – Automatically replaces unhealthy instances.
5. **Traffic Handling** – Adjusts resources dynamically for fluctuating workloads.

Steps to Work with Auto Scaling (EC2 Example)

Step 1: Create a Launch Template

- Go to **EC2 Console → Launch Templates → Create Launch Template**.
- Configure **AMI, instance type, security group, and key pair**.
- Define user data if needed (e.g., install software on boot).

Step 2: Create an Auto Scaling Group (ASG)

- Go to **EC2 Console → Auto Scaling Groups → Create Auto Scaling Group**.
- Select the **Launch Template**.
- Define the **VPC & Subnets** (for high availability).

Step 3: Configure Scaling Policies

- Set **Minimum, Desired, and Maximum instance count**.
- Choose **Scaling Policy** (e.g., **Target Tracking** for CPU > 50%).
- Attach **CloudWatch Alarms** to trigger scaling actions.

Step 4: Attach Load Balancer (Optional, but Recommended)

- Attach **Application Load Balancer (ALB)** for distributing traffic.
- Enable **Health Checks** to replace unhealthy instances.

Step 5: Monitor & Optimize

- Use **CloudWatch Metrics** to analyze Auto Scaling performance.
- Set up **SNS Notifications** for scaling events.
- Optimize cost using **Spot Instances** in ASG.

Real-World Example I Worked On:

- Configured **Auto Scaling for a Jenkins Master-Slave setup** to dynamically add/remove Jenkins agents based on job queue size.
- Implemented **ECS Fargate Auto Scaling** for a containerized microservices application, scaling based on CPU & Memory usage.
- Used **Scheduled Scaling** for **batch processing workloads**, ensuring more instances were available during peak hours.

3. What is AMI and what is difference between useradd and AMI?

An **Amazon Machine Image (AMI)** is a pre-configured template that contains:

- **OS** (e.g., Amazon Linux, Ubuntu, Windows)
- **Pre-installed software** (e.g., web server, database)
- **Configurations** (e.g., security settings, dependencies)
- **EBS snapshots** (root volume and additional volumes)

AMI is used to launch EC2 instances with a predefined setup, making it easy to deploy identical servers.

Difference Between useradd and AMI

Feature	AMI (Amazon Machine Image)	useradd Command
Definition	A pre-configured image to launch EC2 instances.	A Linux command used to create new users.
Purpose	Used for creating new EC2 instances with predefined configurations.	Used for adding user accounts on an existing Linux system.
Scope	System-wide (entire EC2 instance setup).	User-specific (adds a new user to the system).
Persistence	Once created, multiple instances can be launched from the same AMI.	Only affects the current system where the command is executed.
Example Use	Creating a custom AMI with pre-installed applications for scaling.	Adding a new developer account on a running EC2 instance.

4. What are load balancers and what is the use of it?

A **Load Balancer** is a networking service that distributes incoming traffic across multiple servers (EC2 instances, containers, or on-prem servers) to ensure high availability, reliability, and performance.

Why Do We Use Load Balancers?

1. **Distributes Traffic** – Spreads load evenly across multiple instances, preventing overload.
2. **High Availability** – Ensures application uptime by redirecting traffic from unhealthy instances.
3. **Scalability** – Works with Auto Scaling to dynamically adjust resources.
4. **Security** – Supports **SSL/TLS termination** for encrypted traffic and DDoS protection.
5. **Improves Performance** – Reduces latency with features like **content-based routing** and caching.

Types of Load Balancers in AWS (ELB – Elastic Load Balancer)

1. **Application Load Balancer (ALB)**
   - Works at **Layer 7 (HTTP/HTTPS)**.
   - Supports **path-based and host-based routing** (e.g., /api → one service, /app → another).
   - Best for **microservices, containerized applications, and APIs**.
2. **Network Load Balancer (NLB)**
   - Works at **Layer 4 (TCP/UDP, TLS)**.
   - Handles **millions of requests per second** with ultra-low latency.
   - Best for **high-performance and real-time applications**.
3. **Classic Load Balancer (CLB) [Legacy]**
   - Works at **Layer 4 & Layer 7** but lacks modern features.
   - Used for **simple applications** (not recommended for new deployments).
4. **Gateway Load Balancer (GWLB)**
   - Used for **third-party security appliances** like firewalls and intrusion detection systems.
   - Best for **traffic inspection and security**.

Real-World Use Cases:

✅ **Hosting a Scalable Web App:** ALB distributes traffic between multiple EC2 instances running a web application.

✅ **High-Performance Applications:** NLB routes TCP traffic efficiently for gaming servers.

✅ **Microservices Architecture:** ALB directs requests to different services based on URL paths.

✅ **API Gateway Alternative:** ALB + Lambda for cost-effective API handling.

5. Difference between app load balancer & n/w balancer with example.

Feature	Application Load Balancer (ALB)	Network Load Balancer (NLB)
Layer	Works at **Layer 7 (HTTP/HTTPS)**	Works at **Layer 4 (TCP/UDP, TLS)**
Routing Type	Routes traffic based on **URL paths, headers, cookies**	Routes traffic based on **IP address & ports**

Feature	Application Load Balancer (ALB)	Network Load Balancer (NLB)
Use Case	Best for **web applications, APIs, and microservices**	Best for **high-performance, low-latency applications**
Performance	Handles **millions** of requests per second	Handles **millions** of connections per second
Latency	Slightly higher due to **content-based routing**	Ultra-low latency (sub-millisecond)
Protocol Support	**HTTP, HTTPS, WebSockets**	**TCP, UDP, TLS**
Target Types	**EC2 instances, ECS, Lambda, IPs**	**EC2 instances, IPs, ALB as target**
Sticky Sessions	Supported using **cookies**	Supported using **source IP**
TLS Termination	Supports **SSL/TLS offloading**	Supports **SSL passthrough**
Health Checks	Application-level checks (**HTTP status codes**)	Network-level checks (**TCP connection**)
Cost	Lower than NLB	Higher than ALB

Examples

✅ **ALB Use Case** (Web Applications, Microservices)

- A website (example.com) uses ALB to direct requests:
  - /api → API server
  - /app → Web frontend
  - /admin → Admin panel

✅ **NLB Use Case** (High Performance, Low Latency)

- A real-time gaming server uses NLB to distribute TCP traffic across multiple backend servers.

When to Use What?

- Use **ALB** for **web apps, REST APIs, microservices** (when content-based routing is needed).
- Use **NLB** for **low-latency, high-throughput applications** (like databases, gaming, financial transactions).

6. How do you configure health checks in application load balancer?

Health checks in ALB **monitor the status** of target instances and **automatically remove unhealthy instances** from the load balancing rotation.

Steps to Configure Health Checks in ALB:

Step 1: Open the AWS Console

- Go to **EC2 Dashboard** → **Load Balancers**
- Select your **Application Load Balancer (ALB)**

Step 2: Edit Target Group Health Check Settings

- Click on the **Target Groups** tab
- Select the **Target Group** associated with the ALB
- Click **Health checks** → **Edit health check settings**

Step 3: Configure Health Check Parameters

- **Protocol** → HTTP or HTTPS
- **Path** → /health (or any application-specific health endpoint)
- **Port** → Select the port the application runs on (e.g., 80 for HTTP, 443 for HTTPS)

Step 4: Define Health Check Conditions

- **Healthy Threshold** – Number of successful checks before marking instance **healthy** (e.g., 3)
- **Unhealthy Threshold** – Number of failed checks before marking instance **unhealthy** (e.g., 2)
- **Timeout** – Time to wait for a response (5s default)
- **Interval** – Time between health check requests (30s default)
- **Success Codes** – Expected response codes (200, 302 for web apps)

Step 5: Save & Verify

- Click **Save changes**
- Under **Target Group → Targets**, check the health status of instances

---

Example Scenario

- If /health returns 200 OK, the instance is **healthy** and ALB routes traffic to it.
- If an instance fails 2 consecutive checks, ALB marks it **unhealthy** and stops sending traffic.
- Once it passes 3 successful checks, it is re-added to the load balancer.

7. How to check if the application url is working fine inside the EC2?

Use curl (Quick Check for Web Apps & APIs)
curl -I http://localhost:PORT
✅ If it returns HTTP 200 OK, the app is running fine.
Use wget (Alternative to curl)
wget -qO- http://localhost:PORT
Check if the Application is Listening on the Port
netstat -tulnp | grep PORT
ss -tulnp | grep PORT
✅ If you see LISTEN, the app is running on that port.
Use telnet to Test Port Connectivity
telnet localhost PORT
✅ If it connects, the application is responding.
Check Application Logs for Errors
sudo journalctl -u <service-name> --no-pager --lines=50
cat /var/log/<app>.log
Restart the Application Service (if needed)
sudo systemctl restart <service-name>
Verify Security Groups & Firewalls

- Ensure the required port is open in AWS **Security Groups** and **NACLs**.
- Check firewall rules with:
  sudo iptables -L -n | grep PORT

Check Bind Address (If Not Accessible Externally)

- Modify the application to listen on 0.0.0.0 instead of 127.0.0.1.

8. Have you worked on Lambda?

Yes, I have worked on **AWS Lambda** to automate tasks, process events, and integrate with AWS services. I have used it for **CI/CD automation, log processing, scheduled tasks, and API backends** in serverless applications.

9. What is serverless?

Serverless computing allows running applications **without managing servers**. AWS automatically handles **provisioning, scaling, and maintenance**, so developers only focus on writing code.

✅ **Example AWS Serverless Services:**

- **Compute:** AWS Lambda
- **Storage:** Amazon S3
- **Database:** DynamoDB
- **API Gateway:** Manages HTTP requests for Lambda

✅ **Key Benefits:**

- No server management
- Automatic scaling
- Cost-efficient (pay only for execution time)
- High availability

10. How can you schedule a function to trigger in lambda?

You can schedule AWS Lambda using:

1. **Amazon EventBridge (Preferred Method)**

- Create a **Rule** in EventBridge.
- Set **Schedule Expression** (cron or rate).
- Set Lambda as the target.

✅ **Example: Run Lambda Every Day at 10 AM**

```
json
CopyEdit
{
  "schedule": "cron(0 10 * * ? *)"
}
```

---

2. **AWS CloudWatch Events (Legacy Method)**
   - Go to **CloudWatch** → **Rules** → **Create Rule**
   - Set **Schedule Expression** (rate(1 hour) or cron(0 12 * * ? *))
   - Select **Lambda** as the target

---

3. **Using S3, DynamoDB, or API Gateway (Event-Driven)**
   - Lambda can be triggered by **S3 file uploads**, **DynamoDB changes**, or **API Gateway requests**.

11. How does lambda know that it has to take this file and process it? How does it interact with S3?

AWS Lambda processes files from S3 using **S3 Event Notifications**. When a file is uploaded, deleted, or modified in an S3 bucket, it triggers Lambda to process it.

How Lambda Interacts with S3?

1. Enable S3 Event Notification for Lambda

- Go to **Amazon S3 Console** → Select the **bucket**
- Click **Properties** → **Event notifications** → **Create event notification**
- Select **Event Type** (e.g., PUT for new file uploads)
- Choose **Destination** as **AWS Lambda**
- Select or create a **Lambda function**

✅ Now, whenever a new file is uploaded, S3 automatically triggers the Lambda function.

---

2. Lambda Function to Process the File

- AWS automatically **passes the file details** (bucket name, file name) as an event to Lambda.

✅ **Example Lambda Function (Python)**

```
import boto3
s3 = boto3.client("s3")
def lambda_handler(event, context):
    # Get bucket and file name from the event
    bucket_name = event['Records'][0]['s3']['bucket']['name']
    file_name = event['Records'][0]['s3']['object']['key']
    print(f"Processing file: {file_name} from bucket: {bucket_name}")
    # Download file from S3
    response = s3.get_object(Bucket=bucket_name, Key=file_name)
    file_content = response['Body'].read().decode('utf-8')
    # Process file content (example: print first 100 characters)
    print(file_content[:100])
    return {"statusCode": 200, "body": "File processed successfully"}
```

How It Works

1. **File Uploads to S3** → Triggers Lambda
2. **S3 Sends Event Data** → Lambda receives the bucket & file name
3. **Lambda Downloads the File** → Uses boto3 to read the file
4. **Processes the File** (e.g., parsing, saving to a database, etc.)

12. How do you login to EC2 without ssh access? Do we have any roles for sessions manager?

If SSH access is not available (e.g., port 22 is blocked or key pair is missing), you can use **AWS Systems Manager (SSM) Session Manager** to connect to the instance **without SSH**.

---

1. Using AWS Systems Manager (SSM) Session Manager

Steps to Enable SSM Session Manager on EC2:

1. **Attach the Required IAM Role to the EC2 Instance**
   - Go to **IAM → Roles → Create Role**
   - Select **AWS Service → EC2**
   - Attach **AmazonSSMManagedInstanceCore** policy
   - Attach this role to your EC2 instance
2. **Enable SSM Agent on the EC2 Instance**
   - Amazon Linux, Ubuntu, and Windows AMIs come with **SSM Agent pre-installed**
   - If missing, install it manually:
     sudo yum install -y amazon-ssm-agent # For Amazon Linux & RHEL
     sudo apt install -y amazon-ssm-agent # For Ubuntu & Debian
3. **Connect Using AWS Console or CLI**
   - Go to **EC2 Console → Select the instance → Click "Connect"**
   - Choose **Session Manager → Start Session**

   OR use CLI:
   aws ssm start-session --target i-xxxxxxxxxxxxxxxxx

✅ **Now you can access the EC2 instance shell without SSH!**

---

2. Alternative Methods to Access EC2 Without SSH

- **EC2 Serial Console:** For Amazon Linux & Ubuntu
- **EC2 Instance Connect:** Works for Amazon Linux 2 & Ubuntu
- **User Data Script:** Use cloud-init to reset SSH keys

Does Session Manager Require an IAM Role?
Yes, the EC2 instance must have an **IAM Role** with the AmazonSSMManagedInstanceCore policy attached.

13. Explain EC2 status checks.

AWS EC2 provides **status checks** to monitor the health of an instance. There are **two types** of status checks:

---

1. System Status Checks (AWS Infrastructure Issues)

✅ **Checks if AWS infrastructure is working properly** for the instance.

❌ If this fails, the issue is with **AWS hardware, networking, or power.**

Examples of Failures:

- Loss of network connectivity
- AWS hardware failure
- AWS software or hypervisor issues
- Loss of system power

Fix:

- **Wait for AWS to resolve** (if an outage)
- **Stop and start the instance** to migrate it to a healthy host

---

2. Instance Status Checks (OS or Software Issues)

✅ **Checks if the instance's OS is running properly.**

❌ If this fails, the issue is inside the instance (OS-level problem).

Examples of Failures:

- Incorrect OS configuration
- Exhausted CPU or memory
- Kernel panic or system crash
- Stuck boot process

Fix:

- Use **SSM Session Manager** to access the instance without SSH
- Check logs (/var/log/messages, /var/log/syslog)
- Restart the instance
- Restore from a backup (if needed)

14. What is Bastion host? Explain its concept.

A **Bastion Host** is a **highly secured** and **publicly accessible** server used to provide **controlled access** to private resources (e.g., EC2 instances in a private subnet). It acts as a **jump server** that allows secure SSH or RDP connections to instances in a **private network**.

◆ **Concept of Bastion Host**
In AWS, EC2 instances inside a **private subnet** cannot be accessed directly from the internet.
Instead of exposing all instances, we:
✅ Deploy a **Bastion Host** in a **public subnet**
✅ Allow SSH/RDP access **only** to the Bastion Host
✅ Use the Bastion Host to connect to instances in a **private subnet**

◆ **How It Works?**
1. **User connects to Bastion Host** (via SSH/RDP).
2. **Bastion Host then connects to private EC2 instances** (via private IP).
3. **Instances in private subnet have no public IP**, so they remain protected.

◆ **Security Best Practices for Bastion Host**
✅ **Restrict Public Access:** Open SSH (port 22) only to specific IPs (e.g., your office/home IP).
✅ **Use IAM Roles:** Grant minimum required permissions to Bastion Host.
✅ **Enable MFA:** Use Multi-Factor Authentication for additional security.
✅ **Disable Password-Based Login:** Use SSH key authentication.
✅ **Use AWS Systems Manager (SSM) Session Manager:** To avoid exposing SSH completely.
✅ **Monitor Access Logs:** Enable AWS CloudTrail and CloudWatch for monitoring.

◆ **How to Set Up a Bastion Host in AWS?**
1. **Launch an EC2 Instance**
   - Choose **Amazon Linux** or **Ubuntu**.
   - Place it in a **Public Subnet**.
   - Assign a **Public IP**.
2. **Configure Security Groups**
   - **Allow SSH (port 22) from trusted IPs**.
   - **Deny all inbound traffic** except SSH.
   - **Allow outbound traffic** to the private subnet.
3. **Connect to Private Instances**
   - First, SSH into the Bastion Host: ssh -i my-key.pem ec2-user@<Bastion-Public-IP>
   - Then, SSH into the private instance from Bastion: ssh -i my-key.pem ec2-user@<Private-Instance-Private-IP>

◆ **Alternative to Bastion Host**
🔹 Instead of using a Bastion Host, AWS recommends using **AWS Systems Manager (SSM) Session Manager**, which allows secure SSH access **without exposing a public IP**.

---

15. On what policy will elb route traffic. Do u know what is round robin?

ELB Type	Traffic Routing Policy
Application Load Balancer (ALB)	Routes traffic **based on HTTP/HTTPS rules**, such as path-based (/api, /login) or host-based (app.example.com).
Network Load Balancer (NLB)	Routes traffic **based on TCP/UDP connections**, sending traffic to the least loaded target.
Classic Load Balancer (CLB)	Uses either **Round Robin** (default) or **Least Outstanding Requests** (LOR) for routing.

📌 **What is Round Robin?**

Round Robin is a **load balancing algorithm** where traffic is distributed evenly across all available servers (EC2 instances) in a circular order.

Example:

If an ELB is distributing traffic to **3 EC2 instances**, the requests will be routed as:

1. First request → EC2 Instance **A**
2. Second request → EC2 Instance **B**
3. Third request → EC2 Instance **C**
4. Fourth request → Back to **EC2 A** (cycle repeats)

✅ **Used in:**

✓ **Application Load Balancer (ALB) for HTTP/HTTPS**

✓ **Classic Load Balancer (CLB) for TCP-based requests**

📌 **Alternative Routing Policies in ELB**

Apart from **Round Robin**, ELB can use:

- **Least Outstanding Requests (LOR)** → Routes new requests to the EC2 instance with the fewest active requests.
- **Sticky Sessions** → Routes requests from the same client to the same instance.

Storage:

1. What is EFS and what are advantages of using it?

Amazon **Elastic File System (EFS)** is a **fully managed, scalable, and elastic NFS (Network File System)** that allows multiple EC2 instances and other AWS services to access a shared file system. It grows and shrinks automatically based on usage.

✅ **Advantages of Using EFS**

1. 📈 **Scalable & Elastic**
   - Automatically scales **from GBs to petabytes** without manual intervention.
2. ⚡ **High Availability & Durability**
   - Data is replicated **across multiple AZs (Availability Zones)** for **99.999999999% durability**.
3. 🔄 **Concurrent Access**
   - Supports **simultaneous access by multiple EC2 instances** (great for distributed applications).
4. 🔌 **NFS-Compatible**
   - Works with **Linux-based workloads** via **NFSv4 protocol** (easy integration).
5. 💰 **Cost-Effective**
   - **Pay only for what you use** (no need to pre-provision storage).
   - **Two pricing tiers**:
     - **Standard (frequent access)**
     - **Infrequent Access (cost-saving for cold data)**
6. 🚀 **High Performance**
   - Supports **throughput up to 10 GBps** and **low-latency access**.
   - **Burst mode** and **Provisioned Throughput** options available.
7. 🔐 **Secure & Integrated**
   - Supports **IAM, VPC security groups, and encryption (at rest & in transit)**.
   - Integrated with AWS **Backup, CloudWatch, and CloudTrail** for monitoring & security.

2. How to list volumes?

Lists all EBS volumes in the current AWS region = **aws ec2 describe-volumes**

Command	Description
`lsblk`	Lists **all attached disks & partitions** with sizes and mount points.
`fdisk -l`	Displays **detailed partition table** for all disks.
`df -h`	Shows **mounted file systems** with used & available space.

Command	Description
`lsblk -o NAME,FSTYPE,SIZE,MOUNTPOINT,UUID`	Lists **EBS volumes** attached to an AWS EC2 instance.
`sudo file -s /dev/xvdf`	Checks if an **EBS volume is formatted**.
`sudo blkid`	Lists **UUID and filesystem types** of attached volumes.
`lvdisplay`	Displays **Logical Volumes (LVs)** in the system.
`vgdisplay`	Shows **Volume Groups (VGs)** details.
`pvdisplay`	Lists **Physical Volumes (PVs)** and their attributes.

3. How can you use multiple windows EC2 machines to access multiple files?

Solution	Protocol	Use Case	Performance	Cost
Amazon FSx for Windows	SMB	Enterprise Windows file sharing	High	$$
Self-Managed Windows File Server	SMB	Small-scale file sharing	Medium	$
Amazon S3 + Storage Gateway	SMB/NFS	Object storage with file caching	Medium	$$

4. If customer wants to resize/scaleup ebs to 100gb from 50gb what are the steps

1. Modify the EBS Volume (AWS Console or CLI)

Using AWS Console:

1. Go to **EC2 Dashboard → Volumes**.
2. Select the **EBS volume** attached to the instance.
3. Click **Modify Volume**.
4. Enter **100 GB** (new size).
5. Click **Modify** → **Yes** to confirm.

Using AWS CLI: `aws ec2 modify-volume --volume-id vol-xxxxxxxx --size 100`

---

2. Verify the Size Increase

Check if the volume has been modified: `aws ec2 describe-volumes --volume-ids vol-xxxxxxxx`

Ensure the Size reflects **100GB**.

---

3. Extend the Filesystem (Inside EC2 Instance)

After resizing, the OS still recognizes the old size. You need to extend the filesystem.

For Linux (ext4/XFS Filesystem)

Check the disk size: lsblk

Extend Partition (if needed)

If using a partitioned volume: sudo growpart /dev/xvda 1

Resize Filesystem (ext4): sudo resize2fs /dev/xvda1

Resize Filesystem (XFS): sudo xfs_growfs /

For Windows

1. Open **Disk Management** (diskmgmt.msc).
2. Right-click on the volume → **Extend Volume**.
3. Follow the wizard and allocate **100GB**.

---

4. Verify New Storage

Run:

df -h # Linux

lsblk # Linux

wmic logicaldisk get size,freespace,caption # Windows

✅ **Final Outcome:**

- EBS volume increased to **100GB**.
- Filesystem resized successfully.
- No reboot required.

5. How to make EBS vol persistent means even after restart we shouldn't lose the data
6. How to mount new vol to ec2

1. Attach EBS Volume to EC2 (If Not Already Attached)

Using AWS Console

1. Go to **EC2 Dashboard → Volumes**.
2. Select the **EBS Volume**.
3. Click **Actions → Attach Volume**.
4. Choose the **EC2 Instance** and specify the **device name** (e.g., /dev/xvdf).
5. Click **Attach**.

Using AWS CLI: `aws ec2 attach-volume --volume-id vol-xxxxxxxx --instance-id i-xxxxxxxx --device /dev/xvdf`

---

2. Mount EBS Volume (Linux)

If the volume is **not formatted**, format it:

```
sudo mkfs -t ext4 /dev/xvdf
```

Create a mount point:

```
sudo mkdir /mnt/ebs
sudo mount /dev/xvdf /mnt/ebs
```

---

3. Make It Persistent (Linux)

To **auto-mount after reboot**, add an entry in **/etc/fstab**:

```
echo "/dev/xvdf /mnt/ebs ext4 defaults,nofail 0 2" | sudo tee -a /etc/fstab
```

Verify: `cat /etc/fstab`

Test by rebooting: `sudo reboot`

After reboot, check if the volume is mounted: `df -h`

---

4. Make EBS Persistent on Windows

Using Disk Management:

1. Open **Disk Management (diskmgmt.msc)**.
2. Right-click on the **EBS Volume** → Click **Change Drive Letter and Paths**.
3. Assign a **drive letter** (e.g., D:).
4. Click **OK**.

Using AWS CLI:

```
Initialize-Disk -Number 1
New-Partition -DiskNumber 1 -UseMaximumSize -AssignDriveLetter
Format-Volume -DriveLetter D -FileSystem NTFS
```

After reboot, the volume remains accessible with the same **drive letter**.

---

5. Enable EBS Backup for Data Protection (Optional)

Enable **automatic snapshots** using Lifecycle Manager:

- Go to **EC2 Dashboard → Lifecycle Manager**.
- Create a policy to **take snapshots** every day/hour.

OR **Manually create snapshots**:

```
aws ec2 create-snapshot --volume-id vol-xxxxxxxx --description "Daily backup"
```

✅ **Final Outcome:**

- **EBS volume remains persistent after reboot**.
- **Automatically mounted on Linux (/etc/fstab)**.
- **Persistent drive letter in Windows (diskmgmt.msc)**.
- **Optional backups via snapshots** for **disaster recovery**.

7. Can u let me know the different S3 bucket classes & when to use

Storage Class	Use Case	Key Features	Cost
S3 Standard	Frequently accessed data	High durability (99.999999999%), low latency	High cost
S3 Intelligent-Tiering	Unknown or unpredictable access patterns	Automatically moves objects between frequent & infrequent access tiers	Slightly lower cost than Standard
S3 Standard-IA (Infrequent Access)	Infrequently accessed but still needed immediately	Lower cost than Standard, retrieval cost per GB applies	Lower storage cost, retrieval fees
S3 One Zone-IA	Infrequently accessed data that can tolerate data loss in case of AZ failure	Stored in a single Availability Zone, lower durability (99.99%)	20% cheaper than Standard-IA
S3 Glacier Instant Retrieval	Archive data that needs millisecond retrieval	Low-cost alternative to Standard-IA	Cheaper than Standard-IA, retrieval cost applies
S3 Glacier Flexible Retrieval	Archive data accessed occasionally (minutes to hours retrieval)	Low-cost archival, retrieval within minutes to hours	Very low storage cost, retrieval fees
S3 Glacier Deep Archive	Long-term archival (retrieval time: hours)	Cheapest storage, ideal for regulatory or compliance data	Lowest cost, high retrieval time

---

📌 **When to Use Each Class?**

- 🔹 **Use S3 Standard** → If you frequently access data (e.g., web apps, real-time analytics).
- 🔹 **Use Intelligent-Tiering** → If data access patterns are unpredictable.
- 🔹 **Use Standard-IA or One Zone-IA** → For infrequent but critical data (e.g., backups, disaster recovery).
- 🔹 **Use Glacier Instant Retrieval** → If archived data needs immediate access.
- 🔹 **Use Glacier Flexible Retrieval** → If data can wait a few minutes or hours to retrieve.
- 🔹 **Use Glacier Deep Archive** → For long-term storage where retrieval takes hours (e.g., compliance, legal records).

8. What kind of objects we store on S3

Object Type	Use Case	Example Scenarios
Static Website Content	Hosting static websites, HTML, CSS, JavaScript files	Web applications, personal blogs
Images & Videos	Storing and streaming media files	CDN, video streaming, social media platforms
Backup & Disaster Recovery	Storing database, server, and application backups	AWS RDS snapshots, EBS backups, business continuity plans
Logs & Analytics Data	Storing logs for monitoring & analysis	CloudTrail logs, ELB access logs, security event logs
Big Data Storage	Large-scale data storage for analysis & AI/ML	Data lakes, Hadoop datasets, machine learning model training
Archives & Compliance Data	Long-term archival for regulatory purposes	Financial records, healthcare data, legal documents
Software Packages & Binaries	Storing application builds, container images	Docker container images, software distribution
IoT Data & Sensor Logs	Storing IoT device data & real-time event logs	Smart devices, industrial IoT applications

Object Type	Use Case	Example Scenarios
Database Snapshots	Storing DB backups and snapshots	RDS, DynamoDB exports, PostgreSQL/MySQL backups
Code Repositories	Storing versioned source code files	Git repositories, deployment artifacts

9. How data will move from 1 class to another

Method	Description	Use Case
S3 Lifecycle Policies	Automatically transitions objects to different storage classes based on age (e.g., move objects from S3 Standard to Glacier after 30 days)	Cost optimization for infrequently accessed data
S3 Intelligent-Tiering	Automatically moves data between frequently and infrequently accessed tiers based on access patterns	Unknown or unpredictable access frequency
Manual Transition	Users manually copy/move data using AWS CLI, SDKs, or Console	One-time migration or immediate storage class change
AWS DataSync	Efficient data transfer between S3 classes or regions	Large-scale data movement, hybrid cloud scenarios

✅ **Steps to Configure Lifecycle Rule in AWS Console**

1. **Go to S3 Console** → Select the bucket
2. **Go to "Management" tab** → Click "Create lifecycle rule"
3. **Set conditions** (e.g., move data after 30 days)
4. **Choose target storage class** (IA, Glacier, etc.)
5. **Save & apply the rule**

10. What is highest & lowest paid class

S3 Standard is the most expensive because it provides **low latency** and **high availability**.
S3 Glacier Deep Archive is the cheapest as it is meant for **cold storage** with long retrieval times.

11. IAM user from one account should access objects in another bucket in another account.

Cross-Account S3 Bucket Access Using IAM Users
To allow an **IAM User from Account A** to access an **S3 bucket in Account B**, follow these steps:

---

1. Grant Bucket Permissions in Account B
Create a **Bucket Policy** in **Account B's S3 bucket** to allow access to the IAM user in **Account A**.

📌 **Bucket Policy (Account B - S3 Bucket Owner)**

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Principal": { "AWS": "arn:aws:iam::ACCOUNT_A_ID:user/IAM_User"
},
            "Action": ["s3:GetObject", "s3:PutObject"],
            "Resource": "arn:aws:s3:::BUCKET_NAME/*"
        }
    ]
}
```

Replace:

- `ACCOUNT_A_ID` → AWS Account ID of Account A
- `IAM_User` → IAM username in Account A
- `BUCKET_NAME` → S3 bucket name in Account B

---

2. Assign IAM Policy in Account A

Attach the following IAM policy to the **IAM User in Account A**.

📌 **IAM Policy (Account A - IAM User)**

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": ["s3:GetObject", "s3:PutObject"],
            "Resource": "arn:aws:s3:::BUCKET_NAME/*"
        }
    ]
}
```

---

3. (Optional) Use AWS CLI to Access S3 Bucket

The IAM user in **Account A** can now access the bucket in **Account B** using AWS CLI:

✅ **To list objects:** `aws s3 ls s3://BUCKET_NAME --profile IAM_User_Profile`

✅ **To upload a file:** `aws s3 cp myfile.txt s3://BUCKET_NAME/ --profile IAM_User_Profile`

◆ **Alternative: Use IAM Role for Secure Access**

Instead of an IAM User, you can set up **cross-account IAM Roles** with `sts:AssumeRole` to grant temporary permissions.

12. What is the maximum image size limit to upload to S3?

The **maximum image (or object) size** that can be uploaded to **Amazon S3** is **5 TB (terabytes)**. However, there are some constraints:

Upload Limits in S3:

Upload Method	Maximum File Size	Notes
Single PUT request	**5 GB**	You cannot upload files larger than **5 GB** in a single request.
Multipart Upload	**5 TB**	Required for files **larger than 5 GB**. Supports up to **10,000 parts**, each between **5 MB - 5 GB**.

13. In storage what all services u have used

Storage Service	Use Case / Purpose
Amazon S3	Object storage for storing static files, backups, and logs. Used for hosting websites and integrating with CloudFront.
Amazon EBS (Elastic Block Store)	Persistent block storage for EC2 instances. Used for storing application data, databases, and snapshots.
Amazon EFS (Elastic File System)	Shared file system for multiple EC2 instances. Used for distributed applications and file storage.
Amazon FSx	Managed file storage (Windows File Server & Lustre). Used for Windows-based workloads and high-performance computing.
AWS Backup	Centralized backup management for EBS, RDS, S3, and DynamoDB.
Amazon Glacier / S3 Glacier	Long-term, archival storage for infrequent access data (cheaper than S3 Standard).
AWS Storage Gateway	Hybrid cloud storage solution to integrate on-premise environments with AWS storage.
Amazon RDS Storage	Used for database storage (MySQL, PostgreSQL, SQL Server, etc.).
Amazon DynamoDB Storage	NoSQL database with SSD-backed storage.

14. For what have you used S3?

Use Case	Description
Static Website Hosting	Hosted static websites by enabling public access and integrating with CloudFront.
Log Storage	Stored application logs, server logs, and audit logs for analysis.
Backup & Archival	Used S3 to store backups of EC2 instances, databases, and application data. Integrated with **Lifecycle Policies** to move data to **Glacier** for cost optimization.
CI/CD Pipeline Artifacts	Stored build artifacts (e.g., .jar, .war, .zip files) for deployment pipelines in Jenkins.
File Sharing & Collaboration	Provided secure access to files using **pre-signed URLs** and **S3 Access Policies**.
Cross-Account Access	Allowed IAM users from one AWS account to access S3 buckets in another account using **bucket policies** and **IAM roles**.
Hosting Media Files	Stored images, videos, and documents for applications and accessed via **CloudFront CDN**.
Disaster Recovery	Used **S3 Replication** to copy data across AWS regions for redundancy.
Terraform State File Storage	Used **S3 with DynamoDB locking** to manage **Terraform remote state** securely.

15. Explain S3 and it's advantages.

Amazon S3 (Simple Storage Service)

Amazon S3 is an **object storage service** that provides **scalability, security, durability, and high availability** for storing and retrieving any amount of data from anywhere. It is widely used for **backup, archival, big data analytics, content distribution, and static website hosting**.

Key Advantages of Amazon S3

Advantage	Description
Scalability	Automatically scales to store unlimited data without managing infrastructure.
Durability	Provides **99.999999999% (11 9's) durability**, ensuring data is safe and redundant across multiple Availability Zones.
Availability	Ensures **99.99% availability** for Standard class, allowing quick access to stored objects.
Security	Supports **IAM roles, bucket policies, encryption (SSE, KMS), MFA delete**, and **Access Control Lists (ACLs)** for secure data access.
Cost-Effective	Offers multiple **storage classes** (S3 Standard, Intelligent-Tiering, Glacier, etc.), helping reduce costs based on access frequency.
Lifecycle Policies	Automates moving objects between storage classes (e.g., Standard to Glacier) to optimize costs.
Versioning & Replication	Supports **versioning** to retain previous object versions and **cross-region replication (CRR)** for disaster recovery.
Data Processing & Analytics	Integrates with **AWS Athena, Glue, and Redshift** to process large-scale data directly from S3.
Static Website Hosting	Can be used to **host static websites** by enabling public access and configuring CloudFront for CDN.
Event-Driven Triggers	Integrates with **Lambda, SNS, and SQS** to trigger automated workflows based on object events.
Integration with AWS Services	Works seamlessly with **EC2, RDS, CloudFront, API Gateway, Lambda, AI/ML models**, and more.

Database:

1. Have you worked on RDS? Have you worked on cost optimization for RDS?

   Yes, I have worked on **Amazon RDS (Relational Database Service)** for managing relational databases like **MySQL, PostgreSQL, SQL Server, and Aurora**.

   - **Tasks I have performed:**
     ✓ **RDS Instance Provisioning** – Setting up & configuring RDS instances.
     ✓ **Automating Backups & Snapshots** – Using automated backups & manual snapshots.
     ✓ **Multi-AZ Deployment** – Ensuring high availability.
     ✓ **Read Replicas** – Scaling read-heavy workloads.
     ✓ **Monitoring & Security** – Using CloudWatch, Performance Insights, IAM policies, and encryption.
     ✓ **Database Migrations** – Using AWS DMS to migrate databases.

◆ **Have You Worked on Cost Optimization for RDS?**
Yes, I have optimized RDS costs using the following strategies:

✅ **Cost Optimization Techniques for RDS**

1. 🛠 **Choose the Right Instance Type**
   - Used **smaller instance types (db.t3.micro, db.t3.small)** for dev/test environments.
   - Used **memory-optimized (db.r5) or compute-optimized (db.c5) instances** based on workload needs.
2. 💰 **Use Reserved Instances (RIs) or Savings Plans**
   - **Reserved Instances** (1-3 years commitment) → Up to **69% cost savings**.
   - **Savings Plans** → More flexible but still offers discounts.
3. 🛑 **Stop Unused RDS Instances**
   - Scheduled **auto stop/start** for non-production databases (e.g., using AWS Lambda).
   - Stopped **idle RDS instances** to avoid unnecessary costs.
4. 📊 **Optimize Storage Type & Size**
   - Used **GP3 instead of GP2** for lower cost & higher IOPS.
   - Right-sized **storage** based on actual usage.
   - **Enabled Storage Auto-scaling** to avoid overprovisioning.
5. 📖 **Use Read Replicas Instead of Scaling Primary DB**
   - Created **read replicas** for read-heavy workloads instead of scaling up primary DB.
6. 🗄 **Optimize Backups & Retention Period**
   - **Reduced backup retention period** for non-production DBs.
   - Used **AWS Backup for cross-region snapshots instead of multi-AZ for DR (cost savings).**
7. 🚀 **Use Amazon Aurora (Instead of RDS MySQL/PostgreSQL)**
   - Aurora **Auto Scaling** dynamically adjusts instances → Lower cost than RDS in some cases.
   - **Aurora Serverless** for infrequent workloads → **Pay per second instead of hourly pricing**.
8. 🔍 **Monitor & Auto Scale with CloudWatch**
   - Used **AWS CloudWatch metrics** & **Performance Insights** to monitor CPU, memory, and connections.
   - Set up **auto-scaling for RDS read replicas** to handle peak loads efficiently.

💡 **Have You Implemented RDS Cost Optimization in Real Scenarios?**
Yes, in my projects, I have:

✅ **Migrated from RDS MySQL to Aurora MySQL** for cost savings.

✅ **Implemented auto-stop/start policies** for non-prod databases.

✅ **Used reserved instances & storage auto-scaling** to cut costs.

2. Do you know DynamoDB?

   Yes, I have worked with **Amazon DynamoDB**, a fully managed NoSQL database service. It is **highly scalable, serverless, and provides low-latency performance** for key-value and document-based workloads.

✅ **Key Features of DynamoDB**

1. 🔄 **Fully Managed & Serverless** – No need to provision or manage servers.
2. ⚡ **Single-digit Millisecond Latency** – Ensures high performance.
3. 📈 **Auto Scaling & On-Demand Mode** – Automatically scales based on traffic.
4. 🔀 **Multi-Region Global Tables** – Provides cross-region replication.
5. 🔐 **Security & Access Control** – Supports IAM, VPC endpoints, and encryption.
6. 📂 **Flexible Schema** – No fixed schema; stores JSON-like data.
7. 🛠 **Streams & Triggers** – Integrates with AWS Lambda for real-time event processing.
8. 💰 **Pay-as-You-Go Pricing** – No upfront costs; billed based on read/write requests.

✅ **Use Cases of DynamoDB**

✓ **Caching Layer** – Storing session data or frequently accessed data.
✓ **IoT Applications** – Handling real-time data ingestion.
✓ **E-Commerce** – Shopping carts, order tracking, and inventory management.
✓ **Gaming Leaderboards** – Handling high-volume, low-latency requests.
✓ **Serverless Applications** – Backend for APIs (integrates with AWS Lambda & API Gateway).

◆ **How Do We Use DynamoDB to Lock Terraform State File?**

When using **Terraform with an S3 backend**, we need a mechanism to prevent **multiple users** from running terraform apply simultaneously, which can cause **state corruption**. **DynamoDB is used for state locking and consistency**.

✅ **Steps to Lock Terraform State Using DynamoDB**

1. **Create an S3 Bucket for Terraform State Storage**
   - This bucket will store the Terraform state file.
2. **Create a DynamoDB Table for Locking**
   - The table **prevents concurrent Terraform executions**.
   - **Primary Key:** LockID (String)
   - **Read/Write Capacity Mode: Provisioned (Recommended: RCU = 5, WCU = 5)**

```
resource "aws_dynamodb_table" "terraform_lock" {
  name         = "terraform-lock-table"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }
}
```

3. **Configure Terraform Backend with S3 & DynamoDB**
   - Define backend configuration in backend.tf:

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state-bucket"
    key            = "terraform/state.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-lock-table"
    encrypt        = true
  }
}
```

4. **How Locking Works?**
   - When Terraform runs, it **creates a lock** in the DynamoDB table.
   - If another process tries to apply changes, Terraform **waits until the lock is released**.
   - Once the operation completes, **the lock entry is deleted**.

✅ **Why Use DynamoDB for Locking?**

✓ **Prevents state corruption** by ensuring only **one Terraform execution at a time**.
✓ **Highly available** & **serverless** → No need to manage infrastructure.
✓ **Seamless integration** with AWS S3 & Terraform.

Monitoring:

1. What is cloud watch? Why are we using it?

Amazon CloudWatch is a **monitoring and observability service** for AWS resources and applications. It helps in collecting, analyzing, and responding to operational data.

✅ **Why Do We Use CloudWatch?**
✓ **Real-time Monitoring** – Tracks CPU, memory, disk, network usage, etc.
✓ **Automatic Alerts** – Sends **alarms (SNS, email, Slack)** for threshold breaches.
✓ **Log Management** – Stores and analyzes application logs (**CloudWatch Logs**).
✓ **Performance & Cost Optimization** – Detects underutilized resources.
✓ **Troubleshooting & Debugging** – Helps in identifying issues and failures.
✓ **Event-Driven Actions** – Triggers AWS Lambda or EC2 Auto Scaling.

✅ **CloudWatch Key Features**

1. **CloudWatch Metrics**
   - Collects **CPU, memory, disk, and network metrics** from EC2, RDS, Lambda, etc.
2. **CloudWatch Alarms**
   - Sends **notifications (SNS, email, Slack, Lambda, Auto Scaling)** when a metric crosses a threshold.
   - Example: **Trigger Auto Scaling when CPU > 80%**.
3. **CloudWatch Logs**
   - Captures logs from **EC2, Lambda, ECS, API Gateway, etc.**
   - Helps in **debugging & auditing** application issues.
4. **CloudWatch Events (EventBridge)**
   - Triggers **Lambda, SNS, SQS, or Step Functions** based on events.
   - Example: **Run a script every day at 8 AM**.
5. **CloudWatch Dashboards**
   - Custom visualizations of metrics & logs on a **centralized dashboard**.

✅ **How Have You Used CloudWatch?**
✓ **Created alarms** to trigger auto-scaling when CPU utilization exceeds 75%.
✓ **Monitored Lambda execution time & errors** using CloudWatch Logs.
✓ **Set up EventBridge rules** to automate daily backups.
✓ **Created dashboards** to visualize system health & performance.

2. What all monitoring tools have you worked with AWS?

✅ **1. Amazon CloudWatch**
- **Use Case: Monitors AWS resources & applications.**
- **Features:**
  ✓ Collects **metrics** (CPU, memory, network, disk)
  ✓ Stores **logs** (EC2, Lambda, ECS, RDS)
  ✓ Sends **alarms** (Auto Scaling, SNS, Lambda triggers)
  ✓ Creates **dashboards** for visualization
  ✓ Triggers **automated actions** using EventBridge

✅ **2. AWS CloudTrail**
- **Use Case: Tracks user activity & API calls for auditing & security.**
- **Features:**
  ✓ Logs **all AWS API requests** (via Console, CLI, SDKs)

✓ Detects **unauthorized access & security threats**
✓ Integrates with **SIEM tools & AWS Config**
✓ Stores logs in **S3, Athena for analysis**

✅ 3. AWS Trusted Advisor

- **Use Case: Provides AWS best practice recommendations.**
- **Features:**
  ✓ Checks for **cost optimization, security, performance, and fault tolerance**
  ✓ Suggests **unused or underutilized resources**
  ✓ Improves **security posture by identifying weak configurations**

✅ 7. AWS Elastic Load Balancer (ELB) Access Logs
Use Case: Captures HTTP requests processed by ALB/NLB.
✓ Stores logs in **S3 for analysis**.
✓ Helps in **troubleshooting request failures & performance issues**.

✅ 8. Amazon RDS Logs
Use Case: Captures database logs (MySQL, PostgreSQL, etc.).
✓ Logs **slow queries, error logs, and system events**.
✓ Can be exported to **CloudWatch Logs for analysis**.

✅ 9. AWS S3 Server Access Logs
Use Case: Logs requests made to an S3 bucket.
✓ Helps in **tracking object access & unauthorized access attempts**.
✓ Stores logs in **another S3 bucket for analysis**.

✅ 10. AWS Lambda Logs
Use Case: Captures execution logs for serverless functions.
✓ Automatically logs to **CloudWatch Logs**.
✓ Helps in **debugging function failures & performance issues**.

📌 Summary Table: AWS Networking Logs

Log Type	Captures	Use Case
VPC Flow Logs	Network traffic (ingress/egress) at VPC level	Security, Troubleshooting
ELB Access Logs	Requests processed by ALB/NLB	Performance, Debugging
Route 53 Query Logs	DNS queries from VPC resources	DNS Troubleshooting
CloudTrail (Networking API Logs)	Changes to networking resources	Auditing, Security
AWS WAF Logs	Web requests (allowed/blocked)	Security, Attack Detection
AWS Network Firewall Logs	Allowed/blocked network connections	Security, Compliance
Global Accelerator Logs	Traffic flow & connection statuses	Debugging, Optimization

---

3. Have you worked on cloudwatch alarms/metrics? How do you delete the metrics?
Yes, I have worked on **Amazon CloudWatch** for monitoring and setting up alarms based on various metrics like **CPU utilization, memory usage, disk space, request count, etc.**

✅ How Do You Delete CloudWatch Metrics?
CloudWatch **does not allow direct deletion** of individual metrics because they are automatically created when an AWS service publishes data to CloudWatch. However, **metrics will automatically expire after 15 months** if no new data is sent.

🔹 **Ways to "Remove" CloudWatch Metrics:**
1. **Stop Sending Data** – If no data is sent, CloudWatch automatically deletes unused metrics after 15 months.
2. **Delete Resources** – Some metrics are tied to AWS resources (EC2, ALB, Lambda, etc.). Deleting the resource will remove its metrics.
3. **Delete Custom Metrics Manually** – If using custom metrics via PutMetricData, you can stop sending data, and they will expire over time.

📌 **Key Point:** You **cannot delete default AWS service metrics manually**, but you can stop logging to prevent further usage.

Architecture:

1. Deploy app and database using LBR. Write architecture diagram.
2. Started with AWS...can u let me know ur approach for multi-tier architecture where the app shld be available always & it should be secured

A **multi-tier architecture** divides an application into multiple layers to enhance **scalability, security, and availability**. Below is an **AWS-based multi-tier architecture approach** that ensures **high availability (HA) and security**.

---

1. Architecture Overview

A **3-tier architecture** typically consists of:
1. **Presentation Layer** → Load Balancer + Web Servers
2. **Application Layer** → Backend Processing (App Servers)
3. **Database Layer** → Relational or NoSQL Databases

📌 **Key AWS Services Used**:
- **Elastic Load Balancer (ALB)** → Distributes traffic
- **EC2 Auto Scaling** → Handles scaling and HA
- **RDS or DynamoDB** → Database layer
- **VPC & Subnets** → Network isolation
- **Security Groups & NACLs** → Network security
- **S3 + CloudFront** → Static content caching
- **IAM Roles & Policies** → Secure access control

---

2. Network Security Design

We will use a **secure VPC** with **multiple subnets** for better availability.

VPC Design

Layer	Resources Used	Security Measures
Public Subnet	ALB + NAT Gateway	Allow only HTTP/HTTPS
Private Subnet	App Servers (EC2)	Only ALB can access
Database Subnet	RDS/DynamoDB	Only App Layer can access
Bastion Host Subnet	Bastion Host	Admin access only

🚀 **Key Security Measures**: ✓ **Security Groups** → Restrict traffic flow

✓ **NACLs** → Additional layer of subnet security

✓ **IAM Roles** → Grant least-privilege access

---

3. High Availability & Scalability

Web Tier (Presentation Layer)
- Deploy **EC2 instances in multiple AZs**.
- Use an **Application Load Balancer (ALB)** to distribute traffic.
- **Auto Scaling Group (ASG)** to scale based on load.

App Tier (Business Logic Layer)
- **EC2 in private subnets** (not exposed to the internet).
- **Use AWS Systems Manager (SSM) for access** instead of SSH keys.
- Use **ECS (Fargate) or EKS** if containerized.

Database Tier (Data Layer)

- **RDS Multi-AZ for HA** or **Aurora Cluster**.
- **Read Replicas** for read-heavy workloads.
- **DynamoDB** for NoSQL workloads.

---

4. Security Best Practices

🔒 **Data Security**: ✓ **Encryption** → Enable AWS KMS for **S3, RDS, and EBS**

✓ **SSL/TLS** → Use ACM for ALB & API Gateway

✓ **IAM Policies** → Follow **least-privilege principle**

✓ **WAF & Shield** → Protect against DDoS attacks

🔍 **Monitoring & Logging**: ✓ **CloudWatch** → Monitor EC2, RDS, and ALB metrics

✓ **CloudTrail** → Track all API calls

✓ **GuardDuty** → Detect threats

---

5. Disaster Recovery (DR) Strategy

✓ **Multi-AZ Deployment** → For HA

✓ **Cross-Region Replication (S3, RDS)** → For DR

✓ **Backups & Snapshots** → RDS & EBS scheduled backups

✓ **Failover Strategies** → Route 53 failover routing

Conclusion

📌 **Key Takeaways**: ✅ **High Availability** → Multi-AZ, Auto Scaling, Load Balancer

✅ **Security** → IAM, NACLs, Security Groups, KMS

✅ **Scalability** → Elastic Load Balancing, Auto Scaling

✅ **Disaster Recovery** → Cross-Region backups, Route 53 failover

3. 3 tier architecture means what all will cm ur mind

A **3-tier architecture** separates an application into **three logical layers** to improve **scalability, security, and availability**.

---

1. Layers in 3-Tier Architecture

Tier	Purpose	AWS Services Used
Presentation Layer (Web Tier)	Handles user requests & UI	ALB, EC2 (Nginx/Apache), CloudFront, S3
Application Layer (Business Logic)	Processes application logic	EC2 (App Server), ECS (Fargate), Lambda, API Gateway
Database Layer (Data Tier)	Stores & manages data	RDS (MySQL/PostgreSQL), DynamoDB, Aurora

---

2. Key Components in Mind

1. **Load Balancer (ALB/NLB)** → Distributes traffic
2. **Auto Scaling Groups (ASG)** → Scales EC2 instances
3. **VPC Design** → Public & Private Subnets
4. **Security Groups & NACLs** → Control access between layers
5. **IAM Roles & Policies** → Enforce least privilege
6. **Multi-AZ & Backups** → High availability & disaster recovery
7. **Monitoring & Logging** → CloudWatch, CloudTrail, GuardDuty

---

Example AWS Setup

✅ **Web Tier**: ALB + EC2 (Public Subnet)

✅ **App Tier**: EC2/ECS (Private Subnet)

✅ **Database Tier**: RDS/Aurora (Private Subnet)

4. Write the architecture structure which you've built for any recent clients.

A **multi-tier architecture** designed for **high availability, security, and automation** using **AWS DevOps practices**.

1. Architecture Components

Component	Technology Used
Infrastructure as Code (IaC)	Terraform
Version Control	GitHub
CI/CD Pipeline	Jenkins, GitHub Actions
Compute Layer	EKS (Amazon Kubernetes Service)
Load Balancing	ALB (Application Load Balancer)
Networking	VPC with Public & Private Subnets
Security	Security Groups, NACLs, IAM Roles
Database	Amazon RDS (PostgreSQL)
Storage	Amazon S3, EFS
Monitoring	CloudWatch, Prometheus, Grafana
Logging	CloudTrail, AWS WAF, GuardDuty
Secret Management	AWS Secrets Manager

---

2. Deployment Architecture

- **Frontend (React App) hosted on CloudFront + S3**
- **Backend (Node.js/Python) running in EKS (Fargate)**
- **API Gateway for authentication & routing**
- **RDS PostgreSQL with Multi-AZ for HA**
- **VPC Peering for inter-region connectivity**
- **Route 53 for DNS management**
- **AWS Lambda for background jobs**

---

3. Security & Best Practices

✅ **IAM Roles & Least Privilege Access**

✅ **WAF & Shield for DDoS Protection**

✅ **Private Subnets for Application & DB**

✅ **SSL Termination at ALB**

✅ **Auto Scaling & Self-Healing Kubernetes Nodes**

---

4. Deployment Workflow (CI/CD)

1. **Developers push code to GitHub**
2. **Jenkins triggers Terraform to provision infra**
3. **GitHub Actions builds & pushes Docker images**
4. **EKS Deploys updated services**
5. **CloudWatch alarms notify failures**

🚀 **Business Outcome**

- **99.99% Uptime** with auto-scaling
- **60% Cost Reduction** using spot instances & scaling
- **Fast & Secure Deployments** via DevOps automation

5. How do you ensure your architecture is 99.99% available?

To ensure **99.99% availability** in AWS architecture, we implement **Multi-AZ and Multi-Region redundancy** across all layers. The compute layer uses **EKS with auto-scaling**, **self-healing pods**, and **ALB for load balancing**. The database layer relies on **Amazon RDS (Multi-AZ), Read Replicas, and DynamoDB Global Tables** for failover protection. Storage

durability is maintained using **S3 with Cross-Region Replication (CRR), versioning, and EFS with Multi-AZ availability**. Networking is secured through **VPC with multi-subnet architecture, AWS WAF, and Route 53 for DNS failover**. Continuous monitoring with **CloudWatch, CloudTrail, and GuardDuty** ensures real-time insights and security. **AWS Shield Advanced** protects against DDoS attacks, while **automated backups and disaster recovery strategies** minimize downtime. By leveraging **Infrastructure as Code (IaC) with Terraform**, we ensure quick infrastructure recovery, making the architecture **highly available, resilient, and secure**.

Networking:

1. Difference between Security groups vs NACL.

Feature	Security Group (SG)	Network ACL (NACL)
Level	Works at **instance level** (associated with EC2, ALB, RDS, etc.)	Works at **subnet level** (applies to all instances in the subnet)
Stateful/Stateless	**Stateful** – Response traffic is automatically allowed	**Stateless** – Both inbound & outbound rules must be explicitly defined
Inbound & Outbound Rules	Each rule applies to both request and response	Need **separate rules** for inbound and outbound traffic
Default Behavior	Denies all traffic by default, must explicitly allow	Allows all traffic by default, unless explicitly denied
Rule Evaluation	Evaluates **all rules** and applies the most permissive one	Evaluates **rules in order** (lowest number first)
Number of Rules Allowed	Limited (per AWS quotas)	Can have up to **20 rules per ACL**
Use Case	Controlling access to individual instances	Controlling access to entire subnets

---

💡 **When to Use What?**

- **Use Security Groups** when managing access at the **EC2 instance level** (default security mechanism).
- **Use NACLs** when controlling traffic **at the subnet level**, useful for **extra security layers** (e.g., blocking specific IP ranges).

2. Difference between VPC endpoints and NACLs.

Feature	VPC Endpoints	Network ACLs (NACLs)
Purpose	Allows secure, private access to AWS services (S3, DynamoDB, etc.) without using the internet	Controls inbound & outbound traffic at the **subnet level**
How It Works	Creates a private connection within AWS using **AWS PrivateLink or Gateway**	Defines **allow/deny rules** for traffic entering/exiting a subnet
Types	Interface Endpoint (ENI-based) and Gateway Endpoint (S3/DynamoDB)	Stateless firewall for **IP-based filtering**
Traffic Direction	Controls **service-to-service communication**	Controls **external traffic flow** to/from a subnet
Security	Avoids internet exposure, reducing attack surface	Blocks or allows specific **IP addresses & ports**
Stateful/Stateless	**Stateful** – Automatically allows response traffic	**Stateless** – Need separate rules for inbound & outbound
Use Case	Accessing AWS services privately (S3, DynamoDB, etc.) without an internet gateway	Controlling access between **subnets & external networks**

💡 **When to Use What?**

✓ **Use VPC Endpoints** to securely connect AWS services **without using public internet**.
✓ **Use NACLs** to enforce **network-level security rules** for inbound/outbound traffic **between subnets and external sources**.

3. What is THCP under VPC? Have you ever got a chance to work on it?

◆ **Transit Hub Control Plane (THCP) in VPC**
AWS **THCP (Transit Hub Control Plane)** is not an official AWS service or feature. However, you might be referring to:
✅ **AWS Transit Gateway (TGW) Control Plane** – A centralized networking hub that connects multiple VPCs, on-premise networks, and AWS services.
✅ **Transit Gateway Route Table Control Plane** – Manages network routing in a scalable manner.

4. What is cidr? How do you decide VPC and subnet CIDR range?

CIDR (Classless Inter-Domain Routing) is a method used to allocate **IP addresses efficiently**. It defines **IP address ranges** using a **prefix notation** (e.g., 192.168.1.0/24).

◆ **Example:**
- 10.0.0.0/16 → Provides **65,536 IPs**
- 10.0.1.0/24 → Provides **256 IPs**

◆ **How to Decide VPC & Subnet CIDR Range?**
✅ **1. Decide VPC CIDR**
- Consider the **number of resources** (EC2, RDS, Lambda, etc.) in your network.
- Choose a **private IP range**:
  - **Common VPC CIDR choices:**
    - 10.0.0.0/16 (65,536 IPs)
    - 192.168.0.0/16 (65,536 IPs)
    - 172.16.0.0/16 (65,536 IPs)

✅ **2. Divide Subnets Within VPC**
- Each subnet should have a **smaller CIDR** than the VPC.
- Assign **subnets per Availability Zone (AZ):**
  - Public Subnet: 10.0.1.0/24 (256 IPs)
  - Private Subnet: 10.0.2.0/24 (256 IPs)
  - Database Subnet: 10.0.3.0/24 (256 IPs)

✅ **3. Consider Future Scalability**
- Avoid **overlapping CIDRs** if using multiple VPCs.
- Plan enough IPs for autoscaling needs.

5. If /number increases will the IP numbers decrease or increase? How do you declare only one IP for a subnet?
Decreases. To declare only **one IP for a subnet**, you need to use a **/32 CIDR block**, which means the subnet will contain only **one IP address**. However, in AWS, you **cannot create a subnet with a /32 CIDR** because the minimum allowed subnet size is **/28**, which provides 16 IPs (but AWS reserves 5(`.0`, `.1`, `.2`, `.3`, `.255`) of them).

6. Worked on VPC Endpoints?

VPC Endpoints allow communication between AWS services **privately** inside a **VPC** without needing an **Internet Gateway (IGW), NAT Gateway, or VPN**.

Types of VPC Endpoints:
1. **Interface Endpoint** (AWS PrivateLink)
   - Uses **Elastic Network Interfaces (ENI)** inside the VPC.
   - Supports **services like S3, DynamoDB, Lambda, etc.**
   - Requires **Security Groups** for access control.
2. **Gateway Endpoint**
   - Only supports **S3 and DynamoDB**.
   - Uses **route tables** for traffic flow.

✅ How Have I Used VPC Endpoints?

1. Secure S3 Access Without Internet

- Created a **Gateway Endpoint** for S3.
- Updated the **Route Table** to route traffic through the endpoint.
- **Result:** EC2 instances accessed **S3 privately** without an Internet Gateway.

2. Access AWS Services Using Interface Endpoints

- Created an **Interface Endpoint** for **AWS Secrets Manager** and **SSM**.
- Allowed **private API access** via **Security Groups**.
- **Result:** Applications inside the VPC accessed AWS services securely.

✅ When to Use VPC Endpoints?

✓ When you need to access AWS services **without public internet**.

✓ To **avoid data transfer costs** of using NAT Gateway.

✓ When working with **private workloads inside VPC**.

7. What is VPC? What are prerequisite for VPC?

VPC (**Virtual Private Cloud**) is a logically isolated **private network** within AWS that allows you to **launch and manage resources securely**. It enables **custom IP addressing, subnets, route tables, security groups, and internet connectivity settings**.

✅ Prerequisites for VPC Creation

Before creating a VPC, you need to consider the following:

1. CIDR Block Planning

- Decide the **CIDR range** for the VPC (e.g., 10.0.0.0/16).
- Ensure that the CIDR block does **not overlap** with other networks.

2. Subnet Design

- Divide the **VPC CIDR** into **public and private subnets**.
- Choose subnet CIDR blocks based on the number of instances needed.

3. Internet Connectivity (If Required)

- **Internet Gateway (IGW)** → Required for public internet access.
- **NAT Gateway / NAT Instance** → Required for **private subnets** to access the internet.

4. Routing Configuration

- **Route Tables** → Direct traffic between subnets, IGW, NAT Gateway, or VPN.
- **VPC Peering / Transit Gateway** → If multiple VPCs need to communicate.

5. Security Settings

- **Security Groups (SGs)** → Control inbound/outbound traffic for instances.
- **Network ACLs (NACLs)** → Control traffic at the subnet level.
- **VPC Flow Logs** → For monitoring traffic.

8. Explain concept of VPC peering.

VPC Peering is a **network connection** between two VPCs that allows them to communicate **privately** using private IP addresses **without using an Internet Gateway, VPN, or AWS Transit Gateway**.
It enables resources in one VPC (e.g., EC2 instances, databases) to communicate with resources in another VPC **as if they were in the same network**.

✅ Key Features of VPC Peering

✓ **One-to-One Connection** → Direct peering between two VPCs.

✓ **Cross-Account Peering** → Peering between VPCs in **different AWS accounts**.

✓ **Cross-Region Peering** → Peering between **VPCs in different AWS regions**.

✓ **No Bandwidth Charges Within AWS Region** → Data transfer between peered VPCs is free within the same region (standard inter-AZ charges apply).

✅ How VPC Peering Works?

1. **Create a VPC Peering Connection** between the two VPCs.
2. **Accept the Peering Request** from the other VPC.
3. **Update Route Tables** in both VPCs to allow communication.
4. **Modify Security Groups & NACLs** to allow traffic.

✅ Limitations of VPC Peering

⊘ **No Transitive Peering** → If VPC A peers with VPC B and VPC B peers with VPC C, A **cannot** communicate with C.

⊘ **Cannot Reference Peered VPC in Security Groups** → You must use CIDR ranges instead.

⊘ **Only One-to-One Connection** → For **multiple VPCs**, AWS Transit Gateway is a better solution.

✅ Use Cases for VPC Peering

✓ Connecting **VPCs in different AWS accounts** (e.g., multi-account architecture).

✓ Allowing **secure communication** between **applications in different VPCs**.

✓ Enabling **private communication** between **development & production VPCs**.

---

9. What are api gateway & how to configure it

Amazon API Gateway is a **fully managed service** that enables developers to create, publish, maintain, secure, and monitor APIs at any scale. It acts as an **entry point** for client requests and forwards them to backend services like **AWS Lambda, EC2, ECS, EKS, DynamoDB, or on-prem servers**.

Key Features of API Gateway

✓ **RESTful and WebSocket APIs** – Supports HTTP, WebSockets, and REST APIs.

✓ **Authentication & Authorization** – Uses IAM, Lambda authorizers, and Cognito.

✓ **Traffic Management** – Rate limiting, throttling, and caching.

✓ **Monitoring & Logging** – Integrated with CloudWatch for logging and metrics.

✓ **Security** – Supports VPC links, WAF (Web Application Firewall), and SSL/TLS.

Steps to Configure API Gateway in AWS

Step 1: Create an API

1. Go to **AWS API Gateway Console**.
2. Click on **Create API** → Choose **REST API (Regional, Edge, or Private)**.
3. Select **New API** → Give it a **name** and **description**.

Step 2: Define Resources & Methods

1. Click **Actions** → **Create Resource** (e.g., /users, /orders).
2. Select a resource and **Create Method** (GET, POST, PUT, DELETE).
3. Choose an **Integration Type**:
   - **Lambda Function** – For serverless execution.
   - **HTTP Endpoint** – If calling an existing web service.
   - **Mock Integration** – For testing without backend logic.
   - **AWS Service** – To interact with DynamoDB, S3, etc.

Step 3: Configure Authentication & Authorization

1. Choose **IAM Roles**, **Lambda Authorizers**, or **Cognito User Pools** for security.
2. Set up **API Keys** for access control (optional).
3. Enable **Cross-Origin Resource Sharing (CORS)** if required.

Step 4: Deploy the API

1. Click **Actions** → **Deploy API**.
2. Create a new **Stage** (e.g., Dev, Staging, Prod).
3. Copy the **Invoke URL** (e.g., https://abc123.execute-api.us-east-1.amazonaws.com/prod).

Step 5: Monitor & Secure API

1. Enable **CloudWatch Logs & Metrics**.
2. Set up **Throttling & Rate Limits**.

3. Add **AWS WAF** for security.
4. Enable **Caching** to optimize performance.

Example Use Case

✅ **Serverless API** – API Gateway with **AWS Lambda** to serve a React frontend.

✅ **Microservices Communication** – Frontend app calls API Gateway → Routes to different backend services.

✅ **Private APIs** – API Gateway with **VPC Links** for accessing internal applications.

10. How api gateway will do Authorization and authentication

Method	Use Case	Authentication Type	Best For
IAM Authorization	AWS users & services	AWS SigV4 Signing	Internal APIs (AWS services)
Lambda Authorizer	Custom token verification	Custom Logic	Third-party auth (Auth0, Okta)
Cognito User Pools	OAuth 2.0 & JWT auth	Token-based	Mobile & Web apps
API Keys	Basic access control	API Key	Rate limiting & monetization

11. What are ssl components & have u configured any

SSL (Secure Sockets Layer) ensures encrypted communication between clients and servers. In AWS, SSL/TLS is used for securing data in transit.

---

1. SSL Components

1. **Certificate** 📜
   - Contains the domain name, organization details, and expiration date.
   - AWS uses **X.509 certificates** for SSL.
   - Example: *.example.com (Wildcard Certificate).
2. **Private Key** 🔑
   - A secret key used to decrypt incoming data.
   - Stored securely and **never shared**.
3. **Certificate Signing Request (CSR)** ✍
   - A request to a Certificate Authority (CA) to issue an SSL certificate.
4. **Certificate Authority (CA)** 🏢
   - A trusted organization that signs & issues SSL certificates.
   - Example: Let's Encrypt, DigiCert, GoDaddy.
5. **Intermediate Certificate** 🔗
   - Bridges the CA and the end-user certificate to establish trust.
6. **SSL/TLS Protocol** 🔒
   - Protocol versions: **TLS 1.2, TLS 1.3 (latest & recommended).**

---

2. Have I Configured SSL in AWS? ✅

Yes, I have configured SSL/TLS for:

1. AWS Load Balancer (ALB/NLB) → SSL Termination

- Used **AWS Certificate Manager (ACM)** to provision & attach an SSL certificate.
- Redirected HTTP traffic (port 80) to HTTPS (port 443).
- Enabled **TLS 1.2** for better security.

Steps to Configure:

1. **Request a certificate** in ACM for mydomain.com.
2. Validate via **DNS or Email**.
3. Attach the certificate to an **Application Load Balancer (ALB)**.

---

2. API Gateway → Securing APIs with SSL

- Used **Custom Domain Names with SSL** for API Gateway.
- Enforced HTTPS using **ACM certificates**.

Steps to Configure:

1. Create a **custom domain name** in API Gateway.
2. Upload an SSL certificate from ACM.

3. Associate it with the API Gateway stage.

---

3. S3 Bucket → Enforcing HTTPS for Static Websites

- Used **CloudFront + SSL** to serve static content securely.

Steps to Configure:

1. **Enable CloudFront** for S3.
2. Upload an SSL certificate in **ACM**.
3. Force **HTTPS redirection** in CloudFront settings.

---

4. EC2 Instance → Installing & Configuring SSL Manually

- Installed SSL certificates using **Let's Encrypt (Certbot)**.

Steps to Configure:

```
sudo apt update && sudo apt install certbot -y
sudo certbot --apache -d example.com
```

- Configured **Nginx or Apache** to serve traffic over HTTPS.

---

5. CloudFront with SSL for CDN

- Configured **CloudFront** to serve content securely with **TLS 1.2/1.3**.
- Allowed only **HTTPS traffic** (no HTTP fallback).

---

3. Conclusion 🚀

✓ **For Load Balancers →** Use **ACM** to provision SSL.

✓ **For API Gateway →** Use **ACM certificates** with a **custom domain**.

✓ **For EC2 Instances →** Use **Let's Encrypt (Certbot) or manually install SSL certs**.

✓ **For S3 Static Hosting →** Use **CloudFront + SSL**.

12. How do you install SSL certificate for website level?

1. If the Website is Hosted on EC2 (Apache/Nginx)

If your website is running on an **EC2 instance**, follow these steps to install an SSL certificate manually.

Steps to Install SSL on EC2

Step 1: Install Certbot (Let's Encrypt)

1. **Update packages & install Certbot**:
   bash
   CopyEdit
   sudo apt update && sudo apt install certbot -y
   (For Amazon Linux, use sudo yum install certbot)

Step 2: Install SSL for Apache or Nginx

2. **For Apache:**
   bash
   CopyEdit
   sudo certbot --apache -d example.com -d www.example.com
   For Nginx:
   bash
   CopyEdit
   sudo certbot --nginx -d example.com -d www.example.com
   - Follow the prompts to configure HTTPS.

Step 3: Automate SSL Renewal

3. Add a cron job for auto-renewal:
   bash
   CopyEdit
   sudo crontab -e
   Add this line:
   bash
   CopyEdit
   0 3 * * * certbot renew --quiet

This will check & renew SSL every day at 3 AM.

---

2. If the Website is Hosted on AWS Load Balancer (ALB/NLB)

For applications behind an **Application Load Balancer (ALB) or Network Load Balancer (NLB)**, you can use **AWS Certificate Manager (ACM)**.

Steps to Install SSL with ALB

Step 1: Request a Free SSL Certificate in ACM

1. Go to **AWS Certificate Manager (ACM)**.
2. Click **Request a Certificate** → Select **Public Certificate**.
3. Enter the domain (e.g., example.com).
4. Choose **DNS validation** (recommended) or **Email validation**.
5. Complete the validation process.

Step 2: Attach the SSL Certificate to ALB

1. Go to **EC2 → Load Balancers**.
2. Select your **ALB** → **Listeners**.
3. Edit the **HTTPS Listener (443)**.
4. Choose the SSL certificate from **ACM**.
5. Save and apply changes.

✅ **Now, all traffic to ALB will be encrypted using SSL.**

---

3. If the Website is Hosted on S3 with CloudFront

If you're hosting a **static website in an S3 bucket**, use **CloudFront** to enable SSL.

Steps to Install SSL with CloudFront

Step 1: Request a Certificate in ACM

- Go to **AWS Certificate Manager (ACM)**.
- Request an SSL certificate for example.com.
- Choose **DNS validation** and validate.

Step 2: Configure CloudFront

- Go to **CloudFront → Create a Distribution**.
- Select **Origin** as your S3 bucket.
- Under **Settings → SSL Certificate**, select **Custom ACM Certificate**.
- Enable **Redirect HTTP to HTTPS**.

✅ Now, your static website in S3 will be accessible via HTTPS.

---

4. If the Website is Hosted on API Gateway

For API Gateway-based applications, enable SSL with a **custom domain**.

Steps to Install SSL with API Gateway

1. Request an SSL certificate in **ACM**.
2. Go to **API Gateway → Custom Domain Names**.
3. Attach the SSL certificate from ACM.
4. Create a **mapping** for your API.
5. Update your DNS (Route 53) to point to the API Gateway.

✅ Your API will now be accessible securely via HTTPS.

Conclusion

✓ **For EC2 (Apache/Nginx)** → Use **Certbot (Let's Encrypt)**

✓ **For ALB/NLB** → Use **AWS Certificate Manager (ACM)**

✓ **For S3 Static Sites** → Use **CloudFront + ACM**

✓ **For API Gateway** → Use **ACM + Custom Domain**

Route 53:

1. Explain about Cloudfront and Route53.

◆ **What is Amazon CloudFront?**

Amazon **CloudFront** is a **Content Delivery Network (CDN)** that speeds up the delivery of static and dynamic web content globally.

🌍 **Key Features of CloudFront**

1. **Global Content Delivery** – Uses **Edge Locations** to cache content closer to users.
2. **Low Latency & High Speed** – Reduces response time by serving content from nearest edge locations.
3. **Security** – Supports **AWS Shield, WAF, SSL/TLS encryption**.
4. **Customizable Caching** – Control TTL (Time-to-Live) and cache behaviors.
5. **Origin Support** – Works with **S3, EC2, ALB, and On-Prem** servers.
6. **DDoS Protection** – Integrated with **AWS Shield**.

🛠 **How CloudFront Works?**

1. **User requests content** (e.g., an image or website).
2. **CloudFront checks edge cache** – If content is cached, it's served immediately.
3. **If not cached**, CloudFront fetches from **origin server** (e.g., S3, EC2).
4. **Caches content** in edge locations for future requests.

✅ **Example Use Case:** Hosting a **global website** with **fast image & video loading** using CloudFront with S3.

◆ **What is Amazon Route 53?**

Amazon **Route 53** is a **highly available and scalable DNS (Domain Name System) service** that connects domain names to AWS resources or external servers.

🌍 **Key Features of Route 53**

1. **Domain Name Registration** – Buy & manage domain names.
2. **DNS Routing** – Directs users to **EC2, S3, ELB, CloudFront, on-prem servers**.
3. **Health Checks & Failover** – Detects **unhealthy endpoints** and reroutes traffic.
4. **Traffic Routing Policies:**
   - **Simple Routing** – Single static IP or domain.
   - **Weighted Routing** – Distribute traffic based on weight.
   - **Latency-Based Routing** – Route users to the lowest-latency AWS region.
   - **Failover Routing** – Redirect to a backup instance if the primary fails.
   - **Geolocation Routing** – Route users based on their country.
   5. **Private DNS for VPCs** – Resolve domain names within a VPC.

🛠 **How Route 53 Works?**

1. **User enters domain name** (e.g., example.com).
2. **Route 53 resolves the domain** to an **IP address** using DNS records (A, CNAME, Alias).
3. **Routes traffic** to the nearest or most optimal AWS service.

✅ **Example Use Case:**

- Hosting a **multi-region website** using **Route 53 + CloudFront** to ensure **low-latency global access**.

◆ **CloudFront vs Route 53 - Key Differences**

Feature	CloudFront (CDN)	Route 53 (DNS)
Purpose	Caches & delivers content faster	Resolves domain names to IP addresses
Speed Improvement	Yes, by caching at edge locations	No, only resolves DNS faster
Security	AWS WAF, DDoS protection	DNS Failover, Health Checks
Works with	S3, EC2, ALB, APIs	Domains, Websites, Applications
Latency Optimization	Yes, serves from nearest edge location	Yes, using latency-based routing

2. Have you used Route53 and for what purpose have you used it?

Yes, I have worked with **Amazon Route 53** in multiple scenarios. Below are some key use cases where I have implemented Route 53:

◆ **Use Cases of Route 53**

✅ **1. Domain Name Registration & DNS Management**

- Registered a domain and managed its **DNS records** (A, CNAME, MX, TXT, etc.).
- Used **Alias records** to map domain names to AWS resources (e.g., **ALB, S3 static website, CloudFront**).

✅ **2. Routing Traffic to AWS Services**

- Configured **Simple Routing** for a single website hosted on an **EC2 instance**.
- Used **Latency-Based Routing** to direct users to the nearest AWS region for **global applications**.
- Implemented **Weighted Routing** to **gradually shift traffic** from an old application version to a new version.

✅ **3. High Availability & Disaster Recovery**

- Set up **Failover Routing** with **health checks** to redirect traffic to a standby EC2 instance when the primary fails.
- Created a **multi-region failover setup** where Route 53 switches traffic to another region if an entire AWS region goes down.

✅ **4. Private DNS for Internal VPC Communication**

- Configured **Private Hosted Zones** to resolve internal service names inside a **VPC without internet access**.
- Allowed applications in multiple VPCs to communicate using domain names instead of IP addresses.

✅ **5. API Gateway & Custom Domains**

- Mapped a custom domain (e.g., api.example.com) to an **API Gateway** endpoint using Route 53.
- Managed **SSL/TLS certificates** using ACM (AWS Certificate Manager) for HTTPS support.

◆ **Example Scenario: Hosting a Website with Route 53**

1. **Registered** a domain (e.g., mywebsite.com) in Route 53.
2. Created an **A record** pointing to an **Application Load Balancer (ALB)**.
3. Enabled **CloudFront** to cache website content and reduce latency.
4. Configured a **failover policy** to switch traffic to a **backup server** in case of failure.

Other Services:

1. Have you worked on EKS?

Yes, I have worked on **Amazon EKS (Elastic Kubernetes Service)** for deploying and managing containerized applications.

How I Used EKS:

1. **Cluster Setup & Management**
   - Created **EKS clusters** using **Terraform** and **eksctl**.
   - Managed node groups using **Managed Node Groups** and **Fargate for serverless workloads**.
2. **Deployment & Scaling**
   - Deployed microservices using **Helm charts & Kubernetes manifests (YAML)**.
   - Configured **Horizontal Pod Autoscaler (HPA)** to scale pods based on CPU/memory utilization.
   - Used **Cluster Autoscaler** to scale worker nodes dynamically.
3. **Networking & Security**
   - Configured **AWS ALB Ingress Controller** for external access.
   - Used **IAM Roles for Service Accounts (IRSA)** to securely grant AWS permissions to pods.
   - Applied **Network Policies** to restrict pod-to-pod communication.
4. **Logging & Monitoring**
   - Integrated **CloudWatch, Prometheus, Grafana, and Loki** for monitoring and logs.
   - Used **Fluent Bit** to collect container logs and push them to CloudWatch.

5. **CI/CD Integration**
    - Automated deployments using **Jenkins, ArgoCD, and GitHub Actions**.
    - Implemented **Blue/Green & Canary deployments** using **Istio & ALB routing**.
6. **Security & Best Practices**
    - Scanned container images using **Trivy & AWS ECR scanning**.
    - Enforced **Pod Security Policies & RBAC (Role-Based Access Control)**.

EKS helped me manage **scalable, highly available, and secure Kubernetes clusters** in AWS.

2. What is Cloud Formation?

AWS **CloudFormation** is an **Infrastructure as Code (IaC)** service that helps automate and manage AWS resources by defining them in **templates** (YAML or JSON).

Key Features of CloudFormation:

1. **Automated Infrastructure Deployment** – Deploys and configures AWS resources consistently.
2. **Version Control & Reusability** – Templates can be stored in Git and reused across multiple environments.
3. **Stack Management** – Groups AWS resources into **Stacks** for easy management and updates.
4. **Drift Detection** – Detects changes in AWS resources that were modified outside of CloudFormation.
5. **Rollback Support** – If deployment fails, CloudFormation rolls back to the last known good state.

How It Works:

1. **Write a CloudFormation Template** (YAML/JSON) defining AWS resources.
2. **Upload to AWS** and create a **CloudFormation Stack**.
3. **CloudFormation provisions resources** based on the template.

Example Use Cases:

✅ **Creating VPC, Subnets, Security Groups**

✅ **Deploying EC2, RDS, S3, Lambda, IAM Roles**

✅ **Automating Multi-Region Infrastructure**

✅ **Managing Kubernetes (EKS) Clusters**

Comparison with Terraform:

Feature	AWS CloudFormation	Terraform
Cloud-Specific	AWS-only	Multi-cloud (AWS, Azure, GCP)
Language	YAML/JSON	HCL (HashiCorp Configuration Language)
State Management	Managed by AWS	Requires external state storage (e.g., S3 + DynamoDB)
Modularization	Nested Stacks	Modules

CloudFormation is great for **AWS-only** environments where native **AWS integration and automation** are needed. 🚀

3. Have you worked with ECS.

Yes, I have worked with **Amazon ECS (Elastic Container Service)** for deploying and managing containerized applications in AWS.

How I Used ECS:

1. **Deployed Microservices** – Used ECS to deploy containerized applications using **Fargate** and **EC2 launch types**.
2. **Service Auto Scaling** – Configured auto-scaling based on CPU and memory usage.
3. **ECS with ALB (Application Load Balancer)** – Routed traffic to ECS services dynamically.
4. **CI/CD Integration** – Automated deployments using **Jenkins/GitHub Actions with ECS**.
5. **IAM Role Management** – Assigned least privilege IAM roles for ECS tasks.
6. **EFS Storage with ECS** – Attached persistent storage for stateful workloads.
7. **Logging & Monitoring** – Used **CloudWatch Logs, X-Ray, and Prometheus/Grafana** for monitoring container performance.

Challenges Faced & Solutions:

- **Task Scheduling Issues** → Used ECS capacity providers for optimized scheduling.

- **Networking & Security** → Configured VPC, Security Groups, and IAM roles correctly.
- **Resource Optimization** → Used **Fargate Spot Instances** for cost savings.

4. Have you worked on SNS and SQS.

Yes, I have worked with **SNS (Simple Notification Service)** and **SQS (Simple Queue Service)** for building decoupled, scalable, and event-driven architectures in AWS.

How I Used SNS & SQS:

Feature	SNS (Pub/Sub)	SQS (Message Queue)
Use Case	Send notifications to multiple subscribers (email, Lambda, HTTP, SQS, etc.)	Store and process messages asynchronously between distributed systems
Integration	Used SNS for sending alerts from CloudWatch, Lambda, and CI/CD pipelines	Used SQS as a queue for processing tasks asynchronously from microservices
Event-Driven Architecture	Sent application notifications (e.g., new order placed) to multiple subscribers (email, SMS, Lambda)	Handled high-throughput event processing using SQS with Lambda workers
Security	Configured **SNS access policies** to restrict subscriptions	Used **dead-letter queues (DLQ)** for failed message processing
Reliability	SNS automatically retries failed deliveries	SQS guarantees message durability and **FIFO queues** ensure order
Cost Optimization	Used SNS **fan-out pattern** to minimize API calls	Used **long polling** to reduce unnecessary API requests

Example Scenarios I Worked On:

1. **CloudWatch Alarm Notifications:** SNS triggered an email/SMS alert when a server's CPU exceeded a threshold.
2. **CI/CD Pipeline Notifications:** SNS sent Slack notifications on Jenkins build failures.
3. **Asynchronous Processing with SQS:**
   - Microservices sent events to SQS (e.g., order processing).
   - A Lambda function consumed messages and updated the database.
4. **SNS → SQS Integration:**
   - SNS published an event.
   - SQS subscribed to SNS and queued messages for further processing.

General:

1. Explain tags. Why we need them?

Tags in AWS are **key-value pairs** that help organize, manage, and track AWS resources efficiently. They provide **metadata** about resources and help in cost allocation, access control, and automation.

Purpose	Description
Resource Organization	Helps categorize AWS resources (e.g., by project, environment, department).
Cost Allocation	Assigns costs to specific projects, teams, or business units using AWS Cost Explorer.
Access Control	Enables fine-grained IAM permissions based on resource tags.
Automation	Allows automated actions (e.g., delete unused resources, trigger Lambda functions).
Compliance & Security	Helps enforce governance by tracking resources and ensuring compliance.
Operational Management	Makes it easier to identify and manage instances, volumes, and networks.

2. What AWS services you have used ..what challenges you have faced in that

Service	Purpose
EC2 (Elastic Compute Cloud)	Virtual servers for hosting applications
S3 (Simple Storage Service)	Object storage for backups, logs, and static files
IAM (Identity and Access Management)	Managing user permissions and security
RDS (Relational Database Service)	Managed database service for PostgreSQL, MySQL, etc.
DynamoDB	NoSQL database for scalable applications
VPC (Virtual Private Cloud)	Secure networking setup for cloud resources
Route 53	DNS management and domain routing
CloudFront	Content delivery network (CDN) for faster access
Auto Scaling	Scaling EC2 instances based on demand
Elastic Load Balancer (ELB)	Distributes traffic among multiple EC2 instances
Lambda	Serverless computing for event-driven applications
CloudFormation	Infrastructure as Code (IaC) for automating deployments
Terraform	Automating AWS infrastructure provisioning
EBS (Elastic Block Store)	Persistent storage for EC2 instances
EFS (Elastic File System)	Shared file storage across multiple EC2s
CloudWatch	Monitoring and logging service
SNS (Simple Notification Service)	Sending notifications for events
SQS (Simple Queue Service)	Message queuing for decoupled applications
Secrets Manager	Secure storage of secrets, passwords, and API keys
CodePipeline & CodeDeploy	CI/CD automation for software deployment
S3 Glacier	Long-term archival storage
VPC Peering & Transit Gateway	Connecting multiple VPCs across regions/accounts

Service	Challenge Faced	Solution Implemented
EC2	Connection timeout issues due to security groups	Checked **SG rules, NACL, and route tables** to allow access
S3	Public access mistakenly enabled, causing security risk	Enabled **Bucket Policies & IAM Policies** for restricted access
IAM	Overly permissive IAM roles leading to security gaps	Applied **Least Privilege Principle** & used **IAM Policies** wisely
RDS	High database costs due to unnecessary always-on instances	Used **Aurora Serverless** for auto-scaling & **RDS Reserved Instances**
DynamoDB	High read/write costs in provisioned mode	Switched to **On-Demand Mode** for cost optimization
VPC	Private EC2 needed internet access for updates	Used **NAT Gateway** instead of IGW for secure internet access
Auto Scaling	Scaling up/down was too slow	Adjusted **CloudWatch Alarms & Cooldown Period** for faster scaling
Lambda	Cold start issues causing high response times	Pre-warmed **Lambda with Provisioned Concurrency**
CloudFormation	Stack failed due to dependency issues	Used **DependsOn** & **Resource ordering** to fix dependencies
Terraform	Terraform state conflicts when multiple users worked	Used **S3 + DynamoDB for state locking**

Service	Challenge Faced	Solution Implemented
CloudWatch	Large log storage costs	Set up **Log Retention Policies** and used **S3 Lifecycle Rules**
Route 53	Slow DNS propagation	Used **Failover Routing** and **Latency-Based Routing**
EFS	High costs due to unmonitored usage	Used **Lifecycle Management** to move infrequent files to lower-cost storage
Secrets Manager	Secret rotation caused app downtime	Used **Lambda for automatic secret rotation** without service disruption

3. How to build a secure and highly available application?

1. Security Best Practices

✅ **Identity & Access Management (IAM)**

- Follow **Least Privilege Principle** (Give only necessary permissions)
- Use **IAM Roles instead of Access Keys** for EC2, Lambda, and other AWS services
- Enable **MFA (Multi-Factor Authentication)** for users

✅ **Network Security**

- Use **VPC with Public & Private Subnets**
- Place databases in **private subnets** (No direct internet access)
- Use **Security Groups & NACLs** to restrict traffic
- Implement **AWS WAF** to block malicious traffic

✅ **Data Encryption**

- Encrypt data at **rest** using **AWS KMS** (EBS, S3, RDS)
- Encrypt data **in transit** using **TLS/SSL**
- Use **AWS Secrets Manager** for storing sensitive credentials

✅ **Monitoring & Logging**

- Enable **CloudTrail** for tracking API actions
- Use **CloudWatch Alarms & Logs** for real-time monitoring
- Enable **AWS Config** to track infrastructure changes

---

2. High Availability & Fault Tolerance

✅ **Multi-AZ Deployment**

- Deploy application servers across **multiple Availability Zones (AZs)**
- Use **RDS Multi-AZ** for database redundancy

✅ **Auto Scaling & Load Balancing**

- Use **Elastic Load Balancer (ALB/NLB)** to distribute traffic
- Enable **Auto Scaling Groups** to handle peak traffic automatically

✅ **Multi-Region Deployment (Disaster Recovery)**

- Use **Route 53 Latency-Based Routing** to serve users from the nearest region
- Deploy **read replicas in another region** for critical databases
- Implement **AWS Backup** for automated backups

---

3. Performance Optimization

✅ **Caching & Acceleration**

- Use **CloudFront (CDN)** to cache static assets globally
- Use **ElastiCache (Redis/Memcached)** for database query caching

✅ **Optimized Compute & Storage**

- Use **EC2 Spot Instances** for cost-effective workloads
- Use **S3 Intelligent-Tiering** to store infrequently accessed data
- Enable **EBS GP3 volumes** for better performance at lower cost

---

4. Disaster Recovery Strategy

✅ **Backup & Restore** (Cost-Effective, Slower)

- Use **AWS Backup** for regular snapshots of EC2, RDS, EBS

- Store backups in **Amazon S3 Glacier** for long-term storage

✅ **Pilot Light** (Faster Recovery)

- Keep **minimal infrastructure running** in a secondary region
- Replicate **database & storage** with **AWS DMS & S3 Replication**

✅ **Hot Standby** (Best for Critical Applications)

- Deploy a **fully functional standby environment** in another region
- Use **Route 53 Failover Routing** to switch traffic during failure

---

Conclusion: AWS Services for Secure & HA Architecture

Category	AWS Services Used
Security	IAM, KMS, AWS WAF, Security Groups, NACLs
Availability	ALB, Auto Scaling, Multi-AZ RDS, Route 53
Performance	CloudFront, ElastiCache, EC2 Spot Instances
Monitoring	CloudWatch, CloudTrail, AWS Config
Disaster Recovery	S3 Glacier, AWS Backup, Route 53 Failover

4. How ull decide the infrastructure to be created?

To decide the infrastructure to be created, first analyze the **application requirements**, including compute (EC2, Lambda, ECS), storage (S3, EBS, EFS), and database (RDS, DynamoDB, ElastiCache) needs. Design a **VPC architecture** with public and private subnets, ensuring security using **IAM, Security Groups, and NACLs**. Implement **scalability and high availability** with **Auto Scaling Groups and Load Balancers** while optimizing costs using **Spot Instances, S3 storage classes, and VPC Endpoints**. Enable **monitoring and logging** through **CloudWatch, CloudTrail, and AWS Config**. For **disaster recovery**, use **Multi-AZ deployments, Route 53 failover, and AWS Backup**. Finally, ensure **cost efficiency** by choosing the right **compute, networking, and storage strategies** while maintaining security and performance.

5. Have you automated AWS infrastructure?

Yes, I have automated AWS infrastructure using **Terraform** and **AWS CloudFormation**. Terraform helps in **Infrastructure as Code (IaC)**, allowing version control, modularization, and reusability of resources. I have written **Terraform scripts** to provision **VPCs, EC2 instances, Auto Scaling Groups, Load Balancers, RDS, IAM roles, S3 buckets, and security policies**. For automation, I have used **CI/CD pipelines (Jenkins, GitHub Actions)** to deploy infrastructure changes efficiently. Additionally, I have used **Ansible** for post-deployment configuration management and **AWS Lambda** for serverless automation tasks. To ensure governance, I have implemented **Terraform state management using S3 and DynamoDB** for locking.

6. Have you worked on cost optimization?

Yes, I have worked on **cost optimization** in AWS by implementing various strategies:

1. **Compute Optimization** – Used **Auto Scaling Groups (ASG)** to scale EC2 instances dynamically, leveraged **Spot Instances** for non-critical workloads, and used **AWS Lambda** for event-driven serverless computing to reduce always-on costs.
2. **Storage Optimization** – Implemented **S3 Lifecycle Policies** to move data to lower-cost storage classes (S3 Glacier, Intelligent-Tiering) and used **EBS Volume Right-Sizing** to avoid over-provisioning.
3. **Networking Optimization** – Used **VPC Endpoints** to reduce data transfer costs and enabled **AWS Savings Plans** for predictable workloads.
4. **Database Optimization** – Deployed **RDS with Reserved Instances**, enabled **read replicas** for performance efficiency, and moved **less critical data to DynamoDB or S3** to minimize database costs.
5. **Monitoring & Governance** – Used **AWS Cost Explorer and Budgets** to track usage and set alerts for unexpected cost spikes. Also, used **AWS Trusted Advisor** to identify unused or underutilized resources.

By implementing these strategies, I was able to significantly reduce unnecessary AWS costs while maintaining high performance and availability.

7. Have you worked with automating deployment in AWS?

Yes, I have worked on **automating deployments in AWS** using various tools and services. Here's how I approached it:

1. CI/CD Pipeline for Deployment

I automated deployments using **Jenkins, GitHub Actions, and AWS CodePipeline** with the following steps:

- **Code Commit:** Developers push code to **GitHub/GitLab/Bitbucket**.
- **Build & Test:** Jenkins or AWS CodeBuild compiles the code and runs tests.
- **Artifact Storage:** The built artifact (JAR, Docker image) is stored in **Amazon S3, ECR, or Nexus**.
- **Security Scans:** Used **SonarQube, Trivy, and OWASP ZAP** for SAST, SCA, and DAST security checks.
- **Deployment:**
  - **For EC2-based apps:** Used **Ansible & Terraform** to deploy new instances.
  - **For containerized apps:** Used **ECS/EKS with Fargate** and **Helm for Kubernetes deployments**.
  - **For serverless apps:** Used **AWS Lambda with API Gateway** and automated deployment using **AWS SAM or Terraform**.

2. Infrastructure as Code (IaC)

- Used **Terraform and CloudFormation** to define and provision AWS infrastructure.
- Managed different environments (**dev, stage, prod**) using **Terraform workspaces**.

3. Zero Downtime Deployment

- Used **Blue/Green and Rolling deployments** in ECS, EKS, and Elastic Beanstalk to avoid downtime.
- Implemented **AWS ALB Weighted Target Groups** to gradually shift traffic to the new version.

4. Monitoring & Alerts

- Used **CloudWatch, Prometheus, and Grafana** to monitor deployments.
- Integrated **Slack & Email notifications** for deployment failures.

This approach ensured **fast, secure, and automated** deployments with **minimal manual intervention**.

Scenario based:

1. I want to copy files from S3 to EC2 how do you do it?

To **copy files from an S3 bucket to an EC2 instance**, follow these steps:

1. Install AWS CLI on EC2

First, ensure that **AWS CLI** is installed on your EC2 instance.

If it's not installed, install it using:

```
sudo apt update && sudo apt install awscli -y  # For Ubuntu/Debian
sudo yum install awscli -y  # For Amazon Linux/RHEL
```

2. Attach IAM Role to EC2 (Recommended Approach) 🔐

Instead of using access keys, attach an **IAM Role** to your EC2 instance with **AmazonS3ReadOnlyAccess** or a custom policy allowing S3 read access.

- Go to **AWS Console** → EC2 → Select your instance → Actions → Security → Modify IAM Role.
- Attach an IAM role with **S3 read permissions**.

3. Copy Files from S3 to EC2 Using AWS CLI 📂

Once the IAM role is attached, use the following command to copy files:

a) Copy a Single File

```
aws s3 cp s3://my-bucket-name/myfile.txt /home/ec2-user/
```

b) Copy an Entire Folder

```
aws s3 cp s3://my-bucket-name/my-folder/ /home/ec2-user/my-folder --
recursive
```

c) Sync S3 Bucket to EC2 (For Large Data)

```
aws s3 sync s3://my-bucket-name /home/ec2-user/my-bucket-data
```

4. Verify the Files on EC2

List the copied files:

```
ls -l /home/ec2-user/
```

✅ **Best Practices**

✓ Use **IAM roles** instead of static credentials for security.

✓ Use `--recursive` for copying folders.

✓ Automate the process using **cron jobs** or **Lambda functions** if needed.

2. If I modify a file in EC2 it should reflect in S3 bucket. How to configure this?

1. Attach an IAM Role to EC2 (Recommended Approach) 🔐

Instead of using access keys, attach an **IAM Role** with **write permissions** to your EC2 instance:

- Go to **AWS Console → EC2 →** Select your instance
- Click **Actions → Security → Modify IAM Role**
- Attach an IAM role with **AmazonS3FullAccess**.

---

2. Install AWS CLI on EC2 (If Not Installed)

Run:

```
sudo apt update && sudo apt install awscli -y  # Ubuntu/Debian
sudo yum install awscli -y  # Amazon Linux/RHEL
```

---

3. Sync Files from EC2 to S3 Using AWS CLI

To manually upload a file from EC2 to S3:

```
aws s3 cp /home/ec2-user/myfile.txt s3://my-bucket-name/
```

To upload an entire directory:

```
aws s3 sync /home/ec2-user/my-folder/ s3://my-bucket-name/
```

---

4. Automate Sync When Files Are Modified (Using inotify + S3 Sync)

Install **inotify-tools** to detect file changes and trigger S3 sync:

```
sudo apt install inotify-tools -y  # Ubuntu/Debian
sudo yum install inotify-tools -y  # Amazon Linux/RHEL
```

Create a script **/home/ec2-user/s3_sync.sh** to watch a directory and sync changes to S3:

```
#!/bin/bash
WATCH_DIR="/home/ec2-user/my-folder"
BUCKET_NAME="my-bucket-name"
inotifywait -m -r -e modify,create,delete "$WATCH_DIR" --format '%w%f' |
while read FILE
do
  echo "File changed: $FILE"
  aws s3 sync "$WATCH_DIR" "s3://$BUCKET_NAME/" --delete
done
```

Give execute permission:

```
chmod +x /home/ec2-user/s3_sync.sh
```

Run the script in the background:

```
nohup /home/ec2-user/s3_sync.sh &
```

---

5. Set Up a Cron Job (Alternative to inotify)

Instead of inotify, you can schedule periodic syncing using **cron**:

```
crontab -e
```

Add this line to run sync every 5 minutes:

```
*/5 * * * * aws s3 sync /home/ec2-user/my-folder/ s3://my-bucket-name/ --
delete
```

✅ **Best Practices**

✓ **Use IAM roles** instead of access keys for security

✓ **Enable S3 Versioning** to keep track of changes

✓ **Use --delete option** to remove deleted files from S3

✓ **Use EventBridge + Lambda** for event-driven syncing

3. My application inside EC2 should access internet to download resources but we should not be able to access the application except it's URL.

✅ **Solution: Allow EC2 to Access the Internet Without Exposing It**
Your requirement is:
✓ **EC2 instance should access the internet** (to download packages, updates, etc.).
✓ **EC2 should NOT be directly accessible** (except via its application URL).

◆ **Solution Architecture**
1. **Use a Private Subnet**: Place the EC2 instance inside a **private subnet** (no direct public access).
2. **Use a NAT Gateway**: Allow outbound internet access from the private subnet via **NAT Gateway**.
3. **Expose the Application via an ALB**: The EC2 instance should only be reachable through an **Application Load Balancer (ALB)**.
4. **Restrict Security Group Rules**: Only allow incoming traffic from the **ALB**.

◆ **Step-by-Step Implementation**
1. VPC & Subnet Configuration
- **Public Subnet** (10.0.1.0/24): Hosts the **NAT Gateway** & **ALB**.
- **Private Subnet** (10.0.2.0/24): Hosts the **EC2 instance**.

2. Configure a NAT Gateway (For Internet Access)
- Deploy a **NAT Gateway** in the **Public Subnet**.
- Attach an **Internet Gateway (IGW)** to the **VPC**.
- Route the **Private Subnet's traffic** (0.0.0.0/0) to the **NAT Gateway**.

3. Set Up an ALB (Application Load Balancer)
- Deploy an **ALB** in the **Public Subnet**.
- Register the **EC2 instance** as a **target** behind the ALB.
- Allow incoming traffic **only on ALB's listener port (e.g., 80 or 443 for HTTPS)**.

4. Security Group Rules

Component	Inbound Rules	Outbound Rules
ALB Security Group	Allow HTTP/HTTPS from the internet	Allow to EC2's **app port** (e.g., 8080)
EC2 Security Group	Allow from **ALB only** (not public)	Allow all outgoing (to internet via NAT)
NAT Gateway	No inbound (managed by AWS)	Allow outbound (0.0.0.0/0)

◆ **Outcome** ✅
✓ **EC2 can access the internet** (via NAT Gateway).
✓ **Only ALB can access EC2**, preventing direct access.
✓ **Users can only reach the application via ALB URL**.

4. AMI I want to use in my friend's account also and also on different region.

✅ **Solution: Share & Copy an AMI Across AWS Accounts & Regions**
Your requirement:
✓ **Use the same AMI in your friend's AWS account.**
✓ **Use the same AMI in a different AWS region.**

◆ **Scenario 1: Share an AMI with Another AWS Account**
If your **friend's AWS account** is in the **same region**, you can **share the AMI** instead of copying it.
Steps to Share AMI Across AWS Accounts
1. **Find the AMI ID**
   - Go to **EC2 Dashboard → AMIs**
   - Note down the **AMI ID** (e.g., ami-1234567890abcdef)
2. **Modify Permissions**
   - Select your AMI → Click **Modify Image Permissions**
   - Add your **friend's AWS Account ID**
   - Click **Save**
3. **Launch Instance in Friend's Account**
   - Your friend can now **see and use the shared AMI** under **Private AMIs** in EC2.

- They can **launch an instance** using this AMI in the same region.

◆ **Scenario 2: Copy AMI to Another Region**
If you want to use the AMI in **another AWS region**, you must **copy it** first.
Steps to Copy AMI to Another AWS Region
1. **Go to EC2 Dashboard → AMIs**
2. **Select the AMI → Click "Actions" → "Copy AMI"**
3. **Choose the Destination Region**
   - Select the **target region** where you want the AMI to be available.
   4. **Start the Copy Process**
   - Click **Copy** and wait for the process to complete.
   5. **Launch an Instance in the New Region**
   - In the target region, go to **AMIs**, select the copied AMI, and launch an instance.

◆ **Scenario 3: Copy AMI to Friend's AWS Account in Another Region**
Since AMIs **cannot be directly shared across regions**, you must **copy it to the target region first, then share it**.
Steps:
1. **Copy AMI to the target region** (Follow **Scenario 2**).
2. **Share the copied AMI** with your friend's AWS account (Follow **Scenario 1**).
3. **Your friend launches an instance** in the target region using the shared AMI.

✅ **Final Outcome**
✓ AMI can be **used in another AWS account**.
✓ AMI can be **used in a different region**.
✓ AMI can be **copied across regions and shared further**.

5. Deployed app in private subnet and how do we access it from internet? (Hint: Bastion host will not come in picture)

✅ **Solution: Access a Private Subnet Application from the Internet**
Your requirement:
✓ **Application is deployed in a private subnet** (No direct internet access).
✓ **You need to access it from the internet** (without a Bastion Host).

◆ **Solution Approach: Use an Application Load Balancer (ALB)**
A **private subnet** does **not** have a public IP, so we cannot directly access the application. Instead, we use an **Application Load Balancer (ALB)** in a **public subnet** to expose the application securely.

◆ **Step-by-Step Implementation**
1. VPC & Subnet Configuration
- **Public Subnet**: Contains **Application Load Balancer (ALB)**.
- **Private Subnet**: Contains **EC2 instance (application server)**.
- **NAT Gateway (Optional)**: If the application needs outbound internet access.

2. Deploy an ALB in a Public Subnet
- ALB will receive traffic from the **internet** and forward it to the **private EC2 instance**.
- Attach a **Security Group** that allows **inbound traffic** on **port 80/443** from the internet.

3. Configure Target Group
- Create a **Target Group** and **register the private EC2 instance**.
- ALB will route requests to the **EC2 instance in the private subnet**.

4. Update Security Groups

Component	Inbound Rules	Outbound Rules
ALB Security Group	Allow HTTP/HTTPS (80/443) from 0.0.0.0/0	Forward to EC2 (Application Port)
EC2 Security Group	Allow traffic **only from ALB Security Group**	Allow outbound traffic as needed

5. Route Traffic via DNS (Optional)
- Point a **custom domain** (e.g., app.example.com) to the ALB's **DNS Name** using **Route 53**.

◆ ✅ **Outcome**

✓ **Application is accessible from the internet via ALB.**

✓ **EC2 in private subnet remains secure (no public IP).**

✓ **Only ALB can communicate with the application (restricted access).**

6. I have the key pair but while connecting to ec2 I'm getting time out error, can u let me know what was the issue here

🔹 Possible Reasons & Solutions

1. Security Group Issue

🔍 **Check:** Ensure your **Security Group (SG) allows inbound SSH (Port 22)** from your IP.

✅ **Fix:** In AWS Console:

- Go to **EC2 → Security Groups**.
- Select the **EC2 Security Group**.
- Add **Inbound Rule**:
  - **Protocol**: TCP
  - **Port**: 22
  - **Source**: **Your IP (My IP)** or 0.0.0.0/0 (not recommended for production).

---

2. EC2 is in Private Subnet (No Public IP)

🔍 **Check:** If **Public IP is missing**, it's in a **private subnet**.

✅ **Fix: Use a Bastion Host** in a public subnet.

- Or **Associate an Elastic IP** to your EC2 instance.

---

3. Network ACL (NACL) Blocking Traffic

🔍 **Check:**

- Go to **VPC → Network ACLs** and find the NACL attached to the subnet.
- Ensure **Inbound Rules** allow:
  - **Port 22 from your IP** (0.0.0.0/0 for testing).
- Ensure **Outbound Rules** allow:
  - **Ephemeral ports (1024-65535)**.

✅ **Fix:** Update the NACL to allow SSH traffic.

---

4. Route Table Misconfiguration

🔍 **Check:** If it's a **public instance**, ensure there is a **default route to the Internet Gateway (IGW)**.

✅ **Fix:**

- Update the **Route Table** for public subnet:
  - Destination: 0.0.0.0/0 → Target: Internet Gateway.
- For **private instances**, use a **Bastion Host**.

---

5. EC2 Instance Issue (Boot Failure)

🔍 **Check:** If the instance **failed to boot**, reboot it from AWS Console.

✅ **Fix:** Try stopping and starting the instance.

---

6. VPN or Firewall Blocking SSH

🔍 **Check:** If connecting from an **office or corporate network**, your **firewall might be blocking SSH**.

✅ **Fix:** Try connecting using **mobile hotspot** or **VPN**.

---

7. Incorrect Key Pair Usage

🔍 **Check:**

- Run: chmod 400 my-key.pem
  ssh -i my-key.pem ec2-user@<public-ip>
- If using Windows, try Git Bash or PuTTY (convert .pem to .ppk).

✅ **Fix:** Ensure the **correct private key** is used.

🔹 Final Steps

1. **Check Security Group (Port 22).**
2. **Ensure EC2 is in a Public Subnet (or use Bastion).**
3. **Verify NACL allows SSH.**
4. **Ensure correct Route Table (Internet Gateway for public).**
5. **Try connecting via VPN or different network.**

7. Ebs & EC2 has to be in same region right but i want to launch EC2 from another region from the same ebs what ull do

✅ Solution: Copy EBS Volume to Another Region

Since EBS volumes are **tied to a specific region**, you need to **create an AMI from the volume** and then copy it to the target region.

◆ **Steps to Move EBS to Another Region**

1. Create a Snapshot of the EBS Volume

- Run the following command:
  ```
  aws ec2 create-snapshot --volume-id vol-xxxxxxxx --description "Snapshot for cross-region migration"
  ```
- Note down the **Snapshot ID** from the output.

---

2. Copy the Snapshot to Another Region

- Run:
  ```
  aws ec2 copy-snapshot --source-region <source-region> --source-snapshot-id snap-xxxxxxxx --destination-region <destination-region> --description "Copy of snapshot"
  ```
- Replace `<source-region>` and `<destination-region>` accordingly.
- This process **replicates the snapshot** to the new region.

---

3. Create a New EBS Volume from the Copied Snapshot

- Run:
  ```
  aws ec2 create-volume --snapshot-id snap-yyyyyyyy --availability-zone <destination-az>
  ```
- Ensure the **Availability Zone (AZ)** is the same as the EC2 instance you plan to launch.

---

4. Launch an EC2 Instance in the New Region

- Create a new EC2 instance in the **destination region**.
- Attach the newly created **EBS volume** to this EC2 instance.

🔹 **Alternative Approach:** Create an AMI and Copy to Another Region

If the **EBS volume contains an OS** (root volume), it's better to **create an AMI** instead of just copying the volume.

1. **Create an AMI from the existing EC2 instance**:
   ```
   aws ec2 create-image --instance-id i-xxxxxxxx --name "MyAMI" --description "AMI for cross-region"
   ```
2. **Copy the AMI to another region**:
   ```
   aws ec2 copy-image --source-region <source-region> --source-image-id ami-xxxxxxxx --destination-region <destination-region> --name "MyAMI-Copy"
   ```
3. **Launch a new EC2 instance from the copied AMI in the destination region**.

🔹 ✅ Final Outcome

✔ EBS data is **migrated to another region**.

✔ EC2 instance can now **use the same data in the new region**.

✔ This works for **both data-only volumes & root volumes** (via AMI).

8. How ull connect to ec2 if we lose keypair what ull do?

If you lose the **private key** of an EC2 instance, you won't be able to connect via SSH. However, you can **recover access** using the following methods:

🔹 Solution 1: Use EC2 Instance Connect (Only for Amazon Linux 2 & Ubuntu)

If the instance supports **EC2 Instance Connect**, follow these steps:

1. Go to the **AWS Console** → EC2 → Instances
2. Select the **instance**
3. Click **Connect** → **EC2 Instance Connect**
4. Click **Connect** to open a web-based terminal

✓ Now you can **generate a new key pair** and replace the old key inside `/home/ec2-user/.ssh/authorized_keys`.

🔹 Solution 2: Manually Replace the Key (For Any OS)

If **EC2 Instance Connect is not supported**, follow this process:

1. Create a New Key Pair

- Go to **AWS Console → EC2 → Key Pairs**
- Click **Create Key Pair**
- Download the **.pem** file (**NewKey.pem**)

---

2. Stop the EC2 Instance

- Go to **AWS Console → EC2 → Instances**
- Select your instance and click **Stop** (Do NOT Terminate)

---

3. Detach the Root Volume

- Go to **EC2 → Volumes**
- Find the **root volume** (attached to your EC2)
- Click **Actions → Detach Volume**

---

4. Attach the Volume to a Temporary EC2 Instance

- **Launch a new temporary EC2 instance** in the same AZ
- Attach the **detached volume** as a secondary volume
- SSH into the new temporary instance:
  ```
  ssh -i TemporaryKey.pem ec2-user@<temp-instance-ip>
  ```
- Mount the attached volume:
  ```
  sudo mkdir /mnt/temp-volume
  sudo mount /dev/xvdf1 /mnt/temp-volume  # Adjust based on device name
  ```

---

5. Add the New Key to Authorized Keys

- Replace the lost key with the new one:
  ```
  sudo nano /mnt/temp-volume/home/ec2-user/.ssh/authorized_keys
  ```
- Add the public key of the new key pair (NewKey.pem)
- Save and exit

---

6. Detach and Reattach the Volume

- Unmount the volume: `sudo umount /mnt/temp-volume`
- Detach it from the **temporary EC2 instance**
- Reattach it as the **root volume** of the original EC2 instance

---

7. Start the Original EC2 Instance

- Go to **EC2 → Instances**
- Select your original instance and **start** it
- Now, you can SSH into the instance using the new key:
  ```
  ssh -i NewKey.pem ec2-user@<instance-ip>
  ```

✅ Alternative Approaches

✓ **Recover from a Backup (If Available)**

✓ **Use AWS Systems Manager (SSM) Session Manager** (If configured)

◆ **Summary**

- **If EC2 Instance Connect is available** → Use it
- **If you have another EC2 instance** → Mount and replace keys
- **If SSM Session Manager is enabled** → Access via AWS Console
- **If backups exist** → Restore from a snapshot

9. I want to scaleout my EC2 during some peak hrs..how you are gonna do it. Based on what criteria

Scaling Out EC2 During Peak Hours (Scheduled & Dynamic Scaling)

To scale out EC2 instances **during peak hours**, you can use **AWS Auto Scaling** with either:

1. **Scheduled Scaling** – Scale at specific times.
2. **Dynamic Scaling** – Scale based on real-time metrics like CPU, memory, requests, etc.

🔹 Solution 1: Scheduled Auto Scaling (Best for Known Peak Hours)

If you know the peak hours (e.g., **9 AM to 6 PM**), you can schedule EC2 scaling.

Steps:

1. **Create an Auto Scaling Group (ASG)** for your EC2 instances.
2. **Define a Scaling Policy** to adjust the number of instances.
3. **Create a Scheduled Action** in AWS Console.

✓ This **increases instances** at **9 AM**

✓ You can also create a **scale-in** rule for **6 PM**

🔹 Solution 2: Dynamic Auto Scaling (Based on Metrics)

If your traffic is **unpredictable**, scale EC2 based on **CPU usage, memory, or request count**.

Steps:

1. **Create an Auto Scaling Group (ASG)**.
2. **Attach a Target Tracking Scaling Policy** based on CPU Utilization.
3. Example: **Scale Out when CPU > 70%**

✓ AWS **monitors CPU usage**

✓ Adds instances when CPU **exceeds 70%**

✓ Removes instances when CPU **drops below 70%**

🔹 Scaling Criteria (Key Metrics to Use)

- **CPU Utilization** (e.g., scale when CPU > 70%)
- **Memory Usage** (requires CloudWatch custom metrics)
- **Request Count per Target** (useful for web servers)
- **Network In/Out** (for data-heavy applications)
- **Latency (Response Time)** (scale when app slows down)

🔹 Summary

✓ Use **Scheduled Scaling** if peak hours are **fixed**

✓ Use **Dynamic Scaling** if demand is **unpredictable**

✓ Configure **Auto Scaling Group (ASG)** with **CPU, memory, or request-based scaling**

10. I have scaled out my EC2 & after the peak time will it scale down automatically?

Yes, your EC2 instances **will scale down automatically IF** you have set up **dynamic scaling policies** correctly in **AWS Auto Scaling**.

◆ **When Will EC2 Scale Down?**

1. **Scheduled Scaling** → If you used a **scheduled action**, AWS will reduce instances at the defined **off-peak time**.
2. **Dynamic Scaling** → If you used **CPU, memory, or request-based scaling**, EC2 will scale down **automatically** when the traffic/load decreases.

◆ **How to Ensure Auto Scale-Down?**
✓ **Ensure a Scale-In Policy is Set**
✓ **Check Min & Max Instance Limits in ASG**
✓ **Set Cooldown Periods Properly**

Example: Scale Down When CPU < 30%
If you've set **CPU-based Auto Scaling**, AWS will remove EC2 instances when CPU **drops below 30%**.
✓ **EC2 will terminate extra instances** when CPU **drops below 30%**.
✓ **Min size in ASG** prevents all instances from shutting down.

◆ **Key Things to Check**
1. **Does ASG Have a Scale-In Policy?** → Needed for automatic scale-down.
2. **What is the Minimum Instance Count?** → AWS won't go below this.
3. **Are Cooldown Timers Configured?** → Prevents unnecessary scaling actions.
4. **Check Auto Scaling Activity Logs** → To debug any issues.

11. When the new EC2 launches from asg will be of same instance type/config of existing one?? Where we will mention all those

✅ **Yes**, new EC2 instances launched by **Auto Scaling Group (ASG)** will have the same **instance type, AMI, and other configurations** as the existing ones.
✅ **Configuration is defined in the Launch Template or Launch Configuration** (used by ASG).

🔹 Where to Define EC2 Configuration for ASG?

You define the **instance type, AMI, key pair, security groups, etc.** in:
1. **Launch Template** (Recommended)
2. **Launch Configuration** (Deprecated)

🔹 Key Takeaways

✓ **New EC2 instances launched by ASG will be identical**
✓ **Configuration is defined in the Launch Template**
✓ **Ensure the Launch Template has the correct instance type, AMI, and security groups**
✓ **For changes, create a new template version and update ASG**

12. I want to move my file to another class after 7 days how ull do it.

You can achieve this using **S3 Lifecycle Policies** to automatically move (transition) objects from one storage class to another after 7 days.

◆ **Steps to Move Files After 7 Days**
1. **Create an S3 bucket (if not already created)**
2. **Define a Lifecycle Rule** to transition objects after 7 days
3. **Specify the destination storage class** (e.g., Standard → Glacier)
4. **Apply the rule to the bucket**

Unlike S3, **Amazon EFS (Elastic File System)** does not use lifecycle policies like S3 but supports **Lifecycle Management** for transitioning files to **Infrequent Access (IA) storage class** to save costs.

◆ **Steps to Move Files to Infrequent Access After 7 Days**
1. **Enable Lifecycle Management for your EFS File System**
2. **Define the age for automatic transition (e.g., 7 days)**
3. **EFS will automatically move files that haven't been accessed in 7 days**
4. **When accessed again, the file moves back to Standard storage**

13. Y we store a file in particular class for 30 days

✅ **Aligns with AWS billing cycle** (monthly)
✅ **Balances cost and accessibility**
✅ **Follows access patterns** (logs, backups, reports)
✅ **Meets compliance and business retention policies**
✅ **Avoids unnecessary API calls and high retrieval costs**

14. How would you design a highly available and scalable application architecture on AWS?

◆ **Real-World VPC Design Example (AWS Best Practices)**

✅ **Scenario:**

We are designing a **highly available** AWS infrastructure with:

- **3-Tier Architecture** (Web, Application, Database layers)
- **Multiple Availability Zones (AZs)**
- **Public & Private Subnets**
- **VPC Peering / VPN Connectivity** for hybrid setup

✅ **Step 1: Choose a VPC CIDR Range**

Since this VPC will support **multiple applications** and services, we choose:

📌 **VPC CIDR:** 10.0.0.0/16 → Provides **65,536 IPs** (large enough for scaling)

✅ **Step 2: Define Subnets & Assign CIDRs**

We divide the VPC into **Public, Private, and Database Subnets** across **two Availability Zones (AZs).**

Subnet Type	AZ	CIDR Range	Purpose
Public Subnet 1	us-east-1a	10.0.1.0/24	Hosts **ALB, Bastion Host**
Public Subnet 2	us-east-1b	10.0.2.0/24	For **high availability**
Private App Subnet 1	us-east-1a	10.0.3.0/24	Hosts **EC2 (App Layer)**
Private App Subnet 2	us-east-1b	10.0.4.0/24	For **redundancy**
Database Subnet 1	us-east-1a	10.0.5.0/24	Hosts **RDS (DB Layer)**
Database Subnet 2	us-east-1b	10.0.6.0/24	Multi-AZ **failover DB**

📌 **Why /24 for Subnets?**

Each /24 subnet provides **256 IPs**, enough for EC2 instances, load balancers, and other resources.

✅ **Step 3: Networking & Security Configuration**

- **Internet Gateway (IGW)** → Attaches to the VPC for **public subnets** (Allows internet access).
- **NAT Gateway** → Deployed in **Public Subnet**, allowing **Private Subnets** to access the internet (for updates, patches, etc.).
- **Security Groups** → Controls **instance-level access** (e.g., allow SSH only from Bastion Host).
- **Network ACLs** → Adds an **extra layer of security** (e.g., blocking unwanted IPs at subnet level).

✅ **Step 4: Connecting On-Premise Networks (Optional)**

- **VPC Peering** – If communication is needed between **multiple VPCs**.
- **AWS Site-to-Site VPN** – If we need to connect **on-premise data center** to AWS.

◆ **VPC Architecture Diagram (Logical View)**

```
VPC: 10.0.0.0/16
│
├── Public Subnet 1 (10.0.1.0/24) - ALB, Bastion
│   ├── Internet Gateway (IGW)
│   ├── NAT Gateway
│
├── Private App Subnet 1 (10.0.3.0/24) - EC2 Instances
│   ├── Connects to Public Subnet via NAT
│
```

```
├── Private DB Subnet 1 (10.0.5.0/24) - RDS DB
│   ├── No direct internet access (private)
│
└── (Similar setup in a second AZ for high availability)
```

✅ **Summary: Why This Design?**
✓ **Scalability:** Uses /16 CIDR for expansion, /24 for subnets.
✓ **High Availability:** Multiple AZs for redundancy.
✓ **Security:** Uses **NACLs, Security Groups, and NAT Gateway**.
✓ **Optimized for Performance:** Keeps **Database Subnet private**.

15. I have a private S3 bucket and I want to give access to a particular user how do we do it?
✅ Use **IAM Policies** for AWS users in the same account.
✅ Use **Bucket Policies** if granting access to users from another AWS account.
✅ Use **Pre-signed URLs** for temporary access.
✅ If the user does **not belong to AWS IAM** (e.g., an external user), you can set an **ACL on individual objects.**

16. I want to create an application running in EC2 instance to another EC2 instance how do you do it? (It's not replication or backup).
✓ Create AMI or manually transfer files
✓ Use SCP, Rsync, or S3 for file transfer
✓ Migrate the database **if needed**
✓ Ensure correct security groups & IAM roles
✓ Update DNS if necessary

17. In what scenarios have you used S3 bucket?
1. **Log Storage & Analysis (CloudTrail, VPC Flow Logs)**
✓ **Scenario:** Store application logs, security logs, or access logs for analysis.
✓ **Example:**
- Store **CloudTrail logs** for auditing.
- Save **VPC Flow Logs** for network monitoring.
- Process logs using AWS Athena or Lambda for insights.
  ✓ **How?**
- Enable **Server Access Logging** in S3.
- Use AWS Athena to query logs stored in S3.

2. **Backup & Disaster Recovery**
✓ **Scenario:** Store backups of EC2, RDS, or EBS snapshots for DR purposes.
✓ **Example:**
- Store MySQL/PostgreSQL backups before deleting old instances.
- Archive old project data in **S3 Glacier** to save cost.

3. **Cross-Account & Cross-Region Data Sharing**
✓ **Scenario:** Share data across AWS accounts or different regions securely.
✓ **Example:**
- Enable **Bucket Policy** & **S3 Access Points** to allow access from another AWS account.
- Enable **Cross-Region Replication (CRR)** to sync buckets across regions.
  Apply IAM policy for cross-account access.
- Set up Cross-Region Replication for backup.

4. **Triggering AWS Lambda for Automation**
✓ **Scenario:** Use S3 **event triggers** to automate processing.
✓ **Example:**
- Automatically **resize images** when uploaded.

- Process CSV files when they are added to S3.
  ✓ **How?**
- Configure **S3 Event Notification** to trigger Lambda.
- Lambda reads file and processes it.

5. **Terraform State Storage**

✓ **Scenario:** Store Terraform state securely to enable collaboration.

✓ **Example:**

- Use S3 as the **backend** for Terraform state.
- Enable **DynamoDB lock** to prevent conflicts.

18. I don't want to connect internet gateway or bastion host but I want my EC2 to install server patches downloading from internet how do you do it?

✅ **Solution: Use AWS SSM with VPC Endpoints**

Instead of direct internet access, use **AWS Systems Manager (SSM) Patch Manager** with **VPC Endpoints** to fetch updates securely.

🔹 Summary

- ✅ No **Internet Gateway or Bastion Host** required.
- ✅ EC2 gets updates via **AWS Systems Manager Patch Manager**.
- ✅ Use **VPC Endpoints for SSM, EC2 Messages, and S3**.
- ✅ Automate patching via **SSM Maintenance Window**.

19. I want to connect VPC 1 from one account to VPC 1-10 in another account how to do? Without VPC peering how to do?

AWS **Transit Gateway (TGW)** acts as a **central hub** to connect multiple VPCs **across different AWS accounts and regions**.

20. How did you handle disaster recovery?

DR Strategy	RTO	RPO	Cost	Use Case
Backup & Restore	High (Hours)	High (Hours)	Low	Non-critical workloads
Pilot Light	Medium (Minutes-Hours)	Low (Minutes)	Medium	Essential services with infrequent failovers
Warm Standby	Low (Minutes)	Low (Minutes)	High	Business-critical applications
Multi-Region Active-Active	Zero	Zero	Very High	Mission-critical workloads requiring no downtime

---

🔥 How I Implemented Disaster Recovery

1. Backup & Restore

✅ **Use Case**: For non-critical applications

✅ **Steps I Took**:

- **Automated Backups**: Used **AWS Backup** to take periodic backups of **EC2, RDS, EBS, DynamoDB, S3**.
- **Cross-Region Replication (CRR)**: Enabled **S3 Replication** and **RDS Read Replicas** in another region.
- **Restore Process**: Defined a runbook to quickly restore instances from snapshots.

---

2. Pilot Light

✅ **Use Case**: For essential services requiring quick recovery

✅ **Steps I Took**:

- **Minimal Infrastructure in DR Region**: Deployed key AWS components (like **RDS, DynamoDB, AMI** copies) but kept EC2 instances powered down.
- **Data Replication**: Used **RDS Read Replicas** and **DynamoDB Global Tables** for live data sync.

- **Failover Process**: Automated instance launch using **Terraform/CloudFormation** in case of failure.

---

3. Warm Standby

✅ **Use Case**: For business-critical applications requiring quick failover

✅ **Steps I Took**:

- **Always Running Smaller Instances in DR Region**: A scaled-down version of the production environment.
- **Real-Time Data Sync**:
  - **RDS Multi-AZ**
  - **DynamoDB Global Tables**
  - **S3 Replication**
- **Automated Scaling in DR**:
  - **Route 53 Failover Routing** to switch traffic to DR.
  - **Auto Scaling Groups (ASG)** to scale resources when needed.

---

4. Multi-Region Active-Active

✅ **Use Case**: For mission-critical applications with zero downtime

✅ **Steps I Took**:

- **Global Load Balancer (Route 53 Geolocation Routing)** for multi-region traffic distribution.
- **AWS Global Accelerator** to route traffic to the nearest healthy region.
- **Active-Active Databases**:
  - **Aurora Global Databases** for RDS.
  - **DynamoDB Global Tables** for NoSQL.
- **Infrastructure as Code (IaC)**:
  - Used **Terraform** to maintain identical infrastructure in both regions.
  - Continuous deployment with **CI/CD Pipelines**.

🛠 **Key AWS Services Used for DR**

- **Backup & Replication**: AWS Backup, RDS Read Replicas, DynamoDB Global Tables, S3 Replication
- **Failover & Routing**: Route 53, AWS Global Accelerator
- **Infrastructure as Code**: Terraform, CloudFormation
- **Scaling & Automation**: Auto Scaling, Lambda, Step Functions

---

21. You have an application load balancer with 4 target groups(EC2) and 3 are running fine but one goes down so what would be the possible reason for this? Apart from network issue what else maybe the issue?

1. Health Check Failures

- The **Application Load Balancer (ALB) health checks** are failing for that specific instance.
- The instance might be responding too slowly or returning **non-200 HTTP status codes**.
- Incorrect **health check path** (e.g., /health returning 500 instead of 200).
- The instance might not have the required **ports open** for health checks.

2. Application or Process Failure

- The application running on the EC2 instance has crashed or is not responding.
- Background services like **database connections, dependencies, or APIs** might be failing.
- The **server logs (e.g., in /var/log/ on Linux)** may show errors.

3. CPU/Memory/Resource Exhaustion

- The instance is running out of **CPU, RAM, or disk space**.
- High load due to traffic spikes or unoptimized queries could make the instance unresponsive.
- **OOM (Out of Memory) Killer** might have terminated critical processes.

4. Security Group or IAM Role Issues

- The security group attached to the EC2 instance might be blocking **inbound traffic**.
- **IAM role issues** could prevent the instance from communicating with external services.

5. Deployment Issues

- A failed deployment might have introduced **broken code** or **configuration issues**.
- Application dependencies might not be installed or **environment variables** might be missing.

6. Disk or Storage Issues

- If **EBS volumes** are detached or corrupted, the instance may fail.
- **High disk I/O or full disk space** can cause slow responses or crashes.

7. Instance-Specific Issues

- The EC2 instance might have been accidentally **terminated or stopped**.
- Auto Scaling might have **replaced the instance** due to policy rules.
- **Kernel panics or OS crashes** could make the instance unresponsive.

📌 **How to Troubleshoot?**

✓ Check **ALB Health Check logs**.

✓ SSH into the instance and **verify running processes**.

✓ Monitor **CPU/Memory/Disk usage** using CloudWatch.

✓ Review **application logs** (/var/log/messages, /var/log/syslog, /var/log/nginx/error.log, etc.).

✓ Ensure **security groups and IAM roles** allow necessary traffic.

✓ Validate recent **deployments or configuration changes**.

22. My EC2 is in private subnet and I want to connect S3 to it how do you do?

Since your **EC2 instance is in a private subnet**, it **cannot directly access the internet** to connect to **S3**. Here's how you can enable S3 access:

◆ **Option 1: VPC Endpoint for S3 (Recommended)**

1. **Create an S3 Gateway Endpoint** in your VPC.
2. Attach it to the **private subnet** where your EC2 is running.
3. Update the **route table** of the private subnet to route **S3 traffic via the VPC endpoint**.
4. Ensure your EC2 instance has an **IAM Role** with **S3 access permissions**.

✅ **Pros:** Secure, No NAT Gateway needed, No additional cost.

⛔ **Cons:** Works only for S3, not for general internet access.

◆ **Option 2: NAT Gateway (If General Internet Access is Required)**

1. **Create a NAT Gateway** in a **public subnet** and associate it with an **Elastic IP**.
2. Update the **route table** of the private subnet to route **internet traffic (0.0.0.0/0) to the NAT Gateway**.
3. Attach an **IAM Role with S3 permissions** to the EC2 instance.

✅ **Pros:** Can access S3 and the internet for updates, patches, etc.

⛔ **Cons:** Higher cost due to NAT Gateway and data transfer charges.

◆ **Option 3: S3 Proxy via a Public EC2**

1. Set up an **EC2 instance in a public subnet** with an **S3 proxy** (like Squid).
2. Route your **private EC2 traffic** through this proxy.
3. Ensure the proxy EC2 has an **IAM Role with S3 access**.

✅ **Pros:** Can restrict S3 access via proxy rules.

⛔ **Cons:** Adds complexity and overhead.

◆ **Recommended Approach?**

👉 If you **only need S3 access**, use **VPC Gateway Endpoint** (Option 1).

👉 If you need **internet access too**, use **NAT Gateway** (Option 2).

23. I have 2 EC2 connecting to S3 bucket and I'm able to connect and access the files from one bucket but I'm not able to access from another EC2 and I'm getting 403 forbidden error. How do we troubleshoot and fix this?

A **403 Forbidden** error when accessing an S3 bucket from one of your EC2 instances means that the request is **being denied** due to insufficient permissions.

1. Ensure EC2 has an **IAM Role** with **S3 permissions**.
2. Verify the **S3 Bucket Policy** doesn't **deny access**.
3. Check if the **S3 ACLs** allow access.
4. If using a private subnet, verify **VPC Endpoints**.

24. How do you connect S3 to EC2 instance from network point of view?

There are multiple ways to **connect an EC2 instance to an S3 bucket** depending on whether your EC2 instance is in a **public subnet (with internet access)** or a **private subnet (without direct internet access)**.

✅ 1. EC2 in a Public Subnet (With Internet Gateway)
If your **EC2 instance is in a public subnet**, it can access S3 **over the internet** via an **Internet Gateway (IGW)**.
Steps:
1. Attach an **IAM role** to EC2 with **S3 access permissions**.
2. Ensure **Security Groups & NACLs** allow outbound HTTPS (port 443).
3. Ensure the **route table** has an **Internet Gateway (IGW)** for internet access.

✅ 2. EC2 in a Private Subnet (Without Internet Gateway)
If your EC2 is in a **private subnet**, it cannot reach the S3 bucket over the internet. Instead, you need to use a **VPC Endpoint for S3**.
Steps to Connect EC2 to S3 in a Private Subnet:
◆ **Option 1: Using VPC Endpoint (Recommended)**
1. **Create an S3 VPC Endpoint**:
   - Go to **AWS Console → VPC → Endpoints → Create Endpoint**.
   - Choose **AWS Service → S3**.
   - Select the **VPC and private subnet**.
   - Attach to the correct **route table**.
   - Choose **"Full Access"** or create a custom **S3 bucket policy** to restrict access.
2. **Update Route Table**:
   - Ensure the route table has an entry for **VPC Endpoint**.
3. **IAM Role with S3 Access**:
   - Attach an IAM role with the **AmazonS3FullAccess** policy to the EC2 instance.
4. **Verify Connection**:
   - Run the following command inside EC2:
     `aws s3 ls s3://your-bucket-name`

◆ **Option 2: Using NAT Gateway (Alternative)**
If VPC Endpoints are **not an option**, you can use a **NAT Gateway** to allow private EC2 instances to connect to S3 via the internet.
1. **Create a NAT Gateway** in a **public subnet**.
2. **Attach an Elastic IP** to the NAT Gateway.
3. **Update the route table** of the **private subnet**:
   - Add a route for **0.0.0.0/0 → NAT Gateway**.
   4. Ensure **IAM Role permissions** allow S3 access.

🎯 Final Summary:

Scenario	Solution	Best Practice?
EC2 in Public Subnet	Uses **Internet Gateway** (IGW)	❌ Not Secure
EC2 in Private Subnet	**VPC Endpoint for S3** (Recommended)	✅ Best for Security
EC2 in Private Subnet	**NAT Gateway** (Alternative)	❌ Costly

25. How do you fix network issues in EC2?

Issue	Possible Cause	Troubleshooting & Fixes
No Internet Access	Missing **Internet Gateway (IGW)**, missing **route table entry**, or **wrong security group rules**.	1. Verify **IGW is attached**. 2. Check **route table** (`0.0.0.0/0 → IGW`). 3. Allow **outbound** traffic in **security groups** & **NACLs**.

Issue	Possible Cause	Troubleshooting & Fixes
EC2 in Private Subnet Has No Internet	Missing **NAT Gateway** for outbound traffic.	1. Attach **NAT Gateway** to the **public subnet**. 2. Ensure **private subnet route table** has `0.0.0.0/0 → NAT Gateway`.
Cannot SSH to EC2	Security group rules, missing **key pair**, wrong **NACLs**.	1. Allow `22/tcp` in **security groups** & **NACLs**. 2. Verify the **correct private key** is used. 3. Check if **EC2 has a public IP** (if in public subnet).
One EC2 instance cannot connect to another	Security groups blocking internal traffic.	1. Allow **inbound** & **outbound** traffic for specific ports in **SG rules**. 2. Verify **VPC CIDR blocks**.
EC2 Cannot Reach S3	Missing **VPC Endpoint**, blocked by NACLs.	1. Create **VPC Endpoint for S3**. 2. Ensure **NACLs allow S3 traffic**. 3. Check **IAM policies** for S3 access.
DNS Resolution Fails	Incorrect `/etc/resolv.conf`, missing **VPC DNS resolver**.	1. Use AmazonProvidedDNS in `/etc/resolv.conf`. 2. Restart **network service**.
Slow Network / High Latency	Routing issues, high load on instance.	1. Run `traceroute` to check route. 2. Check **CloudWatch metrics** for high network load.

26. How do you build a highly available application using application load balancer, auto scaling groups?

To build a **highly available application** using **Application Load Balancer (ALB) and Auto Scaling Groups (ASG)** in AWS:

1. **Deploy ALB**:
   - Create an **Application Load Balancer (ALB)** across **multiple Availability Zones (AZs)**.
   - Configure **target groups** and **listeners** to route traffic.
2. **Auto Scaling Group (ASG)**:
   - Deploy EC2 instances in an **Auto Scaling Group** spanning multiple AZs.
   - Define **minimum, desired, and maximum instances** to maintain availability.
   - Attach ASG to the **ALB target group** for load balancing.
3. **Networking & Security**:
   - Place instances in a **VPC** with **public/private subnets**.
   - Use **Internet Gateway (IGW)** for public-facing apps or **NAT Gateway** for private EC2s.
   - Configure **Security Groups (SGs) & Network ACLs (NACLs)** for secure traffic flow.
4. **Health Checks & Auto Recovery**:
   - ALB performs **health checks** to route traffic only to healthy instances.
   - ASG automatically **replaces unhealthy instances** to ensure availability.
5. **Scaling Strategy**:
   - Use **CPU utilization, request count, or custom metrics** for **auto-scaling up/down**.
   - Enable **Elastic Load Balancing (ELB) stickiness** if needed.

This setup ensures **fault tolerance, automatic failover, and high availability** for your application. 🚀

27. We have an EC2 & we are not able to reach/ssh into it. How ull troubleshoot

1. Check Network Connectivity

- **Verify Public IP**: Ensure you're using the correct **public IP** or **Elastic IP** (for public instances).
- **Check VPC & Subnet Configuration**:
  - If the instance is in a **private subnet**, you need a **Bastion Host** or **VPN** for access.
  - Ensure a **NAT Gateway or VPC Endpoint** exists if internet access is required.

2. Security Group & NACLs

- **Check Security Group (SG) Rules**:
  - Inbound Rule: Allow **port 22** (SSH) from your **IP (0.0.0.0/0 for testing)**.
  - Outbound Rule: Allow **all traffic (0.0.0.0/0)** for outgoing connections.
- **Check Network ACLs**:
  - Ensure **Inbound & Outbound** rules allow SSH (port 22).
  - If there's a **deny rule**, remove it.

3. Verify Key Pair

- **Correct Private Key**: Ensure you are using the right **.pem** file (chmod 400 key.pem).
- **Correct Username**: Use the right **default username** for the AMI:
  - **Amazon Linux** → ec2-user
  - **Ubuntu** → ubuntu
  - **RHEL** → ec2-user or root
  - **Debian** → admin

4. Instance-Level Issues

- **Check EC2 Instance Status**:
  - If **stopped**, **start it**.
  - If in **failed state**, check AWS **console logs**.
- **Check System Logs** (EC2 > Actions > Monitor and Troubleshoot > Get System Log)
  - Look for errors like Network is unreachable or Permission denied.

5. Verify Route Table & Internet Gateway (for Public Instances)

- Ensure the **route table** is correctly set up:
  - Public Subnet → Route 0.0.0.0/0 via **Internet Gateway (IGW)**
  - Private Subnet → Route 0.0.0.0/0 via **NAT Gateway**

6. Use AWS SSM Session Manager (if enabled)

- If SSH is not working, try **AWS Session Manager** (if configured) to access EC2 via AWS Console.

7. Reattach Key Pair (if lost)

- If the key is lost, create a new one, stop the instance, **detach the root volume**, attach it to another EC2, and replace the authorized_keys file with the new key.

28. Suppose EC2 is fluctuating then how do you optimise it using which AWS services?

If an **EC2 instance is fluctuating** (i.e., experiencing performance issues, frequent restarts, or high resource utilization), you can optimize it using the following **AWS services**:

1. Auto Scaling Group (ASG) 📈

- **Solution**: Add your EC2 instance to an **Auto Scaling Group** (ASG) to automatically scale up or down based on demand.
- **How**:
  - Define **scaling policies** (e.g., scale out when CPU > 70%).
  - Use **minimum and maximum instance limits** to prevent over-scaling.

2. Amazon CloudWatch 📊

- **Solution**: Monitor **CPU, memory, disk I/O, and network traffic** to detect spikes in resource usage.
- **How**:
  - Set up **CloudWatch Alarms** to trigger scaling actions or notify admins.
  - Use **CloudWatch Logs & Metrics** to analyze performance bottlenecks.

3. AWS Compute Optimizer 🛠️

- **Solution**: Provides recommendations on whether your instance is **under-provisioned** or **over-provisioned**.
- **How**:
  - Suggests better EC2 instance types based on actual usage patterns.
  - Helps in cost optimization by downsizing underutilized instances.

4. AWS Lambda + Auto Healing 🚑

- **Solution**: Use **Lambda functions** to detect unhealthy instances and automatically replace them.
- **How**:
  - CloudWatch triggers Lambda when instance **fails health checks**.
  - Lambda **terminates and relaunches** the instance using ASG.

5. Elastic Load Balancer (ELB) 🔀

- **Solution**: If traffic fluctuates, use **Application Load Balancer (ALB)** or **Network Load Balancer (NLB)** to distribute requests across multiple instances.
- **How**:
  - Prevents overload on a single instance.
  - Ensures high availability by routing traffic to healthy instances.

6. Spot Instances + Savings Plans 💰

- **Solution**: If fluctuations are due to **variable workload**, use **Spot Instances** for cost-effective scaling.
- **How**:
  - Use **On-Demand + Spot mix** in an ASG for cost savings.
  - Use **Savings Plans** for predictable workloads.

7. AWS Systems Manager (SSM) 🔍

- **Solution**: Troubleshoot and patch EC2 instances without SSH.
- **How**:
  - Run **SSM Run Command** to diagnose issues.
  - Automate **patch updates** for security and stability.

Final Optimization Strategy

- **Short-Term Fix**: Check **CloudWatch Metrics**, restart, or change instance type if needed.
- **Long-Term Fix**: Implement **ASG, ELB, AWS Compute Optimizer, and SSM** for auto-scaling, load balancing, and proactive monitoring.

29. Useradd while creating EC2 (add 10 users).

```
resource "aws_instance" "web_instance" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
  key_name      = "my-key"
  subnet_id     = aws_subnet.public_subnet.id
  security_groups = [aws_security_group.web_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    for i in {1..10}; do
      sudo useradd "user$i"
      echo "user$i:password123" | sudo chpasswd
    done
  EOF

  tags = {
    Name = "SingleWebInstance"
  }
}
```

30. I have my EC2 in private subnets with private IP address. How do I access it?

1. **For secure SSH access: Use a Bastion Host**
2. **For hassle-free management: Use AWS SSM Session Manager** (Attach IAM Role to EC2 with SSM permissions (`AmazonSSMManagedInstanceCore`))
3. **For corporate setup: Use VPN/Direct Connect:** If your VPC is connected to an **on-premise network** via **AWS VPN** or **AWS Direct Connect**, you can access the private EC2 instance **as if it's on your local network**.

31. How ull migrate a legacy application to AWS.. can u briefly explain all services u require

Migrating a legacy application to AWS involves **modernizing the infrastructure** while ensuring **minimal downtime** and **high availability**. Below is a high-level approach along with the AWS services required at each stage:

---

1. Assess & Plan Migration

◆ **AWS Services Used:**
✅ **AWS Migration Hub** → Centralized tracking of migration progress.
✅ **AWS Application Discovery Service** → Analyzes on-premise infrastructure.
✅ **AWS Well-Architected Tool** → Assesses workloads against best practices.

---

2. Set Up AWS Infrastructure
◆ **AWS Services Used:**
✅ **Amazon VPC** → Creates a secure, isolated network.
✅ **AWS Direct Connect or VPN** → Connects on-premise data center to AWS.
✅ **Amazon Route 53** → Manages DNS routing.

---

3. Choose the Right Compute Option
◆ **AWS Services Used:**
✅ **Amazon EC2** → Lift-and-shift workloads (Rehost).
✅ **Amazon ECS/EKS (Containers)** → If refactoring into microservices.
✅ **AWS Lambda** → Serverless migration for event-driven tasks.
✅ **Auto Scaling & ALB** → Ensures high availability & load balancing.

---

4. Migrate Databases
◆ **AWS Services Used:**
✅ **AWS Database Migration Service (DMS)** → Migrates on-premise databases to AWS with minimal downtime.
✅ **Amazon RDS (Managed DB)** → MySQL, PostgreSQL, MSSQL, etc.
✅ **Amazon Aurora** → High-performance relational database alternative.
✅ **Amazon DynamoDB** → NoSQL database for scalability.

---

5. Storage & File Systems
◆ **AWS Services Used:**
✅ **Amazon S3** → Stores application files, logs, backups.
✅ **Amazon EBS** → Persistent block storage for EC2 instances.
✅ **Amazon FSx / EFS** → Managed file systems for shared storage.

---

6. Security & IAM Management
◆ **AWS Services Used:**
✅ **AWS IAM (Identity & Access Management)** → Controls user permissions.
✅ **AWS WAF & Shield** → Protects applications from cyber threats.
✅ **AWS Secrets Manager** → Manages credentials securely.
✅ **AWS KMS (Key Management Service)** → Encrypts sensitive data.

---

7. Monitoring & Logging
◆ **AWS Services Used:**
✅ **Amazon CloudWatch** → Monitors logs & application performance.
✅ **AWS X-Ray** → Traces application requests for debugging.
✅ **AWS CloudTrail** → Logs API activity for auditing.

---

8. CI/CD & Automation
◆ **AWS Services Used:**
✅ **AWS CodePipeline & CodeBuild** → Automates CI/CD pipelines.
✅ **AWS CloudFormation or Terraform** → Infrastructure as Code (IaC).
✅ **AWS Step Functions** → Orchestrates workflows across AWS services.

Migration Strategies (6 R's)

1. **Rehost (Lift & Shift)** → Move as-is to AWS EC2.
2. **Replatform (Lift & Tweak)** → Move to managed services (RDS, EKS).
3. **Refactor** → Modernize app (Microservices, Lambda, Containers).
4. **Repurchase** → Move to a SaaS alternative.
5. **Retain** → Keep some workloads on-premise.
6. **Retire** → Decommission unused applications.

Final Thoughts

- Choose **AWS Application Migration Service** for minimal downtime migration.
- Use **AWS Landing Zone** for enterprise-level security & compliance.
- **Test & validate** performance before full cutover.

32. What all security measures ull take to secure the application on AWS

1. Network Security & Access Control

✅ **Use Amazon VPC** → Isolate the application in a private network.
✅ **Security Groups & NACLs** → Restrict inbound/outbound traffic.
✅ **AWS WAF & AWS Shield** → Protect against DDoS & web attacks.
✅ **VPC Endpoints** → Securely connect to AWS services without internet exposure.
✅ **Bastion Host** → Secure SSH/RDP access to private EC2 instances.

---

2. Identity & Access Management (IAM)

✅ **Use IAM Roles Instead of Access Keys** → Avoid storing credentials in code.
✅ **Implement Least Privilege Access** → Grant only necessary permissions.
✅ **Enable Multi-Factor Authentication (MFA)** → Protect IAM users & root accounts.
✅ **AWS Secrets Manager** → Secure API keys, database credentials, and sensitive data.
✅ **Use AWS Organizations & SCPs** → Restrict permissions at an account level.

---

3. Data Security & Encryption

✅ **Enable Encryption for S3, RDS, and EBS** → Use **AWS KMS (Key Management Service)** for encryption.
✅ **Use S3 Bucket Policies & Object Lock** → Prevent accidental deletion & unauthorized access.
✅ **Enable Transport Layer Security (TLS)** → Use SSL/TLS for data encryption in transit.
✅ **Enable RDS SSL/TLS Connections** → Secure database access.
✅ **AWS Macie & GuardDuty** → Detect sensitive data exposure & suspicious activities.

---

4. Application Security & Monitoring

✅ **Use AWS IAM Roles for Applications** → Avoid hardcoded credentials.
✅ **Enable AWS CloudTrail & Config** → Track API activity & compliance.
✅ **Monitor Logs with Amazon CloudWatch & AWS X-Ray** → Detect anomalies.
✅ **Perform Regular Penetration Testing** → Simulate attacks to find vulnerabilities.
✅ **Automate Security Patching** → Use **AWS Systems Manager Patch Manager**.

---

5. Backup, Disaster Recovery & Compliance

✅ **Automate Backups with AWS Backup** → Ensure data recovery.
✅ **Enable Cross-Region Replication (CRR)** → Protect against regional failures.
✅ **Implement Multi-AZ RDS & DynamoDB Global Tables** → High availability.
✅ **Use AWS Audit Manager** → Ensure compliance with industry standards (SOC2, HIPAA, ISO).

---

6. Secure CI/CD & Deployment

✅ **Use AWS CodePipeline with IAM Restrictions** → Secure deployment pipelines.
✅ **Enable Static Code Analysis (SAST) & DAST** → Use SonarQube, OWASP ZAP.

✅ **Use Infrastructure as Code (IaC) Security** → Use tools like **Checkov** & **TFSec** to scan Terraform templates.

✅ **Automate Secrets Management in CI/CD** → Integrate AWS Secrets Manager with GitHub Actions, Jenkins, or GitLab CI.

Final Thoughts

- Follow **AWS Shared Responsibility Model**: AWS secures the cloud, **you secure your workloads**.
- Use **AWS Security Hub** for centralized security management.
- **Regularly test and audit** security policies.

33. How ull store the secrets?

1. AWS Secrets Manager (Recommended ✅)

✅ **Fully Managed** → Secure storage for API keys, DB credentials, and passwords.

✅ **Automatic Secret Rotation** → Integrates with RDS, Redshift, and other services.

✅ **IAM-based Access Control** → Restrict access using IAM policies.

✅ **AWS SDK & CLI Support** → Fetch secrets programmatically.

✅ **Encryption using AWS KMS** → Secrets are stored **encrypted** by default.

📌 **How to Store a Secret in AWS Secrets Manager?**

```
aws secretsmanager create-secret --name my-secret-key \
--secret-string '{"username":"admin", "password":"mypassword"}'
```

---

2. AWS Systems Manager (SSM) Parameter Store

✅ **Supports Plaintext & Encrypted Storage**

✅ **Integrated with IAM for Access Control**

✅ **Can Store Configuration Values** (e.g., database endpoints, API URLs)

✅ **Cheaper than AWS Secrets Manager**

📌 **How to Store a Secret in SSM Parameter Store?**

```
aws ssm put-parameter --name "/myapp/db-password" \
--value "mypassword" --type SecureString \
--key-id "alias/aws/ssm"
```

---

3. HashiCorp Vault (Self-Managed)

✅ **Enterprise-Grade Secrets Management**

✅ **Supports Dynamic Secrets & Auto Expiry**

✅ **Can Run On-Premises & Multi-Cloud**

📌 **Example:** Store a secret in HashiCorp Vault

```
vault kv put secret/myapp db_password=mypassword
```

---

4. Environment Variables (Least Secure 🔓)

✅ **Quick & Easy** → Use .env files in applications.

❌ **Risky if Not Encrypted** → Can be exposed in logs or version control.

📌 **Example:** Store secrets in .env file

```
DB_PASSWORD=mypassword
```

⚠️ **Never commit .env files to GitHub! Use .gitignore.**

Best Practices for Storing Secrets in AWS

✓ **Use AWS Secrets Manager or SSM Parameter Store** → Managed & secure.

✓ **Grant Least Privilege Access** → Restrict access with IAM policies.

✓ **Rotate Secrets Automatically** → Prevent stale or compromised credentials.

✓ **Avoid Hardcoding Secrets** → Use environment variables or fetch dynamically.

✓ **Enable Encryption** → Use **AWS KMS** for additional security.

34. How do you make sure your application is highly available?

1. Multi-AZ Deployment (For Fault Tolerance)

✅ **EC2 Instances in Multiple AZs** → Ensures availability in case of AZ failure.
✅ **RDS Multi-AZ** → Automatic failover for database redundancy.
✅ **ElastiCache Multi-AZ** → Ensures high availability for caching services.
✅ **Amazon S3** → Data stored across **multiple Availability Zones** by default.
Example: Deploy EC2 in Multi-AZ using Auto Scaling Group

---

2. Load Balancing with Auto Scaling
✅ **Application Load Balancer (ALB)** → Distributes traffic across multiple EC2s.
✅ **Auto Scaling Groups (ASG)** → Scales instances **based on demand**.
✅ **Cross-Zone Load Balancing** → Ensures traffic is evenly distributed.
Example: ALB with ASG

---

3. Database High Availability
✅ **Amazon RDS Multi-AZ** → Automatic failover to a standby DB instance.
✅ **Read Replicas for Scaling** → RDS Read Replicas for read-heavy workloads.
✅ **DynamoDB Global Tables** → Replicates NoSQL data across multiple regions.

---

4. Route 53 for DNS Failover
✅ **Route 53 Health Checks** → Detects failures and routes traffic accordingly.
✅ **Multi-Region Failover** → Routes traffic to a **secondary region** if needed.

---

5. Disaster Recovery (DR) Strategy
✅ **Backup and Restore** → Use **AWS Backup** for automated snapshots.
✅ **Pilot Light** → Keep a **minimal running version** of the app in another region.
✅ **Warm Standby** → Keep a **scaled-down copy** of the app in another region.
✅ **Multi-Region Active-Active** → Deploy the app in **multiple regions** for **instant failover**.

---

6. CI/CD for Quick Recovery
✅ **Automate Deployment with CodePipeline & CodeDeploy**
✅ **Infrastructure as Code (IaC) with Terraform** → Quick redeployment.
✅ **Blue/Green Deployments** → Minimize downtime during updates.

Final Summary:
✓ **Multi-AZ, ALB, and Auto Scaling** for redundancy.
✓ **RDS Multi-AZ & DynamoDB Global Tables** for DB HA.
✓ **Route 53 DNS Failover** for regional availability.
✓ **CI/CD, Backups, and Disaster Recovery** for rapid recovery.

35. How have you designed disaster recovery plan for your application in AWS?

DR Strategy	Description	Use Case	AWS Services Used
Backup & Restore	Regularly backs up data & restores when needed	Non-critical apps	AWS Backup, S3, RDS Snapshots
Pilot Light	A minimal version of the app is always running in another region	Medium-critical apps	EC2, RDS, DynamoDB, AMIs, Route 53
Warm Standby	A scaled-down version is live, ready to scale up during failures	Business-critical apps	Auto Scaling, ALB, RDS Multi-AZ, Route 53
Active-Active Multi-Region	Fully redundant systems in multiple AWS regions	Mission-critical apps	Global Accelerator, Route 53, DynamoDB Global Tables

---

🛠 Key AWS Services Used for DR
✅ **1. Backup & Restore**

✓ **AWS Backup** → Automates periodic backups of EC2, RDS, DynamoDB, and EFS.
✓ **S3 Lifecycle Policies** → Move backups to **Glacier** for cost efficiency.
✓ **RDS Snapshots & DynamoDB Point-in-Time Recovery (PITR)**

✅ **2. Pilot Light**
✓ Only **core services (DB, minimal EC2)** run in the **secondary region**.
✓ Other services are **spun up during failover** using **AMI snapshots, IaC (Terraform)**.
✓ **Route 53** switches traffic via **DNS failover**.
Example Pilot Light Setup:

- Store **EC2 AMIs, RDS snapshots, S3 replication** in another region.
- Use **Terraform** to quickly provision infrastructure if primary fails.

✅ **3. Warm Standby**
✓ A **scaled-down** version of the app is **running** in another region.
✓ **Auto Scaling** increases capacity during failover.
✓ **RDS Multi-AZ, DynamoDB Global Tables** replicate DB in multiple regions.

✅ **4. Active-Active Multi-Region**
✓ Application runs **fully redundant** in multiple AWS regions.
✓ **Global Accelerator** ensures **low-latency routing**.
✓ **Route 53 Latency-based Routing** directs traffic to the closest healthy region.
✓ **DynamoDB Global Tables** replicate data across regions.

📡 Monitoring & Testing
✓ **CloudWatch Alarms & Logs** to detect failures.
✓ **AWS Systems Manager** for automated failover scripts.
✓ **Chaos Engineering** → Use **AWS Fault Injection Simulator** (FIS) to test DR.

🔥 **Summary**
✓ **Choose DR strategy based on RTO/RPO** (Backup → Pilot Light → Warm Standby → Multi-Region).
✓ **Use AWS Backup, Route 53, Auto Scaling, RDS Multi-AZ, and DynamoDB Global Tables** for resilience.
✓ **Regularly test the DR plan with simulated outages** using AWS Fault Injection Simulator.

36. Scenario where few users are able to access the application but others are not able to access it. How do you troubleshoot and fix?

If **some users** can access the application while others **cannot**, it indicates a **networking, DNS, authentication, or application-layer issue**. Here's how to troubleshoot and fix it:

🔍 Step-by-Step Troubleshooting Guide
1. Check Load Balancer & Target Groups
✅ **Issue:** If the application is behind an **Application Load Balancer (ALB)**, some target instances may be **unhealthy**.
🔧 **Fix:**

- Check **ALB Target Group health status** in **AWS Console → EC2 → Load Balancers → Target Groups**.
- Ensure all **target instances are healthy** and receiving traffic.
- If one or more targets are **unhealthy**, check logs on those instances.

```
aws elbv2 describe-target-health --target-group-arn <target-group-arn>
```

---

2. Verify Security Groups & Network ACLs

✅ **Issue:** Some users might be blocked due to **restrictive security groups** or **Network ACLs**.

🔹 **Fix:**

- Ensure **Security Groups** allow inbound traffic on the required ports (e.g., **80/443** for HTTP/HTTPS).
- Check **Network ACLs** for any rules blocking specific **IP ranges**.
- Try allowing traffic temporarily from all (0.0.0.0/0) for debugging.

```
aws ec2 describe-security-groups --group-ids <security-group-id>
```

---

3. Check DNS Resolution

✅ **Issue:** Some users may be using an outdated or misconfigured DNS server.

🔹 **Fix:**

- Flush DNS cache:
  ```
  nslookup myapp.example.com
  ipconfig /flushdns   # Windows
  sudo systemd-resolve --flush-caches   # Linux
  ```
- Ensure **Route 53 is correctly resolving DNS records**.
  ```
  nslookup myapp.example.com
  dig myapp.example.com
  ```

---

4. Verify CDN (CloudFront) or WAF Rules

✅ **Issue:** If **CloudFront** or **AWS WAF** is used, some users might be blocked by **Web Application Firewall (WAF) rules**.

🔹 **Fix:**

- Check **AWS WAF logs** to see if traffic is getting blocked.
- If using CloudFront, check if the **edge locations** are delivering the latest version of the app.

```
aws wafv2 list-logging-configurations
```

---

5. Test from Multiple Locations & ISPs

✅ **Issue:** A specific **region or ISP** might have routing issues.

🔹 **Fix:**

- Test using **VPN** or mobile hotspot.
- Check **CloudWatch Metrics** for request patterns.
- Use **traceroute** to detect routing issues.

```
traceroute myapp.example.com
```

---

6. Check Application Logs

✅ **Issue:** If the backend is rejecting specific users, there may be **session, authentication, or rate-limiting issues**.

🔹 **Fix:**

- Check **application logs** in CloudWatch or /var/log/ on EC2.
- Look for errors related to **authentication, rate limits, or timeouts**.

```
sudo tail -f /var/log/nginx/access.log
```

✅ Final Fix: Based on Findings

🔹 **If an EC2 instance is unhealthy** → Restart it or replace it in the **Auto Scaling Group**.

🔹 **If WAF is blocking users** → Modify WAF rules.

🔹 **If CloudFront is serving outdated content** → Invalidate the cache.

🔹 **If DNS is misconfigured** → Update **Route 53 records** and check propagation.

37. I want to process an excel file and feed it to database. What all services you can use in AWS and how do you achieve this?

Service	Purpose
Amazon S3	Store the uploaded Excel file.
AWS Lambda	Process the Excel file (convert it to CSV/JSON).

Service	Purpose
Amazon SQS	Queue the processed data for batch processing.
AWS Glue	Transform and load the data.
Amazon RDS / DynamoDB	Store the processed data in a relational (MySQL/PostgreSQL) or NoSQL database.
Amazon EventBridge / S3 Event Notifications	Trigger processing when a new file is uploaded.

🚀 Implementation Steps

1. Upload Excel File to S3

- A user uploads an Excel file to an **S3 bucket** (e.g., my-data-bucket).
- Enable **S3 Event Notifications** to trigger processing.

---

2. Trigger Lambda Function for Processing

- S3 **event notification** triggers an **AWS Lambda function**.
- The function:
  - **Reads the Excel file** using **pandas (Python)**.
  - **Converts it to CSV or JSON**.
  - **Stores processed data** in another S3 bucket.

Python Code for Lambda:

```
import boto3
import pandas as pd
import io

s3_client = boto3.client('s3')

def lambda_handler(event, context):
    bucket_name = event['Records'][0]['s3']['bucket']['name']
    file_key = event['Records'][0]['s3']['object']['key']

    # Download the file from S3
    response = s3_client.get_object(Bucket=bucket_name, Key=file_key)
    file_content = response['Body'].read()

    # Read the Excel file
    df = pd.read_excel(io.BytesIO(file_content))

    # Convert to CSV and upload back to S3
    csv_buffer = io.StringIO()
    df.to_csv(csv_buffer, index=False)
    s3_client.put_object(Bucket='processed-data-bucket',
Key='processed_data.csv', Body=csv_buffer.getvalue())

    return {"statusCode": 200, "body": "File processed and saved to S3"}
```

---

3. Send Data to SQS for Asynchronous Processing (Optional)

- If the data is large, we can send it to **Amazon SQS** for batch processing.
- Lambda sends messages to **SQS** for database insertion.

```
sqs_client = boto3.client('sqs')
queue_url = "https://sqs.us-east-1.amazonaws.com/123456789012/my-queue"

for index, row in df.iterrows():
    message_body = row.to_json()
    sqs_client.send_message(QueueUrl=queue_url, MessageBody=message_body)
```

---

4. Insert Processed Data into RDS/DynamoDB

- Another **Lambda function** (or AWS Glue Job) reads messages from SQS and inserts data into **Amazon RDS** (MySQL/PostgreSQL) or **DynamoDB**.

Example for RDS (MySQL) insertion:

```
import pymysql
```

```
connection = pymysql.connect(
    host="mydb-instance.xyz.us-east-1.rds.amazonaws.com",
    user="admin",
    password="mypassword",
    database="mydatabase"
)

cursor = connection.cursor()

insert_query = "INSERT INTO employees (name, age, department) VALUES (%s,
%s, %s)"
for index, row in df.iterrows():
    cursor.execute(insert_query, (row['name'], row['age'],
row['department']))

connection.commit()
connection.close()
```

🎯 Final Architecture Workflow

1. **User uploads Excel file** → Stored in **S3**
2. **S3 triggers AWS Lambda** → Converts Excel to CSV/JSON
3. **Lambda stores processed data in S3** or **sends it to SQS**
4. **Lambda or AWS Glue reads from SQS** → Inserts into **Amazon RDS/DynamoDB**
5. **Database is updated with processed data** ✅

🔥 Alternative Approach Using AWS Glue (For Large Files)

- Use **AWS Glue Crawler** to scan S3 and create a schema.
- **Glue ETL Job** processes data and loads it into **Amazon RDS or Redshift**.

📌 Summary

- Use **S3 for storage**, **Lambda for processing**, and **RDS/DynamoDB for storage**.
- Use **SQS for asynchronous processing** if needed.
- AWS Glue is useful for **large-scale data transformation**.

38. Is EBS regional or global? I want to access ap-south-1a EBS in us-east-1a how do you do?

🔹 **Amazon EBS (Elastic Block Store) is a Regional Service** but is **tied to a specific Availability Zone (AZ)** within a region.

🔹 This means an EBS volume **cannot be accessed across different AWS regions** (e.g., from `ap-south-1a` to `us-east-1a`) directly.

❌ **Can You Directly Attach EBS in One Region to an EC2 in Another Region?**

No ❌, **EBS volumes cannot be accessed across AWS regions** because they are designed to be available within the same Availability Zone (AZ).

✅ **How to Access an EBS Volume in Another Region?**

Since direct access is not possible, you need to **create a snapshot** and then **copy it to the target region** to create a new EBS volume.

Steps to Migrate EBS from `ap-south-1a` to `us-east-1a`

1. **Create a Snapshot of the EBS Volume**
   - Go to AWS **EC2 Dashboard → EBS Volumes**
   - Select your EBS Volume in **ap-south-1a**
   - Click **"Create Snapshot"**
   - Give it a name (e.g., `my-ebs-snapshot`)
2. **Copy the Snapshot to the Target Region (`us-east-1`)**
   - Go to **EC2 → Snapshots**
   - Select the snapshot you created
   - Click **"Copy Snapshot"**

- Choose **Destination Region = `us-east-1`**
- Click **"Copy"**

3. **Create a New EBS Volume in `us-east-1a`**
   - Go to **EC2 → EBS Volumes** in `us-east-1`
   - Click **"Create Volume"**
   - Select **Snapshot ID (copied snapshot)**
   - Choose **Availability Zone = `us-east-1a`**
   - Click **"Create"**
4. **Attach the EBS Volume to an EC2 Instance in `us-east-1a`**
   - Select the newly created volume
   - Click **"Attach Volume"**
   - Choose the **EC2 instance** in us-east-1a
   - Attach the volume ✅

🎯 **Key Takeaways**
✅ **EBS is a regional service but is tied to a single Availability Zone**
✅ **You cannot directly attach an EBS volume across regions**
✅ **The correct way is to create a snapshot, copy it to another region, and create a new volume**
✅ **Use AWS CLI or Terraform to automate this**

39. I have an AMI in ap-south-1 and I want to access it from Tokyo region how to do?

An **Amazon Machine Image (AMI) is region-specific**, meaning an AMI created in `ap-south-1` (Mumbai) **cannot be directly accessed** in another AWS region like `ap-northeast-1` (Tokyo).
To use an AMI in another region, **you need to copy the AMI to the desired region** and then launch an EC2 instance from it.

✅ **Steps to Copy an AMI from `ap-south-1` to `ap-northeast-1`**

1. **Find the AMI in `ap-south-1`**
   - Go to **AWS Console → EC2 Dashboard**
   - Click **"AMIs"** in `ap-south-1`
   - Select the **AMI you want to copy**
2. **Copy the AMI to `ap-northeast-1`**
   - Click **"Actions" → "Copy AMI"**
   - Choose **Destination Region = `ap-northeast-1`**
   - Keep the same encryption settings
   - Click **"Copy AMI"** ✅
3. **Launch an EC2 Instance in `ap-northeast-1`**
   - Switch to **Tokyo (`ap-northeast-1`)** region
   - Go to **EC2 → AMIs**
   - Find the **copied AMI**
   - Click **"Launch Instance"** from this AMI 🚀

🎯 **Key Takeaways**
✅ **AMI is region-specific** and **cannot be accessed across regions directly.**
✅ **You must copy the AMI to the target region (`ap-northeast-1`).**
✅ **Use AWS Console or AWS CLI to copy and launch the instance.**
✅ **You can also automate this using Terraform**.

40. How elb will know to route traffic to particular EC2.

AWS **Elastic Load Balancer (ELB)** automatically distributes incoming traffic across multiple EC2 instances based on **target groups and health checks**.

✅ **How ELB Determines Which EC2 Instance to Route Traffic To?**

1. **Target Group Association**
   - Each **EC2 instance is registered** with a **Target Group**.

- The **Target Group** is linked to the **ELB Listener**, which listens on a specific port (e.g., HTTP:80, HTTPS:443).

2. **Health Checks**
   - The ELB **performs periodic health checks** (e.g., pinging `/health` endpoint).
   - If an instance is **healthy**, it **remains in the target group** and can receive traffic.
   - If an instance **fails health checks**, ELB **stops routing traffic** to it.
3. **Load Balancing Algorithm**

ELB uses **routing algorithms** to distribute traffic:

- **Round Robin** (default for ALB & CLB) → Traffic is evenly distributed to all healthy instances.
- **Least Outstanding Requests** (for ALB) → Routes to the instance with the fewest requests pending.
- **IP Hash** (for NLB) → Ensures a client consistently hits the same instance.

4. **Listener Rules** (For Advanced Routing)
   - ELB can route traffic based on **Host-Based Routing** (`app.example.com` → EC2 A)
   - Path-Based Routing (`/api` → EC2 B, `/static` → EC2 C)

🎯 **Summary**

✅ ELB **routes traffic** using **Target Groups** and **Health Checks**

✅ ELB uses **Round Robin, Least Outstanding Requests, or IP Hash** for routing

✅ Listener **rules** allow **host-based & path-based** traffic routing

✅ ELB only **routes traffic to healthy instances**

41. I have a file in S3 bucket which I access yearly once so in which storage class do you store it?

🎯 **S3 Glacier Deep Archive**

✅ **Cheapest storage option** (lowest cost per GB)

✅ **Designed for long-term archival**

✅ **Retrieval time**: 12-48 hours (if you need the file faster, S3 Glacier may be a better option)

🛠 **Alternative Option:**

If you need **faster retrieval (within minutes to hours),** use **S3 Glacier** instead:

- **S3 Glacier** → Retrieval in minutes to hours, slightly higher cost than Deep Archive.

Terraform:

Terraform state file:

1. Explain about terraform state management. Why is it important?
2. Where do we store state file? Where ull specify that statefile as to be stored in that particular bucket? How do you store terraform state file in backend?
3. How do you lock the State file? Explain terraform state file lock-in mechanism.

Terraform uses **state files** to keep track of resources it manages. This **state management** is crucial for tracking infrastructure changes and ensuring consistency between the actual cloud environment and Terraform configurations.

---

1. What is Terraform State?

- Terraform **stores metadata** about resources in a **state file (terraform.tfstate)**.
- It helps Terraform understand **which resources exist** and **what changes need to be applied**.
- Without state management, Terraform would **not know** what resources exist, leading to duplication or unintended deletions.

---

2. Why is Terraform State Important?

✅ **Tracks Resource Mapping** – Maps Terraform configurations to actual cloud resources.

✅ **Detects Changes Efficiently** – Helps Terraform identify which resources **need updates, deletions, or creations**.

✅ **Supports Remote Collaboration** – Stores state in a central location (S3, Terraform Cloud, etc.) so

teams don't overwrite changes.
✅ **Locks State to Prevent Conflicts** – Ensures only one Terraform process modifies infrastructure at a time.
✅ **Stores Sensitive Data** – Keeps details like VPC IDs, database endpoints, etc.

---

3. Types of Terraform State Management

State Type	Description
Local State	Default. Stored in terraform.tfstate on the local machine. Not recommended for production.
Remote State	Stored in **S3, Terraform Cloud, Azure Blob, etc.**, allowing multiple team members to work safely.

---

4. Terraform Remote State Setup (Example: AWS S3)
1. Create an S3 bucket to store the state:
aws s3 mb s3://my-terraform-state-bucket
2. Update Terraform to use remote state:

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state-bucket"
    key            = "dev/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-lock"
  }
}
```

3. Enable State Locking with DynamoDB:

```
aws dynamodb create-table \
    --table-name terraform-lock \
    --attribute-definitions AttributeName=LockID,AttributeType=S \
    --key-schema AttributeName=LockID,KeyType=HASH \
    --billing-mode PAY_PER_REQUEST
```

✅ **Now Terraform state is stored remotely, preventing conflicts!**

---

5. Key Terraform State Commands

Command	Description
`terraform state list`	Lists all resources tracked in the state.
`terraform state show <resource>`	Shows details of a specific resource.
`terraform state pull`	Downloads the latest state.
`terraform state mv <old> <new>`	Moves a resource in the state file.
`terraform state rm <resource>`	Removes a resource from state (without deleting it).

---

6. Best Practices for Terraform State
✓ **Use Remote State** – Store state in S3, Terraform Cloud, or Azure Blob for team collaboration.
✓ **Enable State Locking** – Prevents multiple users from modifying state at the same time.
✓ **Use State Encryption** – Protect sensitive data stored in the state file.
✓ **Automate Backups** – Keep backups of your terraform.tfstate to avoid accidental loss.
✓ **Never Edit State Manually** – Always use Terraform commands to manage state.

---

7. Summary

Feature	Terraform State
Purpose	Tracks infrastructure resources
Storage Options	Local (terraform.tfstate) or Remote (S3, Terraform Cloud, etc.)
Key Benefits	Prevents duplication, enables collaboration, supports locking

Feature	Terraform State
Best Practice	Use **Remote State + Locking** for team environments

4. If we lose state management file what will happen.

Losing the **Terraform state file (terraform.tfstate)** can cause **serious issues**, as Terraform won't know the current state of your infrastructure. Here's what happens:

---

1. Terraform Loses Track of Resources

- Terraform won't know **what has already been created**.
- Running terraform apply may try to **recreate existing resources**, causing conflicts or duplicate resources.
- Running terraform destroy might not **delete all resources**, leaving orphaned resources.

---

2. How to Recover a Lost Terraform State File?

✅ **Method 1: Restore from Backup**

- If you used **Remote State (S3, Terraform Cloud, etc.)**, check if there's a **previous version** available.
- **S3 Example (Retrieve Previous State Version):**
  ```
  aws s3 cp s3://my-terraform-state-bucket/dev/terraform.tfstate.backup .
  ```

✅ **Method 2: Manually Recreate State Using terraform import**

- Terraform allows you to **re-import existing resources** back into the state file.
- Example (for an EC2 instance):
  ```
  terraform import aws_instance.my_instance i-0abcd1234xyz
  ```
- Repeat this for all resources.

✅ **Method 3: Generate a New State File Using terraform refresh**

- If you still have the main.tf configurations, run:
  terraform refresh
  This will update the state file based on existing infrastructure.

---

3. How to Prevent State File Loss?

✓ **Use Remote State Storage (S3, Terraform Cloud, etc.)**
✓ **Enable State File Versioning (S3 Versioning, Git, Backups)**
✓ **Restrict State File Deletion (IAM Policies, Access Controls)**
✓ **Enable Terraform State Locking (DynamoDB Locking in AWS)**

---

4. Summary

Scenario	What Happens?	How to Fix?
Lost Local State	Terraform doesn't recognize existing resources	Restore from backup, terraform import
Lost Remote State	Team collaboration breaks	Recover from S3 versioning or Terraform Cloud
No Backup Available	Risk of duplication or orphaned resources	Manually import using terraform import

5. When statefile will get corrupted? If a statefile is corrupted how ull resolve it?

A **Terraform state file (terraform.tfstate)** can get corrupted due to various reasons, leading to an inconsistent or unreadable state.

---

1. Common Causes of Terraform State Corruption

Cause	Explanation
Concurrent State Modifications	Multiple users running Terraform at the same time without state locking.

Cause	Explanation
Manual Edits to terraform.tfstate	Editing the state file manually can introduce syntax errors.
Partial Apply Failure	If Terraform crashes or is forcefully stopped during terraform apply, the state might become inconsistent.
Storage Issues	If the state file is stored remotely (e.g., S3), issues like **network failures** or **misconfigured permissions** can corrupt it.
Terraform Version Mismatch	Running Terraform with different versions may lead to state incompatibility.
Failed Remote State Uploads	If Terraform fails while updating the remote state, it may save a partially written file.

---

2. How to Fix a Corrupted Terraform State File?

✅ **Method 1: Restore from Backup (Recommended)**

- **If you are using Remote State (S3, Terraform Cloud, etc.), check if a previous version exists.**
- **Example (AWS S3: Retrieve Previous Version)**
  ```
  aws s3 ls s3://my-terraform-state-bucket/dev/
  aws s3 cp s3://my-terraform-state-bucket/dev/terraform.tfstate.backup ./terraform.tfstate
  ```
- If using Terraform Cloud, restore the latest **healthy** state snapshot.

✅ **Method 2: Manually Fix the State File**

- If corruption is **syntax-related**, open terraform.tfstate in a text editor and **fix JSON errors**.
- Use jq to check if the state file is a valid JSON:
  ```
  jq . terraform.tfstate
  ```
  If it throws an error, try fixing the JSON format.

✅ **Method 3: Use terraform refresh**

- If Terraform can still detect existing infrastructure, run:
  ```
  terraform refresh
  ```
  This **rebuilds the state file** based on the actual infrastructure.

✅ **Method 4: Use terraform import (If State is Lost)**

- If the state is completely corrupted and **cannot be restored**, re-import each resource manually:
  ```
  terraform import aws_instance.my_instance i-0abcd1234xyz
  ```

✅ **Method 5: Use terraform state push (For Remote State)**

- If using **remote state**, you can push a valid backup to override the corrupted one:
  ```
  terraform state push terraform.tfstate.backup
  ```

---

3. How to Prevent Terraform State Corruption?

✓ **Use Remote State Storage (S3, Terraform Cloud, etc.)**
✓ **Enable State File Versioning (S3 Versioning, Backups)**
✓ **Enable Terraform State Locking (DynamoDB Locking in AWS)**
✓ **Avoid Manual Edits to terraform.tfstate**
✓ **Use a Single Terraform Version for All Users**
✓ **Restrict Access to the State File (IAM Policies, RBAC)**

---

4. Summary Table

Issue	Solution
Accidentally Deleted State File	Restore from backup (S3, Terraform Cloud, local backup).
JSON Syntax Error in State File	Manually fix JSON or use jq for validation.
Corrupted State Due to Partial Apply	Run terraform refresh to sync state.
Completely Lost State	Re-import using terraform import.

6. Why do we store statefile in S3 not in github?

Feature	S3 (Recommended)	GitHub (Not Recommended)
Security	Uses **IAM roles & encryption**	No encryption, risks of accidental exposure
Concurrency Handling	Supports **DynamoDB locking**	No state locking, leading to conflicts
Automatic Backups	Supports **S3 versioning**	Requires manual commits
Scalability	Handles large state files	Repo size increases, slowing down Git operations

Commands:

1. Explain Terraform commands.
2. What does init,plan & apply do?
3. From where ull run Terraform apply cmd.

Where to Run?	When to Use?	Best For
Local Machine (CLI)	Small projects, personal use	Development & testing
CI/CD Pipeline (Jenkins, GitHub Actions)	Automated deployments	Production, large teams
AWS CloudShell / EC2	Managing AWS resources securely	Cloud-based execution
Terraform Cloud / Enterprise	Remote state & execution	Large-scale enterprise use

4. Cmd to create workspace
5. What is terraform init?
6. How do you validate if your resources are correctly configured? What does terraform validate command do?

Command	Explanation
`terraform init`	Initializes the Terraform working directory.
`terraform validate`	Validates the configuration files for syntax errors.
`terraform plan`	Shows the execution plan before applying changes.
`terraform apply`	Creates or updates resources as per the plan.
`terraform destroy`	Destroys all managed infrastructure resources.
`terraform fmt`	Formats Terraform code for better readability.
`terraform show`	Displays the state or plan file details in a human-readable format.
`terraform output`	Shows output values from the Terraform state.
`terraform refresh`	Updates the state file with real-world resource changes.
`terraform state list`	Lists all managed resources in the state file.
`terraform state show <resource>`	Displays details of a specific resource from the state.
`terraform state rm <resource>`	Removes a resource from the state file without deleting it.
`terraform import <resource> <id>`	Imports an existing resource into Terraform state.
`terraform taint <resource>`	Marks a resource for recreation in the next apply.
`terraform untaint <resource>`	Removes the taint from a resource.
`terraform workspace list`	Lists available Terraform workspaces.
`terraform workspace select <name>`	Switches to a specific workspace.

Command	Explanation
`terraform workspace new <name>`	Creates a new Terraform workspace.
`terraform graph`	Generates a visual representation of Terraform resources.
`terraform version`	Displays the installed Terraform version.
`terraform providers`	Lists the providers used in the configuration.

Terraform modules:

1. How do we import module in terraform?
2. Terraform modules. How to use and explain it's structure and why do we use it?

1. Importing a Module in Terraform

Terraform **modules** allow you to reuse and organize infrastructure code efficiently. You can import and use a module by defining it in your main.tf file.

---

2. Steps to Import a Module

✅ Step 1: Define the Module in main.tf

```
module "ec2_instance" {
  source   = "./modules/ec2"  # Path to the local module
  ami      = "ami-12345678"
  instance_type = "t2.micro"
}
```

✅ source can be:

- **Local path** → source = "./modules/ec2"
- **GitHub Repo** → source = "github.com/user/repo//module"
- **Terraform Registry** → source = "terraform-aws-modules/ec2-instance/aws"

✅ Step 2: Create the Module Directory

- Inside your Terraform project, create a **separate folder** for the module:

```
mkdir -p modules/ec2
```

- Add the module definition in modules/ec2/main.tf:

```
resource "aws_instance" "example" {
  ami           = var.ami
  instance_type = var.instance_type
}
```

✅ Step 3: Define Variables for the Module (modules/ec2/variables.tf)

```
variable "ami" {
  description = "Amazon Machine Image ID"
  type        = string
}

variable "instance_type" {
  description = "Instance type"
  type        = string
}
```

✅ Step 4: Initialize & Apply Terraform

Run the following commands:

```
terraform init    # Initialize the working directory
terraform plan    # Preview changes
terraform apply   # Apply the changes
```

---

3. Summary: How to Import a Terraform Module

Step	Action
Define a module	Create a folder (modules/ec2/main.tf)

Step	Action
Use the module	Call it in main.tf using module "<name>" {}
Provide variables	Define in variables.tf and pass values in main.tf
Initialize & Apply	Run terraform init, terraform plan, and terraform apply

1. Basic Folder Structure of a Terraform Module
A Terraform module typically contains the following files:

```
terraform-project/
├── main.tf          # Defines resources and module calls
├── variables.tf     # Defines input variables
├── outputs.tf       # Defines output values
├── provider.tf      # Configures cloud providers
├── terraform.tfvars # Optional: Defines variable values
├── modules/         # Directory for custom modules (optional)
│   ├── ec2/         # Example custom module
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── outputs.tf
│   │   ├── README.md
│   ├── vpc/
│   ├── s3/
```

---

2. Explanation of Key Files

File	Purpose
main.tf	Defines resources, such as EC2, S3, and RDS. Calls modules if needed.
variables.tf	Declares input variables to make configurations dynamic.
outputs.tf	Defines output values (e.g., instance IP, S3 bucket name).
provider.tf	Configures cloud provider settings (AWS, Azure, GCP).
terraform.tfvars	Stores variable values (used for automation).
modules/	Directory to organize reusable custom modules (like EC2, VPC, RDS).
README.md	(Optional) Documents how to use the module.

---

3. Summary: Terraform Module Best Practices
✓ Keep each module in a **separate directory**.
✓ Use **variables.tf** to define inputs and **outputs.tf** for outputs.
✓ Store reusable modules in **Git repositories** or **Terraform Registry**.
✓ Use **Terraform Workspaces** for managing multiple environments.

Scenario based:

1. If I create 10 ec2 resources using terraform and manually delete 2 EC2 from console then how will I know and what should I do from terraform to bring back my 2 instances?

1. How to Know That 2 Instances Are Missing?
Run the following Terraform command to check the current state:
```
terraform plan
```
✅ Terraform will detect that **2 EC2 instances are missing** and show a plan to recreate them.
Example Output:
```
- aws_instance.ec2[2] has been deleted
- aws_instance.ec2[5] has been deleted

Plan: 2 to add, 0 to change, 0 to destroy.
```
This means Terraform identified that 2 EC2 instances are missing and plans to recreate them.

---

2. What Should You Do to Restore the 2 Deleted Instances?

To bring back the missing instances, run:
```
terraform apply
```
✅ Terraform will **recreate only the missing instances** and restore the desired state.

---

3. Why Does Terraform Not Automatically Remove the 2 Deleted Instances from State?

- Terraform maintains a **state file (terraform.tfstate)** that tracks all resources it manages.
- When an EC2 instance is **manually deleted**, Terraform does not update its state automatically.
- Instead, Terraform **detects drift** (difference between actual resources & state) and **recreates missing resources** to match the configuration.

---

4. What If Terraform Still Shows Deleted Resources? (Force Syncing the State)
If terraform plan still shows the deleted instances in the state file, **refresh the state**:
```
terraform refresh
```
This updates the state file to match actual infrastructure.

---

5. What If I Want Terraform to Forget the Deleted EC2 Instances Without Recreating Them?
If you don't want Terraform to recreate them, remove the deleted resources from Terraform state manually:
```
terraform state rm aws_instance.ec2[2]
terraform state rm aws_instance.ec2[5]
```
🚨 **Warning:** This will remove the instances **from the state file only**, but won't delete actual instances if they exist.

---

6. Summary Table

Action Taken	Terraform Command	Result
Check if instances are missing	terraform plan	Shows missing EC2s
Restore missing instances	terraform apply	Recreates missing instances
Update state file if needed	terraform refresh	Syncs state with real-world infra
Remove missing instances from state (without recreating them)	terraform state rm <resource>	Removes from state file

2. If someone has deleted or done some any changes to my infrastructure using terraform. How do I know what changes were executed and who executed those changes?

✅ **Method 1: Use terraform plan to Detect Drift**
Run the following command to check if the infrastructure has changed:
terraform plan
🔹 If any resources were **deleted, modified, or added**, Terraform will display the drift and plan to **restore the original configuration**.

✅ **Method 2: Use AWS Resource Change Tracking**
Terraform itself does **not log changes made outside Terraform**, but you can track manual changes using **AWS CloudTrail**.

- **AWS CloudTrail Logs** track API actions (e.g., terraform apply, terraform destroy, manual edits).
- You can query CloudTrail logs to find out **who modified your resources**.

✅ **Method 3: Use Terraform State History**
If changes were made using Terraform, check the Terraform **state history** (if remote backend like **S3 with DynamoDB lock** is used).
terraform state list

Scenario	How to Find Out?
Changes made via Terraform	Check Terraform remote backend logs (e.g., S3, DynamoDB, Git commit history).
Changes made manually via AWS Console	Use **AWS CloudTrail** to check API logs.
Changes made via AWS CLI or SDK	Check **CloudTrail logs** with lookup-events.
Changes made via Terraform but not committed to Git	Check **Terraform state history** and **Git history**.

Best Practices to Prevent Unauthorized Changes

✅ **Enable AWS CloudTrail** – Monitor all AWS API calls.

✅ **Use Terraform Remote Backend (S3 + DynamoDB Locking)** – Prevents unauthorized state file changes.

✅ **Implement IAM Policies** – Restrict Terraform access based on roles.

✅ **Use Git for Version Control** – Always commit main.tf and terraform.tfstate.

✅ **Set Up Terraform Audit Logs** – Store logs of who applied changes.

3. If your code (terraform) is in GitHub and you have to make some changes so how do you do it from first step? Don't you use pull request?

Create a Pull Request (PR) on GitHub

- Go to **GitHub → Your Repository**.
- Click on **Compare & pull request**.
- Add a description of the changes.
- Assign reviewers for code review.

Review and Merge the Pull Request

- Once approved, merge the PR into the main branch.
- You can do this via the GitHub UI or via CLI:

```
git checkout main
git pull origin main
git merge feature/update-ec2-instance
git push origin main
```

4. How do you execute same scripts in various environments? How do we know which environment/workspace we are in and how exactly terraform knows to pick it from variables.tf?

Method	Best For	How to Use
Workspaces	Isolated environments using the same code	terraform workspace new dev → terraform apply
Variable Files (.tfvars)**	Parameterized configurations per environment	terraform apply -var-file="dev.tfvars"
Separate State Files (S3 Backend)	Team collaboration & remote state management	Configure backend "s3" with different keys

- **Instead of using workspaces, you can define different variable files(dev.tfvars, staging.tfvars, prod.tfvars) for each environment:

Example dev.tfvars:

```
instance_type = "t2.micro"
region        = "us-east-1"
```

staging.tfvars:

```
instance_type = "t3.medium"
region        = "us-west-2"
```

🔽 Apply Terraform with the Environment-Specific Variable File

```
terraform apply -var-file="dev.tfvars"
terraform apply -var-file="staging.tfvars"
terraform apply -var-file="prod.tfvars"
```

🔹 **How Terraform Knows Which Variables to Pick?**

- Terraform picks values from the **specified .tfvars file** when you pass the -var-file flag.
- If no -var-file is passed, Terraform looks for terraform.tfvars by default.

5. In Terraform your colleague manually created few resources & after few days when u r trying to create the same resource will it get recreate or not

Scenario	Will Terraform Recreate It?	Solution
Resource exists but not in Terraform state	❌ No, but Terraform will try to create it again, leading to conflicts.	Use terraform import
Resource exists but Terraform does not know	✅ Yes, if AWS allows duplicate resources. ❌ No, if AWS enforces uniqueness.	Import resource or manually delete the old one.
Resource exists in state but was modified manually	🔄 Terraform will override manual changes to match .tf configuration.	Use terraform refresh or update .tf files before apply.

✅ **Best Practice:** Always **import manually created resources** into Terraform before managing them to avoid conflicts.

6. If I want to create 10 EC2 machines for multiple accounts what will be ur approach in Terraform?

Approach	Use Case	Best For
AWS Profiles (~/.aws/credentials)	Local execution with CLI profiles	Developers working locally
Assume Role with Provider Aliases	Multi-account deployments in automation	CI/CD pipelines, Terraform Cloud

🔥 **Recommended for Production:**

- Use **Assume Role (assume_role)** to avoid storing credentials.
- Store **Terraform state in an S3 backend** for consistency across accounts.

7. I do not want to store my access key and secret key in terraform as it's not a good practice. How you'll use terraform in this case?

Best Practices for Secure Authentication in Terraform

1. Use AWS IAM Roles with Assume Role (Best for CI/CD & Multi-Account Setup)
2. Use AWS CLI Credential Profiles (For Local Execution)
3. Use Environment Variables (For CI/CD Pipelines)
4. Use AWS SSO (For Enterprise & Large Organizations)

🚀 **Final Recommendation**

For **production environments**, always use **IAM roles (assume_role)** or **AWS SSO** instead of storing credentials.

For **local development**, use **AWS CLI profiles** or **environment variables**.

8. How ull create Terraform files for ft,stg

Approach	Best For
Workspaces	Small to medium projects with **minimal differences** across environments.
Separate Files	Large-scale projects with **different configurations** for each environment.

9. I want to launch services (EC2) in different regions using same terraform script how do you do it?

Approach	Best For
Multiple provider blocks (Approach 1)	When you need to manually define & control instances in specific regions.
Loop (for_each) (Approach 2)	When you need **scalability & automation** across multiple regions dynamically.

10. I want to create 5 instances/100 instance how do we do using single terraform form?

Approach	Best For
`count`	When all instances are **identical** (e.g., 100 t3.micro instances).
`for_each`	When each instance has a **unique configuration** (e.g., different AMI, instance type).

11. How do you refer below data as variables and create the resources?

```
instance_config={
  instance1 = { instance_type = "t2.micro", ami = "ami-12345" }
  instance2 = { instance_type = "t2.medium", ami = "ami-67890" }
}
```

Answer: Use for_each

```
resource "aws_instance" "ec2_instances" {
  for_each      = var.instance_config
  ami           = each.value.ami
  instance_type = each.value.instance_type
  tags = {
    Name = each.key
  }
}
```

12. I have to create my resources in various cloud providers (multi provider) like Azure, AWS and in various regions. How do I do it?

To deploy resources **across multiple cloud providers** (AWS, Azure) **and multiple regions**, follow these best practices:

✅ **Use multiple provider blocks** (with alias for different regions/providers).

✅ **Use for_each or count to iterate over regions dynamically**.

✅ **Use modules** to structure the code for better management.

13. Two users will be working in creating infra and the state file is locked, Dev B is not able to work. How do we unlock it?

🔒 **Why is the State File Locked?**

- **Terraform state is stored remotely (e.g., AWS S3 with DynamoDB, Azure Storage, or Terraform Cloud).**
- **Locking prevents simultaneous updates, ensuring consistency.**
- **If an operation (e.g., terraform apply) crashes or is interrupted, the lock may remain active.**

Force Unlock the State (Use with Caution!)

If you are sure no other user/process is modifying the state, run:

```
terraform force-unlock <LOCK_ID>
```

14. If you're storing your terraform script in GitHub and two developers are trying to access it then what will be the issue? How it'll work?

When two developers try to access and modify the **same Terraform script** stored in **GitHub**, potential issues can arise, such as:

Possible Issues:

1. **State File Locking Conflict** – If both developers apply changes at the same time, the **Terraform state file** (stored remotely, e.g., in S3 with DynamoDB locking) may get **locked**, preventing concurrent updates.
2. **Merge Conflicts** – If both developers modify the same script and push changes to GitHub, **merge conflicts** may occur.
3. **Unintended Overwrites** – If one developer runs terraform apply without pulling the latest changes, it could **overwrite** the other's modifications.

How It Works & Best Practices:

1. **Use Remote State with Locking** – Store the Terraform state file in **S3 with DynamoDB locking** to prevent concurrent updates.

2. **Use Feature Branches** – Developers should create **separate branches**, work on changes, and then merge via **Pull Requests (PRs)** to avoid conflicts.
3. **Plan Before Apply** – Run terraform plan before terraform apply to preview changes and avoid unintended modifications.
4. **CI/CD Integration** – Implement a **GitHub Actions pipeline** or **Terraform Cloud** to automate validation and enforce policies before applying changes.
5. **Role-Based Access Control (RBAC)** – Use **IAM roles and GitHub permissions** to restrict direct state file edits.

Other Topics in terraform:

1. What are provisioners in Terraform? What are some of the built-in provisioners available in Terraform?

Provisioners in **Terraform** are used to **execute scripts or commands** on a **local or remote machine** during resource creation or destruction. They help in **configuration management** by setting up dependencies, installing software, or executing custom scripts after a resource is provisioned.

⚠ **Best Practice**: Avoid using provisioners when possible, as Terraform is declarative, and provisioners introduce **imperative** steps that can cause issues in automation. Instead, use configuration management tools like **Ansible, AWS SSM, or cloud-init**.

Built-in Provisioners in Terraform

Provisioner	Description	Use Case
local-exec	Runs a command on the **machine running Terraform**	Sending notifications, logging, triggering external scripts
remote-exec	Runs a command **inside a remote resource** (e.g., an EC2 instance)	Running configuration scripts, installing software
file	Transfers files from the **local machine** to a **remote machine**	Copying configuration files, application code
chef *(deprecated)*	Runs Chef configuration on a VM	Legacy configuration management
puppet *(deprecated)*	Runs Puppet configuration on a VM	Legacy configuration management

---

Key Takeaways:

✅ **Use provisioners only as a last resort** – prefer user-data, cloud-init, or configuration tools.

✅ **Provisioners should be idempotent** – ensure that running them multiple times does not cause issues.

✅ **Use on_failure to handle failures** – e.g., on_failure = continue to avoid Terraform errors stopping the process.

2. How do you manage AWS credentials and permissions in Terraform?

Managing AWS credentials properly is **crucial for security** and **best practices**. Instead of hardcoding credentials, use secure methods like environment variables, IAM roles, or AWS credentials files.

---

1. Using IAM Roles (Best Practice for EC2 & AWS Services)

- If running Terraform on an **EC2 instance, Lambda, or ECS**, use **IAM roles** with the necessary permissions.
- Terraform will automatically authenticate using the **EC2 instance profile**.

✅ **Steps:**

1. Create an **IAM Role** with required policies.
2. Attach the IAM Role to the EC2 instance.
3. Terraform will automatically authenticate.

```
provider "aws" {
  region = "us-east-1"
}
```

- **No need to store credentials manually.**
- **Most secure method** when running on AWS services.

---

2. Using Environment Variables (Secure & Recommended)

Set AWS credentials as **environment variables** on your local machine or CI/CD pipeline.

✅ **Steps:**

1. **Set environment variables** on your machine:
   ```
   export AWS_ACCESS_KEY_ID="your-access-key"
   export AWS_SECRET_ACCESS_KEY="your-secret-key"
   ```
2. Terraform automatically picks up these variables.

```
provider "aws" {
  region = "us-east-1"
}
```

- **No hardcoding in Terraform files.**
- Works across different environments.

---

3. Using AWS Credentials File (~/.aws/credentials)

- Store credentials in a local AWS credentials file.
- Terraform will automatically read them.

✅ **Steps:**

1. Configure AWS CLI:
   ```
   aws configure
   ```
2. Store credentials in ~/.aws/credentials:
   ```
   [default]
   aws_access_key_id = your-access-key
   aws_secret_access_key = your-secret-key
   ```
3. Terraform automatically reads them:

```
provider "aws" {
  region = "us-east-1"
  profile = "default"
}
```

- Useful when switching between multiple AWS accounts.

---

4. Using terraform.tfvars (Not Recommended)

- Credentials can be stored in a .tfvars file, but this is **not secure**.
- **Never commit this file to Git!**

```
aws_access_key = "your-access-key"
aws_secret_key = "your-secret-key"
```

Pass it during execution:

```
terraform apply -var-file="credentials.tfvars"
```

Use variables in Terraform:

```
provider "aws" {
  region     = "us-east-1"
  access_key = var.aws_access_key
  secret_key = var.aws_secret_key
}
```

🚨 **Avoid this method unless necessary**.

---

5. Using AWS SSO for Authentication

- If your organization uses AWS **SSO (Single Sign-On)**, you can authenticate Terraform with **SSO-based credentials**.

✅ **Steps:**

1. **Login using AWS SSO**:
   ```
   aws sso login --profile my-profile
   ```
2. Configure Terraform to use the profile:
   ```
   provider "aws" {
     region  = "us-east-1"
     profile = "my-profile"
   }
   ```

- **No need to store long-term credentials.**
- **Good for enterprise environments.**

Best Practices for AWS Credentials in Terraform

Method	Security Level	Best For
IAM Roles	🔒🔒🔒🔒🔒	EC2, Lambda, ECS, CI/CD
Environment Variables	🔒🔒🔒🔒	Local Development, CI/CD
AWS Credentials File	🔒🔒🔒	Developers switching accounts
AWS SSO	🔒🔒🔒🔒	Enterprise authentication
terraform.tfvars (Hardcoded)	❌ (Not Secure)	Avoid using this method

Key Takeaways

✅ **Never store credentials in Terraform files (main.tf).**

✅ **Use IAM roles** when running Terraform on AWS.

✅ **Use environment variables or AWS credentials file** for local development.

✅ **AWS SSO is best for enterprises** using SSO-based authentication.

3. What is output variables and .vars file in terraform? Why and how to use it?

1. Terraform Output Variables

Output variables in Terraform allow you to display values after the execution of a Terraform plan or apply. They are useful for sharing information between modules or showing essential infrastructure details.

Why Use Output Variables?

✅ Print important values like **instance ID, IP address, or S3 bucket name** after deployment.

✅ Share information between Terraform **modules**.

✅ Reference values in **scripts or automation workflows**.

How to Use Output Variables?

Define outputs in a outputs.tf file:

```
output "instance_public_ip" {
  value = aws_instance.my_instance.public_ip
  description = "The public IP of the EC2 instance"
}
```

💡 **How to Display Outputs?**

After running terraform apply, view outputs using:

```
terraform output
```

To get a specific output:

```
terraform output instance_public_ip
```

2. Terraform .tfvars **File**

A **.tfvars file** is used to **store variable values separately** instead of defining them directly in the Terraform configuration.

Why Use .tfvars?

✅ **Organize variable values** instead of hardcoding in variables.tf.

✅ **Easier environment management** (e.g., dev, staging, production).

✅ **Automate Terraform commands** without manually passing -var arguments.

How to Use .tfvars?

1. Define Variables in variables.tf

```
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
}

variable "instance_count" {
  description = "Number of EC2 instances"
  type        = number
}
```

2. Store Values in a .tfvars File (terraform.tfvars)

```
instance_type  = "t2.micro"
instance_count = 3
```

3. Use the .tfvars File in Terraform Commands

Terraform automatically loads terraform.tfvars, but you can also specify it manually:

```
terraform apply -var-file="terraform.tfvars"
```

Key Differences: Output Variables vs .tfvars

Feature	Output Variables	.tfvars File
Purpose	Show values after execution	Provide input values before execution
Usage	Display outputs, share between modules	Store variables for configuration
Execution	Used **after** terraform apply	Used **before** terraform plan/apply
Example	terraform output instance_public_ip	terraform apply -var-file="terraform.tfvars"

4. How do you install and use terraform in visual studio/visual studio code?

1. Install Terraform

You need to download Terraform before using it in VS Code.

◆ **Install Terraform on Windows**

1. **Download Terraform Binary**
   - Visit Terraform's official website
   - Download the **Windows** version (.zip file).
2. **Extract and Set Environment Path**
   - Extract the .zip file.
   - Move the `terraform.exe to C:\terraform (or any preferred location).`
   - Add `C:\terraform` to the **System Environment Variables**:
     - Go to **Control Panel → System → Advanced system settings**
     - Click **Environment Variables → System Variables**
     - Select **Path** → Click **Edit** → Click **New** and add C:\terraform
   - Restart **Command Prompt** (cmd) or **PowerShell**.
   - Verify installation by running: `terraform -version`

◆ **Install Terraform on Linux/Mac**

1. **Use a package manager**:
   ```
   sudo apt update && sudo apt install -y terraform  # Ubuntu/Debian
   brew install terraform  # macOS
   ```
2. **Verify Installation**: `terraform -version`

---

2. Install VS Code and Terraform Extension

1. **Download and Install Visual Studio Code**
   - Get it from: VS Code Official Site
2. **Install Terraform Extension in VS Code**
   - Open VS Code
   - Go to **Extensions (Ctrl+Shift+X)**
   - Search for **"Terraform"**
   - Install **"HashiCorp Terraform"** extension

---

3. Setup Terraform Project in VS Code

◆ **Create a New Terraform Project**

1. Open VS Code and create a **new folder** (e.g., `terraform-project`).
2. Inside the folder, create a **new file** called main.tf.
3. Add the following Terraform configuration to `main.tf`:
   ```
   provider "aws" {
     region = "us-east-1"
   }

   resource "aws_instance" "example" {
   ```

```
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
}
```

---

4. Initialize and Apply Terraform Configuration
   1. Open **Terminal (Ctrl+`)** in VS Code.
   2. Run the following Terraform commands:
      ```
      terraform init       # Initialize Terraform project
      terraform plan       # Preview changes before applying
      terraform apply      # Deploy resources
      ```
   3. If prompted, type yes to proceed.
   4. After applying, Terraform will create the resources defined in main.tf.

---

5. Destroy Terraform Resources
To delete all created resources, run: `terraform destroy`
Confirm with yes when prompted.

🎯 **Summary**

✅ Install Terraform and configure the system path.

✅ Install VS Code and Terraform extension.

✅ Create a Terraform project (`main.tf`).

✅ Initialize (`terraform init`), plan (`terraform plan`), and apply (`terraform apply`) configurations.

✅ Destroy (`terraform destroy`) resources when needed.

5. Are you aware of Terraform cycles?

Yes, Terraform cycles refer to **circular dependencies** that can occur in resource relationships, causing errors in infrastructure deployment.

◆ **What are Terraform Cycles?**

A **Terraform cycle** happens when a resource **directly or indirectly depends on itself**, creating an infinite loop that Terraform cannot resolve.

◆ **Example of a Terraform Cycle Issue**

```
resource "aws_instance" "example" {
  ami           = "ami-123456"
  instance_type = "t2.micro"

  network_interface {
    network_interface_id = aws_network_interface.example.id
  }
}

resource "aws_network_interface" "example" {
  subnet_id   = "subnet-123456"
  private_ips = ["10.0.0.10"]

  attachment {
    instance = aws_instance.example.id  # 🚨 CIRCULAR DEPENDENCY!
  }
}
```

Why is this a cycle?

- The aws_instance needs aws_network_interface to attach.
- The aws_network_interface also requires aws_instance to attach.
- This creates a dependency loop.

◆ **How to Fix Terraform Cycles?**

✅ **1. Use depends_on to Explicitly Define Dependencies**

```
resource "aws_instance" "example" {
  ami           = "ami-123456"
  instance_type = "t2.micro"
```

```
  depends_on = [aws_network_interface.example]  # 🚀 Fixes dependency loop
}

resource "aws_network_interface" "example" {
  subnet_id   = "subnet-123456"
  private_ips = ["10.0.0.10"]
}
```

✅ `depends_on` tells Terraform to **create `aws_network_interface` first** before creating `aws_instance`.

✅ 2. Use terraform apply in Steps

If there are complex interdependencies, you can **apply resources step by step** instead of all at once.

```
terraform apply -target=aws_network_interface.example
terraform apply
```

This ensures **network interfaces** are created **before** instances reference them.

✅ 3. Modularize the Infrastructure

If cycles persist, split dependent resources into **separate Terraform modules**. This reduces direct dependencies.

◆ Summary

- **Terraform cycles** occur due to circular dependencies.
- **Fix them** using `depends_on`, modularization, or staged `terraform apply`.
- Terraform **automatically detects and prevents cycles**, ensuring correct dependency order

6. What are providers in Terraform?
7. Can I have multiple same providers in terraform configuration?

A **provider** in Terraform is a plugin that allows Terraform to interact with **cloud platforms (AWS, Azure, GCP), SaaS services (GitHub, Kubernetes), and other APIs**. It acts as a bridge between Terraform and the service you're managing.

◆ Examples of Terraform Providers

Provider	Use Case
aws	Managing AWS resources (EC2, S3, VPC, etc.)
azurerm	Provisioning resources in Microsoft Azure
google	Managing Google Cloud resources
kubernetes	Deploying applications in Kubernetes
github	Managing GitHub repositories, teams, and settings
helm	Deploying Helm charts in Kubernetes
vault	Interacting with HashiCorp Vault for secrets management

---

◆ How to Define a Provider in Terraform?

You must define providers in your **Terraform configuration file (main.tf)** before using them.

Example: AWS Provider

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"  # Locking to a specific version
    }
  }
}

provider "aws" {
```

```
  region  = "us-east-1"
  profile = "default"  # Uses AWS credentials from ~/.aws/credentials
}
```

✅ This configuration ensures that Terraform will use the **AWS provider** to create resources in `us-east-1`.

◆ How to Use Multiple Providers in Terraform?

You can define multiple providers **in a single Terraform configuration**.

Example: Deploying in AWS & Azure

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

provider "azurerm" {
  features {}
}
```

✅ This allows Terraform to create resources in **both AWS and Azure**.

◆ Using Multiple AWS Accounts with Provider Aliases

If you're working with multiple AWS accounts, you can use **provider aliases**.

Example: Deploying in Two AWS Accounts

```
provider "aws" {
  alias  = "account1"
  region = "us-east-1"
  profile = "account1-profile"
}

provider "aws" {
  alias  = "account2"
  region = "us-west-2"
  profile = "account2-profile"
}

resource "aws_s3_bucket" "bucket1" {
  provider = aws.account1
  bucket   = "my-bucket-1"
}

resource "aws_s3_bucket" "bucket2" {
  provider = aws.account2
  bucket   = "my-bucket-2"
}
```

✅ This example **deploys S3 buckets in two different AWS accounts**.

◆ Where Does Terraform Store Providers?

Terraform **downloads providers** from the Terraform Registry and stores them in:

- **Linux/Mac:** `~/.terraform.d/plugins/`
- **Windows:** `%APPDATA%\terraform.d\plugins\`

You can manually install providers if needed.

◆ **How to Upgrade a Terraform Provider?**
To upgrade providers, use:
```
terraform init -upgrade
```
✅ This updates all providers to the latest compatible versions.

◆ **Summary**

Feature	Details
What is a provider?	A plugin that allows Terraform to interact with cloud services
Examples	AWS, Azure, GCP, Kubernetes, GitHub, Helm
Multiple Providers	Can configure multiple providers (AWS + Azure)
Aliases	Used for deploying resources in multiple AWS accounts
Version Management	Lock provider versions with required_providers
Updating Providers	Use terraform init -upgrade

8. What kind of version are you using for Terraform? Open-source or licensed? Which version of Terraform do you use?

I primarily use **Terraform Open-Source (Community Edition)** unless the project requires **Terraform Enterprise (licensed)** or **Terraform Cloud (SaaS version)** for collaboration and governance.
As for the **Terraform version**, I prefer using the latest stable version. The most commonly used versions are:

- **Terraform 1.5.x / 1.6.x** – Latest stable versions with improved performance, CLI enhancements, and better state management.
- **Terraform 0.14 - 1.0.x** – Older versions but still used in some legacy projects.

9. Have u worked on Terraform Target nodes.

Yes, I have worked with **Terraform target nodes** using the `-target` flag.
The `-target` option in Terraform allows you to apply or destroy specific resources without affecting the entire infrastructure. It is useful for troubleshooting, incremental deployments, and managing individual components.
Example:
If you have an EC2 instance in your Terraform configuration and want to apply changes only to it, you can run:
```
terraform apply -target=aws_instance.my_instance
```
Similarly, to destroy a specific resource:
```
terraform destroy -target=aws_instance.my_instance
```

Problem Statement 1: I was working on an AWS infrastructure project where we had multiple resources—EC2 instances, RDS databases, S3 buckets, and a VPC. A change was required for an **EC2 instance**, but applying Terraform to the entire infrastructure wasn't an option due to strict approval processes for modifying databases and networking components.

Problem Statement 2: We had an AWS infrastructure with multiple **EC2 instances, S3 buckets, and an RDS database**. Due to a cost-optimization initiative, we needed to delete **only a specific EC2 instance** without affecting other resources. Running terraform destroy would remove everything, which was not an option.

10. Have you created any AWS services using terraform and what all have you created using terraform?

Yes, I have extensively used Terraform to **automate AWS infrastructure provisioning**. Below are some key AWS services I have created using Terraform:
1. Compute Services:

- **EC2 Instances** (with specific AMI, instance type, and security groups)
- **Auto Scaling Groups** (for scaling EC2 instances based on demand)

2. Networking & Security:

- **VPC, Subnets, Route Tables, Internet Gateway, NAT Gateway**

- **Security Groups and NACLs** (Network ACLs for traffic control)
- **Application Load Balancer (ALB)**

3. Storage & Database:

- **S3 Buckets** (with lifecycle policies, encryption, versioning)
- **EBS Volumes & EFS (Elastic File System)**
- **RDS Databases** (MySQL, PostgreSQL, Aurora)

11. How do you fetch data from one resource to another in terraform?

In Terraform, you fetch data from one resource to another using **resource references** and **data sources**. Below are the primary methods:

1. Using Resource Attributes Directly

You can use the attributes of one resource in another by referencing it with `${resource_type.resource_name.attribute}`.

Example: Passing VPC ID to a Subnet

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "public_subnet" {
  vpc_id     = aws_vpc.main.id  # Fetching VPC ID from aws_vpc.main
  cidr_block = "10.0.1.0/24"
}
```

2. Using data Sources

When you want to fetch existing resources (not created by Terraform), use **data sources**.

Example: Fetching an Existing VPC

```
data "aws_vpc" "existing_vpc" {
  filter {
    name   = "tag:Name"
    values = ["my-vpc"]
  }
}

resource "aws_subnet" "public_subnet" {
  vpc_id     = data.aws_vpc.existing_vpc.id  # Fetching existing VPC ID
  cidr_block = "10.0.1.0/24"
}
```

3. Using output Variables

When you need to pass data between modules, use **output variables**.

Example: Output VPC ID

```
output "vpc_id" {
  value = aws_vpc.main.id
}
```

You can then **fetch this output** in another module by using:

```
module "network" {
  source = "./network-module"
}

resource "aws_instance" "web" {
  subnet_id = module.network.vpc_id  # Fetching VPC ID from another module
}
```

12. What are the various important files present in terraform?

File Name	Purpose
`main.tf`	Defines the primary infrastructure resources.
`variables.tf`	Declares input variables for configuration.
`terraform.tfvars`	Provides default values for variables.
`outputs.tf`	Defines default output values to be used elsewhere.

File Name	Purpose
`providers.tf`	Configures provider details (AWS, Azure, etc.).
`backend.tf`	Configures remote state storage backend.
`data.tf`	Fetches external or existing AWS resources.
`modules/`	Contains reusable Terraform modules.
`.terraform.lock.hcl`	Locks provider versions to maintain consistency.
`.terraform/`	Stores provider plugins and state metadata.
`terraform.tfstate`	Stores the current state of the infrastructure (DO NOT EDIT manually).
`terraform.tfstate.backup`	Backup of the last known state.
`.gitignore`	Prevents committing sensitive files (e.g., terraform.tfstate).

13. How do you execute terraform on multiple environments?

Terraform allows you to manage multiple environments (e.g., **dev, staging, production**) using **workspaces, directories, or variable files**. Below are the different approaches:

1. Using Workspaces

Terraform workspaces allow you to create isolated state files for different environments.

Steps to use workspaces:

```
terraform workspace new dev      # Create a new workspace for development
terraform workspace new staging  # Create another workspace for staging
terraform workspace select dev   # Switch to the dev workspace
terraform apply                  # Apply changes specific to the dev
environment
```

💡 *Each workspace maintains a separate Terraform state file.*

---

2. Using Separate Variable Files (terraform.tfvars)

Each environment can have its own variable file (**dev.tfvars, staging.tfvars, prod.tfvars**).

Example:

```
terraform apply -var-file="dev.tfvars"
terraform apply -var-file="staging.tfvars"
terraform apply -var-file="prod.tfvars"
```

💡 *This approach is useful when you need minor differences across environments.*

---

3. Using Different Folders for Each Environment

Create separate directories like dev/, staging/, prod/ and maintain Terraform configurations separately.

Example Folder Structure:

```
terraform/
├── environments/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── terraform.tfvars
│   ├── staging/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── terraform.tfvars
│   ├── prod/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── terraform.tfvars
```

💡 *Useful when environments need different configurations or permissions.*

---

14. Explain folder structure of terraform.

A well-structured Terraform project ensures **maintainability, reusability, and scalability**. Below is a **recommended folder structure** for organizing Terraform configurations effectively.

---

1. Basic Terraform Project Structure

For a **single environment**, the folder structure looks like this:

```
terraform-project/
│── main.tf             # Main configuration file
│── variables.tf        # Input variables
│── outputs.tf          # Output values
│── provider.tf         # Provider configuration (AWS, Azure, etc.)
│── terraform.tfvars    # Variable values
│── backend.tf          # Remote backend configuration
│── .terraform/         # Terraform hidden folder (auto-generated)
│── terraform.tfstate   # State file (local or remote)
│── terraform.tfstate.backup  # Backup of state file
│── .terraform.lock.hcl # Dependency lock file
```

✅ *Best for small projects or single-environment deployments.*

---

2. Multi-Environment Terraform Structure

For managing multiple environments (**dev, staging, production**), use separate folders:

```
terraform-project/
│── environments/
│    ├── dev/
│    │    ├── main.tf
│    │    ├── variables.tf
│    │    ├── terraform.tfvars
│    │    ├── backend.tf
│    ├── staging/
│    │    ├── main.tf
│    │    ├── variables.tf
│    │    ├── terraform.tfvars
│    │    ├── backend.tf
│    ├── prod/
│         ├── main.tf
│         ├── variables.tf
│         ├── terraform.tfvars
│         ├── backend.tf
│── modules/              # Reusable Terraform modules
│    ├── networking/      # Networking module (VPC, Subnets, etc.)
│    ├── compute/         # Compute module (EC2, AutoScaling, etc.)
│    ├── database/        # Database module (RDS, DynamoDB, etc.)
│── global/               # Global configurations (backend, IAM policies,
etc.)
│── scripts/              # Automation scripts (init, destroy, etc.)
│── README.md             # Documentation
│── .gitignore            # Ignore sensitive files like .terraform.tfstate
```

✅ *Best for managing multiple environments with reusable modules.*

---

3. Modularized Terraform Structure

For **large-scale infrastructure**, modularizing Terraform code is recommended:

```
terraform-project/
│── modules/
│    ├── vpc/              # VPC module
│    │    ├── main.tf
│    │    ├── variables.tf
│    │    ├── outputs.tf
│    ├── ec2/              # EC2 module
│    │    ├── main.tf
│    │    ├── variables.tf
│    │    ├── outputs.tf
│    ├── rds/              # RDS module
│    │    ├── main.tf
│    │    ├── variables.tf
│    │    ├── outputs.tf
│── environments/
```

```
│   ├── dev/
│   │   ├── main.tf       # Calls modules
│   │   ├── variables.tf
│   │   ├── terraform.tfvars
│   ├── prod/
│   │   ├── main.tf       # Calls modules
│   │   ├── variables.tf
│   │   ├── terraform.tfvars
├── backend.tf            # S3/DynamoDB backend configuration
├── provider.tf           # AWS/Azure/GCP provider configuration
├── outputs.tf            # Outputs for resources
├── terraform.tfvars      # Default variable values
├── .gitignore            # Ignore Terraform state files
└── README.md             # Documentation
```

✅ *Best for large teams working on different parts of infrastructure.*

Which structure should you use?

- ✅ **Small Project** → Basic structure
- ✅ **Multiple Environments** → Multi-env structure
- ✅ **Enterprise / Large Teams** → Modularized structure

15. What is count, for and for_each in terraform?

Terraform provides **count**, **for**, and **for_each** to handle loops, dynamic resources, and iterating over maps/lists.

---

1. count (Index-Based Loop)

- Used to create **multiple instances** of a resource using an index.
- Best for **creating multiple resources of the same type**.

Example

Creates **3 EC2 instances** with dynamic names:

```
resource "aws_instance" "example" {
  count         = 3
  ami           = "ami-12345678"
  instance_type = "t2.micro"

  tags = {
    Name = "EC2-${count.index}"
  }
}
```

🔹 `count.index` starts from **0**, so instances will be named **EC2-0, EC2-1, EC2-2**.

🔹 `count` works with **lists and numbers** but is **not ideal for maps**.

---

2. for (Loop Inside Expressions)

- Used inside **expressions** like locals, output, and variables.
- Helps in transforming **lists/maps**.

Example: Convert List to Map

```
variable "names" {
  default = ["dev", "staging", "prod"]
}

output "env_map" {
  value = { for idx, name in var.names : name => idx }
}
```

🔹 Output will be:

```
{
  "dev": 0,
  "staging": 1,
  "prod": 2
}
```

---

3. for_each (Key-Value Iteration)

- Used to loop over **maps or sets**.

- More flexible than count, allowing **dynamic resource creation** with custom keys.

Example: Creating Security Groups Dynamically

```
variable "ports" {
  default = {
    "http"  = 80
    "https" = 443
    "ssh"   = 22
  }
}

resource "aws_security_group_rule" "allow_ports" {
  for_each          = var.ports
  type              = "ingress"
  from_port         = each.value
  to_port           = each.value
  protocol          = "tcp"
  security_group_id = "sg-12345678"
  cidr_blocks       = ["0.0.0.0/0"]
}
```

🔹 This dynamically creates security rules for **ports 80, 443, and 22**.

🔹 Unlike `count`, it allows **non-sequential keys (map-based iteration)**.

Comparison Table

Feature	`count`	`for`	`for_each`
Works with	Lists, Numbers	Lists, Maps (inside expressions)	Maps, Sets
Best For	Creating multiple similar resources	Transforming data	Iterating over maps/sets dynamically
Uses index?	✅ Yes (count.index)	✅ Yes (for idx, value in list)	❌ No (uses each.key and each.value)
Supports maps?	❌ No	✅ Yes	✅ Yes

When to Use What?

✅ Use **`count`** when creating **identical resources** (like multiple EC2 instances).

✅ Use **`for`** inside expressions to **transform data** (like list-to-map conversion).

✅ Use **`for_each`** when **iterating over maps or sets dynamically** (like security groups).

Scripts using terraform:

1. Terraform script for LBR.
2. What & all u created with Terraform...I wanna build a application from Terraform what will be ur approach
3. Give the terraform configuration for creating a single EC2 instance on AWS?
4. Can you explain terraform script with EC2 and Application Load balancer and target groups and how do you link EC2 with load balancers?

To link EC2 instances with a Load Balancer (ALB or ELB) in AWS, you need to register the instances with a **Target Group**, which is associated with the Load Balancer.

5. Write a resource block for EC2 in terraform script.
6. Explain how do we and what do we require to configure vpc and subnet in terraform.
7. Recent project worked on terraform.

I worked on automating the provisioning of a highly available and scalable **web application infrastructure** in AWS using **Terraform**. The goal was to deploy a **multi-tier architecture** with automated scaling, security, and monitoring.

Architecture Components

1. Networking (VPC & Subnets)

- Created a **custom VPC** with **three-tier architecture** (Public, Private, and Database Subnets).
- Configured **Internet Gateway & NAT Gateway** for internet access.
- Defined **Route Tables** for proper routing between subnets.

2. Compute (EC2 & Auto Scaling)

- Provisioned **EC2 Instances** (Application Servers) in **private subnets**.
- **Auto Scaling Group** (ASG) with policies to handle traffic spikes.
- Configured an **Application Load Balancer (ALB)** to distribute traffic.

3. Database (RDS & DynamoDB)

- Deployed an **RDS MySQL database** in a **private subnet** with high availability.
- Implemented **DynamoDB for session management**.

4. Storage (S3 & EFS)

- Created an **S3 bucket** for **static content** and **logs**.
- Configured **EFS (Elastic File System)** for shared file storage between EC2 instances.

5. Security & IAM

- **IAM Roles & Policies** for EC2, S3, and RDS access.
- Defined **Security Groups & NACLs** for restricted access.
- Implemented **AWS KMS encryption** for security.

6. Monitoring & Logging

- Configured **CloudWatch Alarms** for **CPU, Memory, and Latency Monitoring**.
- Set up **CloudTrail for logging API calls**.
- Implemented **VPC Flow Logs** for network monitoring.

7. CI/CD Pipeline with Terraform & Jenkins

- Integrated **Terraform with Jenkins** for infrastructure deployment.
- Used **S3 backend with DynamoDB locking** for Terraform state management.
- Created **GitHub Actions for Terraform validation & automated deployments.**

Tools & Technologies Used

- **Terraform** (Infrastructure as Code)
- **AWS Services** (EC2, VPC, RDS, S3, ALB, Auto Scaling, CloudWatch, DynamoDB)
- **Jenkins & GitHub Actions** (CI/CD)
- **S3 + DynamoDB** (Terraform state management)

Master Script for all above questions:

```
provider "aws" {
  region = "us-east-1"
}

# VPC Creation
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
}

# Subnets
resource "aws_subnet" "public_subnet" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true
  availability_zone       = "us-east-1a"
}

resource "aws_subnet" "private_subnet" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.2.0/24"
  availability_zone = "us-east-1b"
}

# Internet Gateway
resource "aws_internet_gateway" "gw" {
  vpc_id = aws_vpc.main.id
}
```

```
# Public Route Table
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id
}

resource "aws_route" "default_route" {
  route_table_id         = aws_route_table.public_rt.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id             = aws_internet_gateway.gw.id
}

resource "aws_route_table_association" "public_assoc" {
  subnet_id      = aws_subnet.public_subnet.id
  route_table_id = aws_route_table.public_rt.id
}

# Security Groups
resource "aws_security_group" "web_sg" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_security_group" "rds_sg" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port       = 3306
    to_port         = 3306
    protocol        = "tcp"
    security_groups = [aws_security_group.web_sg.id]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# Allocate an Elastic IP for NAT Gateway
resource "aws_eip" "nat_eip" {
  domain = "vpc"
}

# NAT Gateway in Public Subnet
resource "aws_nat_gateway" "nat_gw" {
  allocation_id = aws_eip.nat_eip.id
  subnet_id     = aws_subnet.public_subnet.id
}

# Private Route Table
resource "aws_route_table" "private_rt" {
  vpc_id = aws_vpc.main.id
```

```
}

# Route for Private Subnet to Use NAT Gateway
resource "aws_route" "private_nat_route" {
  route_table_id         = aws_route_table.private_rt.id
  destination_cidr_block = "0.0.0.0/0"
  nat_gateway_id         = aws_nat_gateway.nat_gw.id
}

# Associate Private Subnet with Private Route Table
resource "aws_route_table_association" "private_assoc" {
  subnet_id      = aws_subnet.private_subnet.id
  route_table_id = aws_route_table.private_rt.id
}

# Launch Template
resource "aws_launch_template" "web_lt" {
  name_prefix   = "web-lt-"
  image_id      = "ami-12345678"
  instance_type = "t2.micro"
  key_name      = "my-key"
  network_interfaces {
    security_groups = [aws_security_group.web_sg.id]
  }
}

# Auto Scaling Group
resource "aws_autoscaling_group" "web_asg" {
  vpc_zone_identifier = [aws_subnet.public_subnet.id]
  desired_capacity    = 2
  min_size            = 1
  max_size            = 3
  launch_template {
    id      = aws_launch_template.web_lt.id
    version = "$Latest"
  }
  target_group_arns = [aws_lb_target_group.web_tg.arn]
}

# Application Load Balancer
resource "aws_lb" "web_alb" {
  name               = "web-alb"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [aws_security_group.web_sg.id]
  subnets            = [aws_subnet.public_subnet.id]
}

resource "aws_lb_target_group" "web_tg" {
  name     = "web-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.main.id
}

resource "aws_lb_listener" "web_listener" {
  load_balancer_arn = aws_lb.web_alb.arn
  port              = 80
  protocol          = "HTTP"
  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.web_tg.arn
  }
}
```

```
# RDS MySQL Database (Private)
resource "aws_db_instance" "web_db" {
  allocated_storage    = 20
  engine               = "mysql"
  instance_class       = "db.t3.micro"
  db_name              = "webappdb"
  username             = "admin"
  password             = "mypassword"
  publicly_accessible = false
  skip_final_snapshot = true
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
  db_subnet_group_name = aws_db_subnet_group.rds_subnet_group.name
}

resource "aws_db_subnet_group" "rds_subnet_group" {
  name       = "rds-subnet-group"
  subnet_ids = [aws_subnet.private_subnet.id]
}

# Single EC2 Instance
resource "aws_instance" "web_instance" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
  key_name      = "my-key"
  subnet_id     = aws_subnet.public_subnet.id
  security_groups = [aws_security_group.web_sg.id]
  tags = {
    Name = "SingleWebInstance"
  }
}

# Register Single EC2 Instance to ALB Target Group
resource "aws_lb_target_group_attachment" "single_instance_attach" {
  target_group_arn = aws_lb_target_group.web_tg.arn
  target_id        = aws_instance.web_instance.id
  port             = 80
}
```

8. How to create EKS using terraform?

My Answer: I haven't worked on such configurations but if required I can do it referring to official documentation.

This Terraform script:

1. **Creates a VPC** with two subnets.
2. **Sets up IAM roles** for the EKS cluster and worker nodes.
3. **Deploys an EKS cluster**.
4. **Creates a node group** with two EC2 instances.
5. **Configures kubectl to access the cluster**.

```
provider "aws" {
  region = "us-east-1"
}

# 🚀 Create VPC
resource "aws_vpc" "eks_vpc" {
  cidr_block = "10.0.0.0/16"
}

# 🚀 Create Public Subnets
resource "aws_subnet" "public_subnet_1" {
  vpc_id                  = aws_vpc.eks_vpc.id
  cidr_block              = "10.0.1.0/24"
  map_public_ip_on_launch = true
```

```
  availability_zone       = "us-east-1a"
}

resource "aws_subnet" "public_subnet_2" {
  vpc_id                  = aws_vpc.eks_vpc.id
  cidr_block              = "10.0.2.0/24"
  map_public_ip_on_launch = true
  availability_zone       = "us-east-1b"
}

# 🚀 Create IAM Role for EKS Cluster
resource "aws_iam_role" "eks_role" {
  name = "eks-cluster-role"

  assume_role_policy = jsonencode({
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "eks.amazonaws.com"
      }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "eks_policy" {
  role       = aws_iam_role.eks_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
}

# 🚀 Create EKS Cluster
resource "aws_eks_cluster" "my_eks" {
  name     = "my-cluster"
  role_arn = aws_iam_role.eks_role.arn

  vpc_config {
    subnet_ids = [aws_subnet.public_subnet_1.id, aws_subnet.public_subnet_2.id]
  }
}

# 🚀 IAM Role for Worker Nodes
resource "aws_iam_role" "eks_worker_role" {
  name = "eks-worker-role"

  assume_role_policy = jsonencode({
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "ec2.amazonaws.com"
      }
    }]
  })
}

resource "aws_iam_role_policy_attachment" "eks_worker_node_policy" {
  role       = aws_iam_role.eks_worker_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
}

# 🚀 Create EKS Node Group (Worker Nodes)
resource "aws_eks_node_group" "eks_nodes" {
  cluster_name    = aws_eks_cluster.my_eks.name
```

```
  node_group_name = "my-node-group"
  node_role_arn   = aws_iam_role.eks_worker_role.arn
  subnet_ids      = [aws_subnet.public_subnet_1.id, aws_subnet.public_subnet_2.id]

  scaling_config {
    desired_size = 2
    min_size     = 1
    max_size     = 3
  }

  instance_types = ["t3.medium"]
}

# 🚀 Output EKS Cluster Name
output "eks_cluster_name" {
  value = aws_eks_cluster.my_eks.name
}

# 🚀 Output EKS Cluster Endpoint
output "eks_cluster_endpoint" {
  value = aws_eks_cluster.my_eks.endpoint
}
```

NOTE:

- A **VPC** defines the network.
- **Subnets** divide the VPC into public and private sections.
- An **Internet Gateway** allows public subnet connectivity.
- **Route Tables** define how traffic moves within the VPC.
- **Security Groups** manage access to resources.
- **NAT Gateway**, which allows instances in the private subnet (such as your RDS instance) to access the internet securely while preventing inbound connections from external sources.